大模型训练与推理

课堂投影片影印本

Facsimile of On-site Lecture Slides · 96 pages

图片文件：	IMG_20261016_*.jpg（96 张原图未随仓库发布）
图片数量：	96 张（IMG_20261016_133219.jpg … IMG_20261016_162659.jpg）
拍摄日期：	2026-10-16
拍摄时段：	13:32:19 — 16:26:59
原件规格：	3264×2448 px 手机翻拍
PDF 生成：	2026-10-18 19:09
文本层：	每页附 OCR 隐形文本层，可全文检索（Ctrl+F）

说明：图片按拍摄时间顺序排列；页脚标注原始文件名、拍摄时刻与该页识别出的标题行。

图片索引

序号 / 拍摄时刻 / 该页识别标题

1	13:32:19	MegaMoE recap
2	13:33:23	MegaMoE support matrix
3	13:34:06	MegaMoE on Blackwell
4	13:35:11	MegaMoE on Blackwell
5	13:35:16	MegaMoE on Blackwell
6	13:35:41	MegaMoE on Blackwell
7	13:36:24	MegaMoE on Blackwell
8	13:37:02	From a 32x4 Trick to a Full TMEM Transpose
9	13:37:58	CuteDSL MegaMoE on Blackwell
10	13:39:19	CuteDSLMegaMoEonBlackwell
11	13:39:33	CuteDSL MegaMoE on Blackwell
12	13:40:13	MegaMoE on Blackwell
13	13:41:07	MegaMoEonBlackwell
14	13:41:43	MegaMoE on Blackwell
15	13:43:20	CuteDSL MegaMoE on Blackwell
16	13:44:21	MegaMoEonBlackwell
17	13:45:05	Overall Architecture: One Kernel Does Everything
18	13:46:14	Warp Specialization
19	13:47:42	The swapAB Variant: the Cure for Small Token Counts
20	13:49:13	Fp8 Quantization: per_tensor vs. blockwise
21	13:50:50	Optimization 1: Cooperative Warp-group Scheduling
22	13:52:16	Optimization 2: Task-tile Ping-pong
23	13:53:39	Optimization 3:Multi-CTA CGA +TMA Multicast
24	13:55:20	Optimization 4: Mainloop & Store Pipelines
25	13:56:54	Optimization 5: All-Register Epilogue Fusion Chain
26	13:58:15	Why Gate/Up Interleave = 8: the WGMMA Register Layout
27	14:00:17	Optimization 6: A Single Active Dispatch Warp
28	14:00:37	Optimization 7: Folding the Producer Warpgroup + Register
29	14:01:52	Optimization 8: Early fc1_done Publish
30	14:03:22	Optimization9:BlockwiseFC2Dual-Accumulator Overlap
31	14:05:46	Optimization 10: Scheduler Groups and the fc1_done Spin
32	14:08:02	Optimization 11: 16-byte FC2 Output Stores
33	14:09:40	Token Sweep Results
34	14:11:07	Summary

图片索引（续）

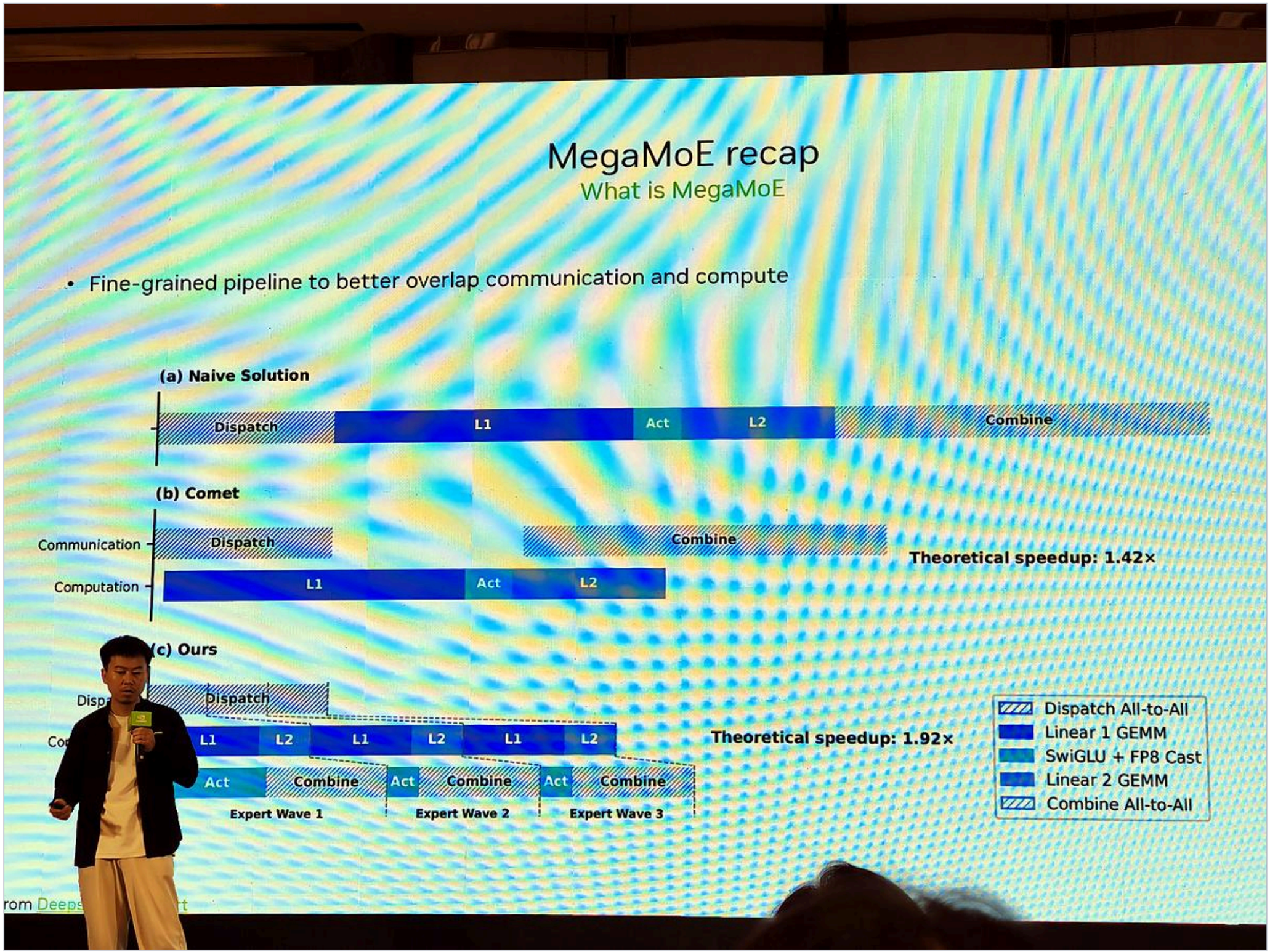
序号 / 拍摄时刻 / 该页识别标题

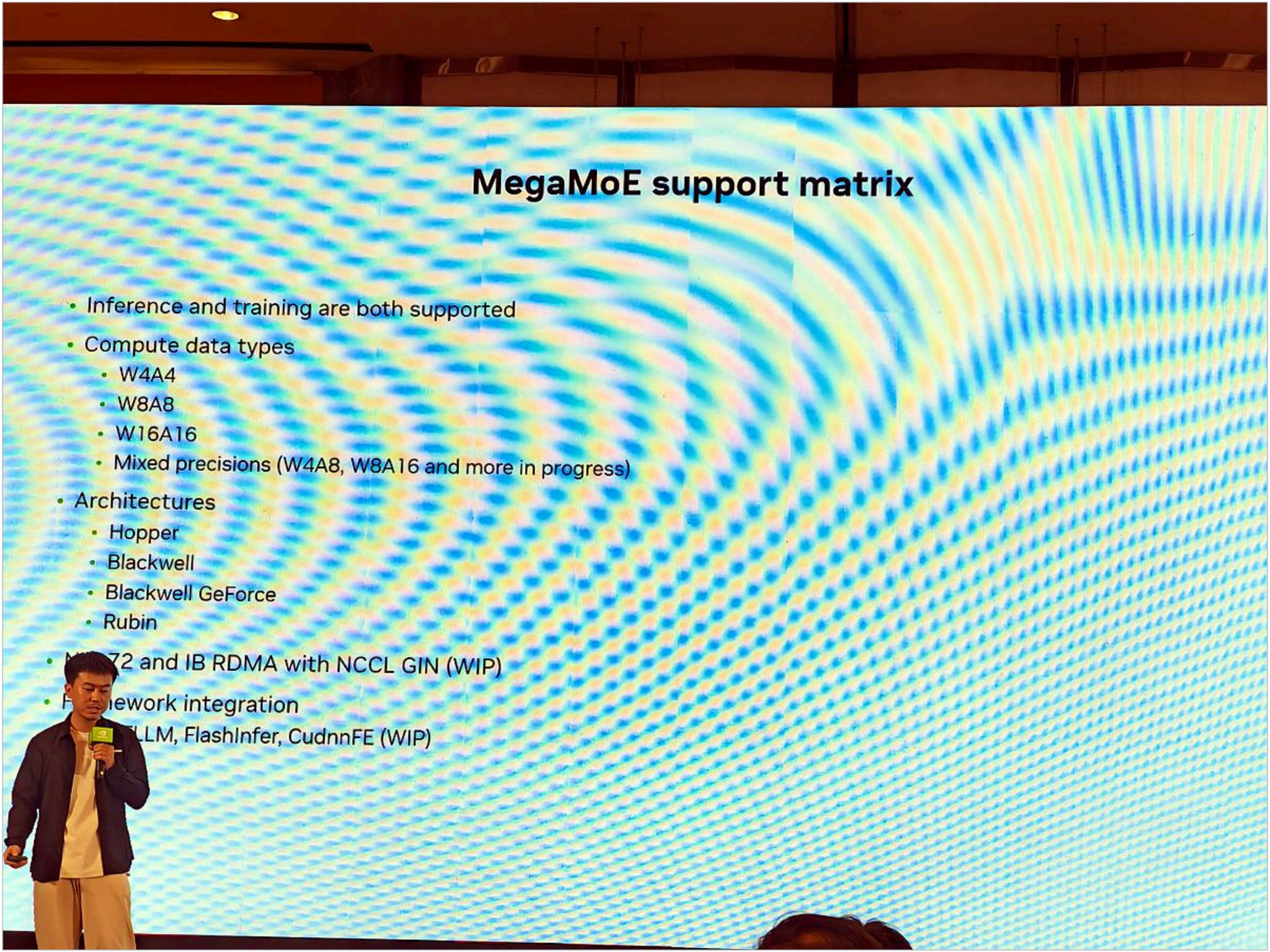
35	14:13:18	Two Challenges Define Training at Scale
36	14:15:51	One Playbook, Two Workloads
37	14:16:47	DeepSeek-V4-Pro Overview
38	14:19:29	Model Features Drive Optimizations
39	14:20:05	Choose the Parallel Layout
40	14:20:38	Choose the Parallel Layout
41	14:24:09	Context Length Drives Memory Policy
42	14:25:09	Context Length Drives Memory Policy
43	14:26:18	The Starting Points
44	14:27:35	STEP 1: Runtime Overhead and Memory Trade-offs
45	14:32:15	WheretheStepTime Goes
46	14:33:36	STEP 2:Optimize the Hybrid Attention
47	14:37:43	STEP 2: Optimize the Hybrid Attention
48	14:38:26	STEP 3:Optimize the MoE
49	14:41:59	STEP 3: Optimize the MoE
50	14:43:38	STEP 4:Balance CP Workloads
51	14:44:55	STEP4I Balance CP Workloads
52	14:49:28	Lessons for Scalable Training
53	14:58:41	Motivation: KV Cache as a Serving Bottleneck
54	14:59:26	KV Cache Is a Serving Bottleneck
55	15:03:08	KV Cache Compression Methods
56	15:04:39	Where KV Compression Fits
57	15:07:42	TurboQuant: Online KV Quantization
58	15:10:01	TurboQuant: Layout Compressed KV for Decode
59	15:11:54	TurboQuant: Tiled MMA Decode Attention on 4-bit KV
60	15:15:29	TurboQuant:Decode Performance
61	15:17:21	KVTC: Compression for Storage and Transfer
62	15:20:28	KVTC: Integration and Pipelined Decompression
63	15:23:01	KVTC: Single-Request TTFT
64	15:23:45	KVTC: Throughput Under Memory Pressure
65	15:25:32	nvCOMP-ANS: Lossless KV Cache Transfer
66	15:27:40	nvCOMP-ANS: Compression and Serving Results TTFTgains ar...
67	15:29:43	Takeaways
68	15:45:57	What Dynamo can do today

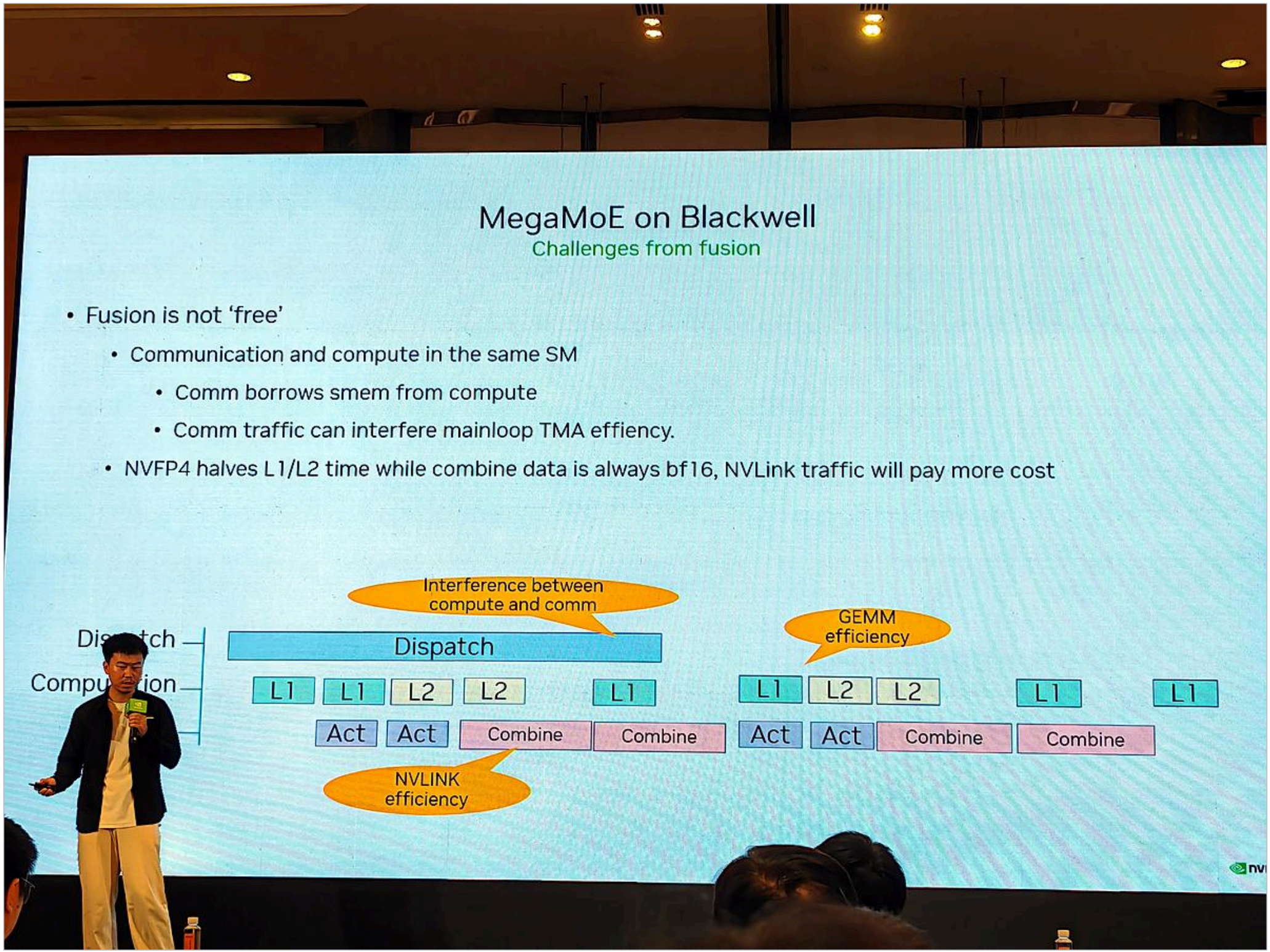
图片索引（续）

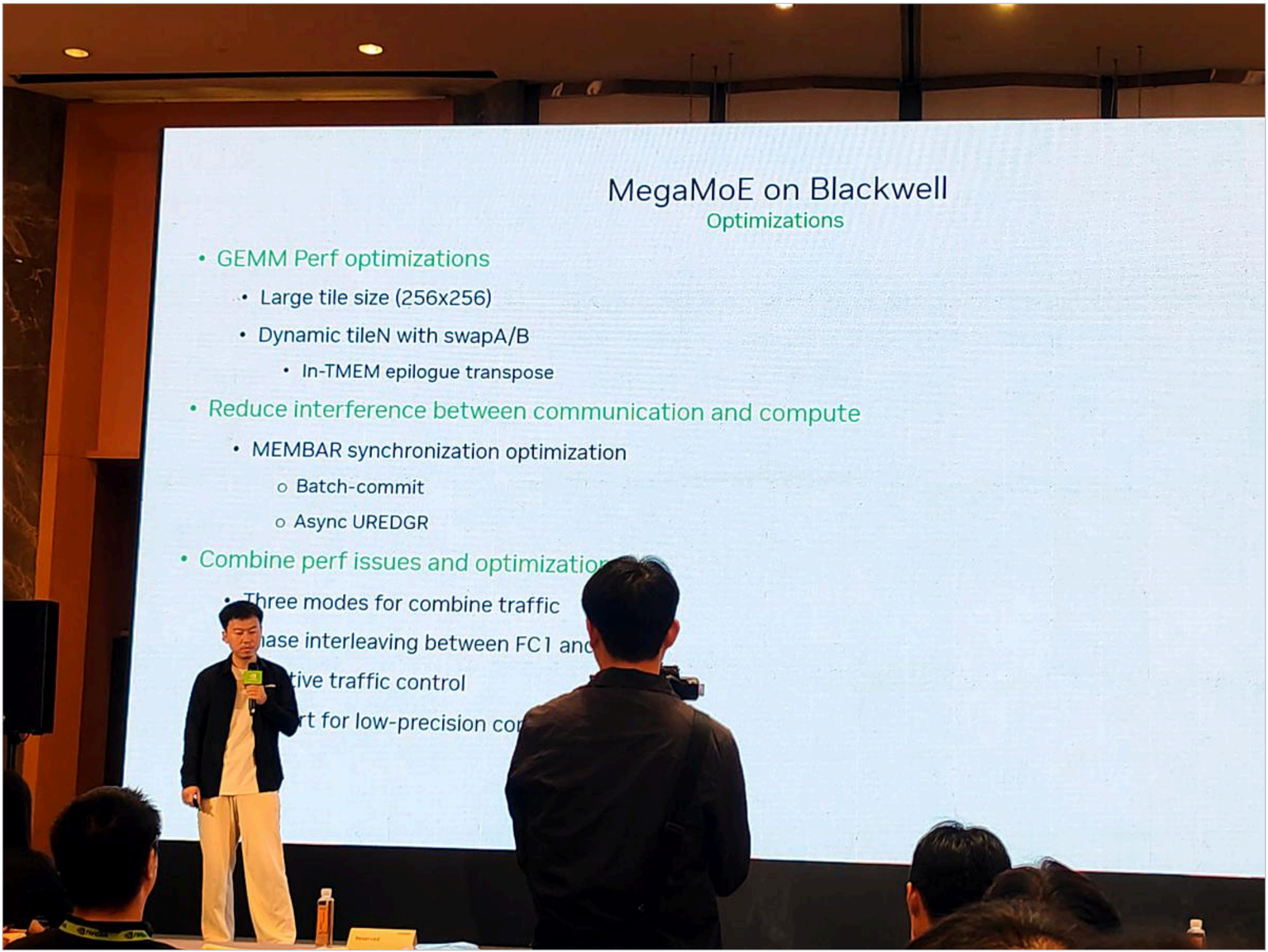
序号 / 拍摄时刻 / 该页识别标题

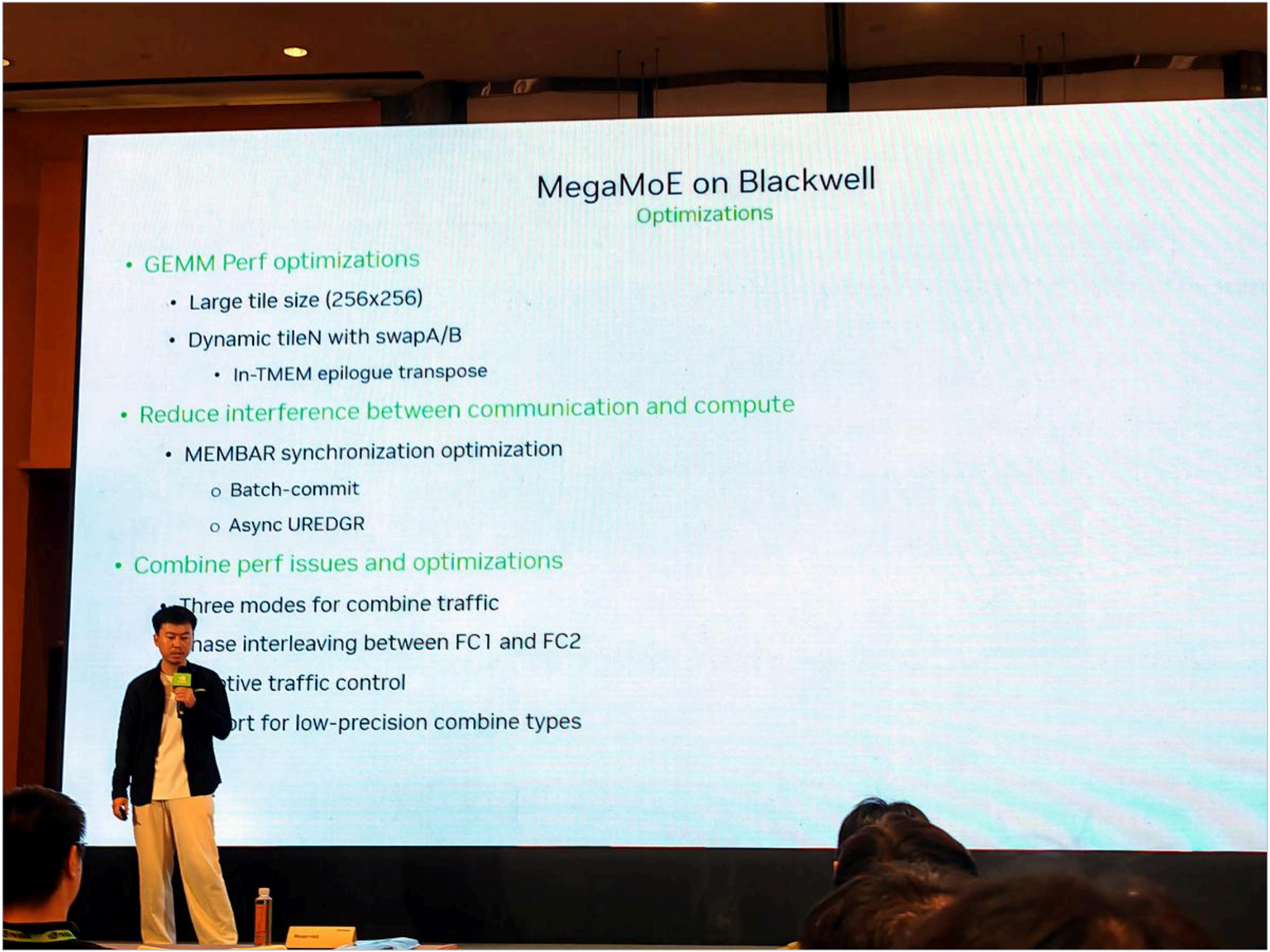
69	15:49:10	KVRouter在生产环境中被广泛验证
70	15:49:29	请求进入后，KVRouter用两类信号为Worker打分
71	15:50:23	Flash Indexer 在链路里做什么
72	15:51:12	索引里存的是一张“位置到worker"的地图
73	15:52:09	GEN01/06 THROUGHPUT
74	15:53:04	GEN04/06
75	15:53:41	GEN06/06 THROUGHPUT
76	15:54:30	性能数据
77	15:56:14	Standalone KV Indexer: 定位
78	15:56:53	Standalone KV Indexer:架构
79	15:58:12	SLAs of New Agentic Era
80	16:00:11	AgentX Trace
81	16:01:22	Single AgentX Session Characteristics
82	16:03:15	3 Layer Software Stack of Agentic Inference
83	16:05:11	AgentX Dashboard
84	16:06:27	Observability - Dynamo Provider Plugin
85	16:07:40	How many tokens fit in 1 GB
86	16:10:18	Goal 1: actual KV hit rate theoretical max
87	16:13:59	KV capacity for diff session cc
88	16:16:25	Goal 2: decode concurrency must keep up
89	16:18:21	Hard affinity mode DEP 下的负载均衡问题
90	16:19:43	Goal3: Agentic Routing Strategy
91	16:22:06	Goal 4 - Clever Session Gateway - Thunder Agent
92	16:23:00	ThunderAgent · how it decides
93	16:24:12	Agentic Serving ThunderAgent Benchmarks
94	16:24:54	ThunderAgent + KV offload
95	16:26:04	Thunderagent in RL rollout - Dynamo VeRL Integration
96	16:26:59	Assets & Future Work

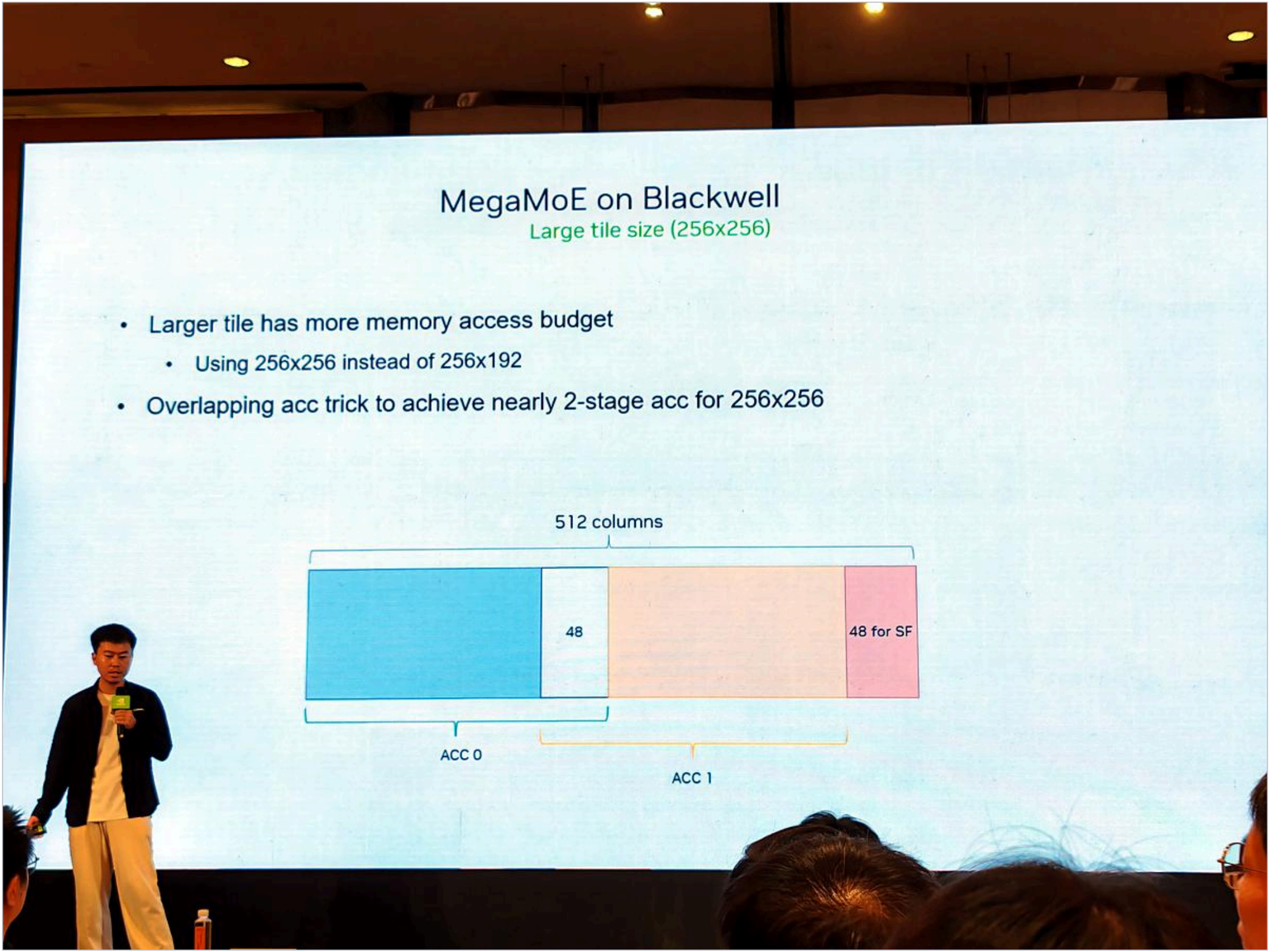

MegaMoE recap
What is MegaMoE
• Fine-grained pipeline to better overlap communication and compute
(a) Naive Solution
Dispatch
L1
Act
L2
Combine
(b) Comet
Communication
Computation
Dispatch
Combine
L1
Act
L2
Theoretical speedup: 1.42×
(c) Ours
Dispatch
L1
L2
Act
Combine
Expert Wave 1
Expert Wave 2
Expert Wave 3
Theoretical speedup: 1.92×
Dispatch All-to-All
Linear 1 GEMM
SwiGLU + FP8 Cast
Linear 2 GEMM
Combine All-to-All

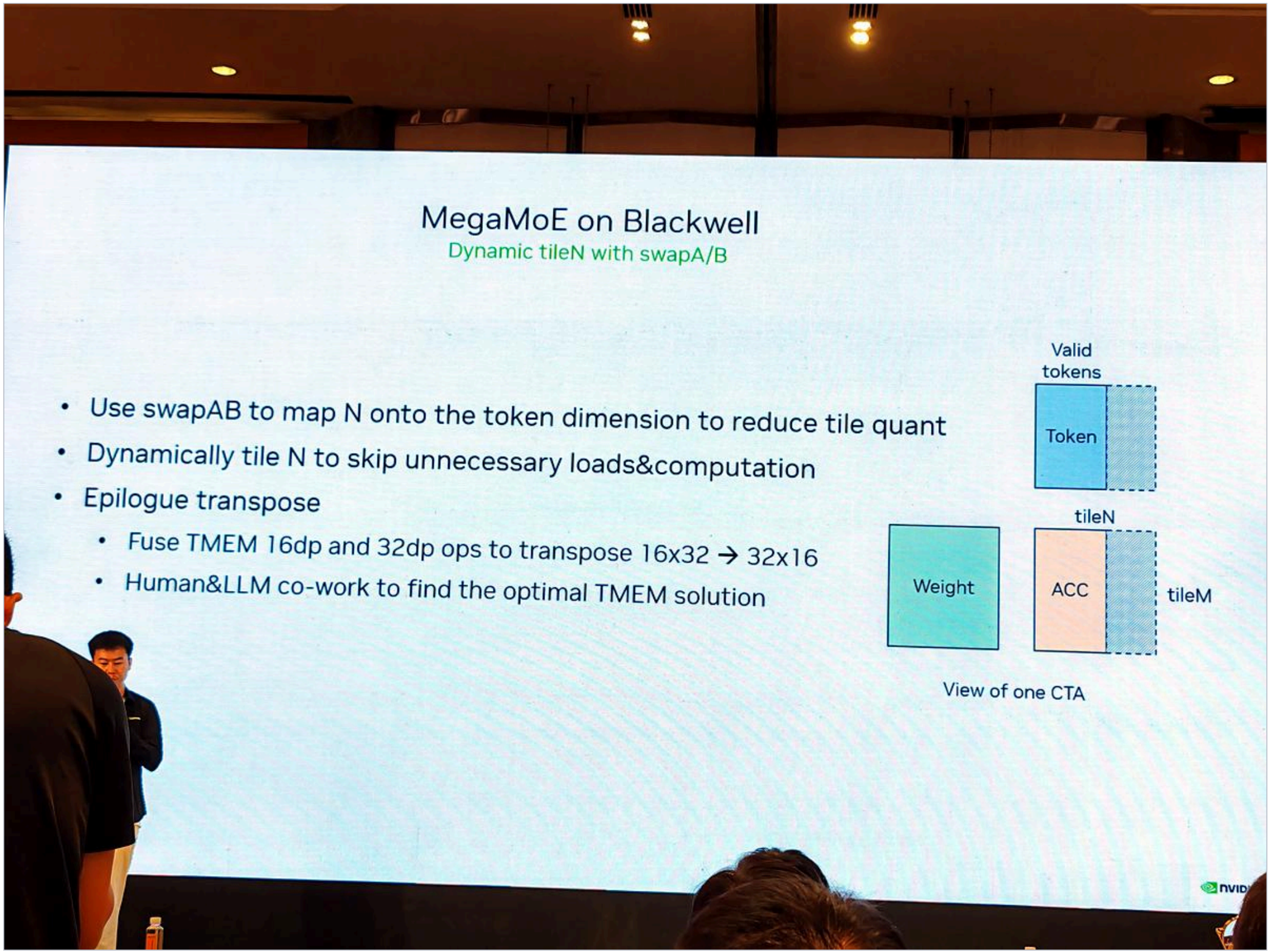

MegaMoE support matrix
• Inference and training are both supported
• Compute data types
• W4A4
• W8A8
• W16A16
• Mixed precisions (W4A8, W8A16 and more in progress)
• Architectures
• Hopper
• Blackwell
• Blackwell GeForce
• Rubin
• Z2 and IB RDMA with NCCL GIN (WIP)
• ework integration
LLM, FlashInfer, CudnnFE (WIP)

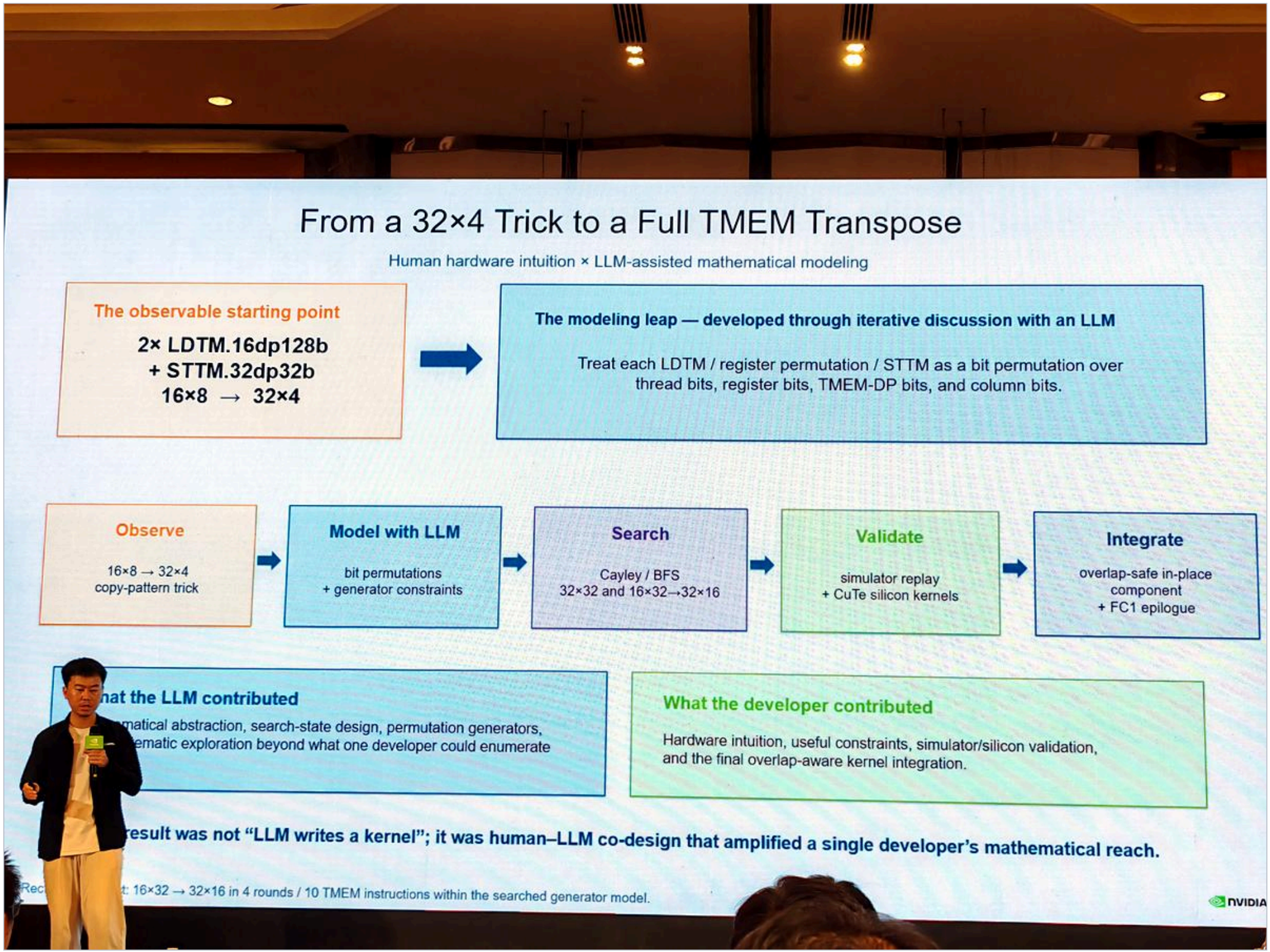

MegaMoE on Blackwell
Challenges from fusion
• Fusion is not 'free'
• Communication and compute in the same SM
• Comm borrows smem from compute
• Comm traffic can interfere mainloop TMA effiency.
• NVFP4 halves L1/L2 time while combine data is always bf16, NVLink traffic will pay more cost
Interference between compute and comm
GEMM efficiency
Dispatch
L1 L1 L2 L2 L1
Act Act Combine Combine
L1 L2 L2 L1 L1
Act Act Combine Combine
NVLINK efficiency

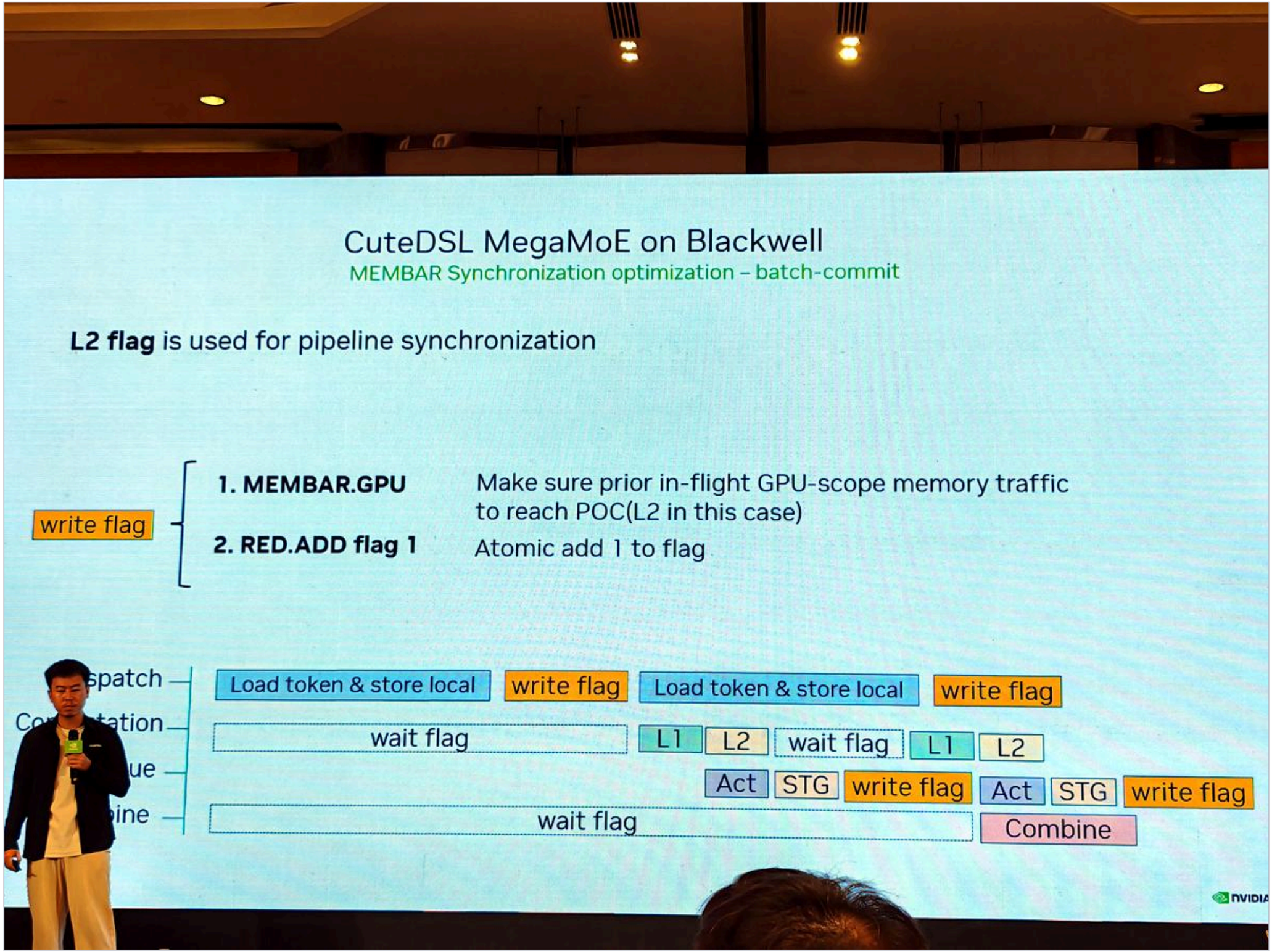

MegaMoE on Blackwell
Optimizations
• GEMM Perf optimizations
• Large tile size (256x256)
• Dynamic tileN with swapA/B
• In-TMEM epilogue transpose
• Reduce interference between communication and compute
• MEMBAR synchronization optimization
o Batch-commit
o Async UREDGR
• Combine perf issues and optimizatio
• hree modes for combine traffic
ase interleaving between FC1 an
tive traffic control
rt for low-precision co

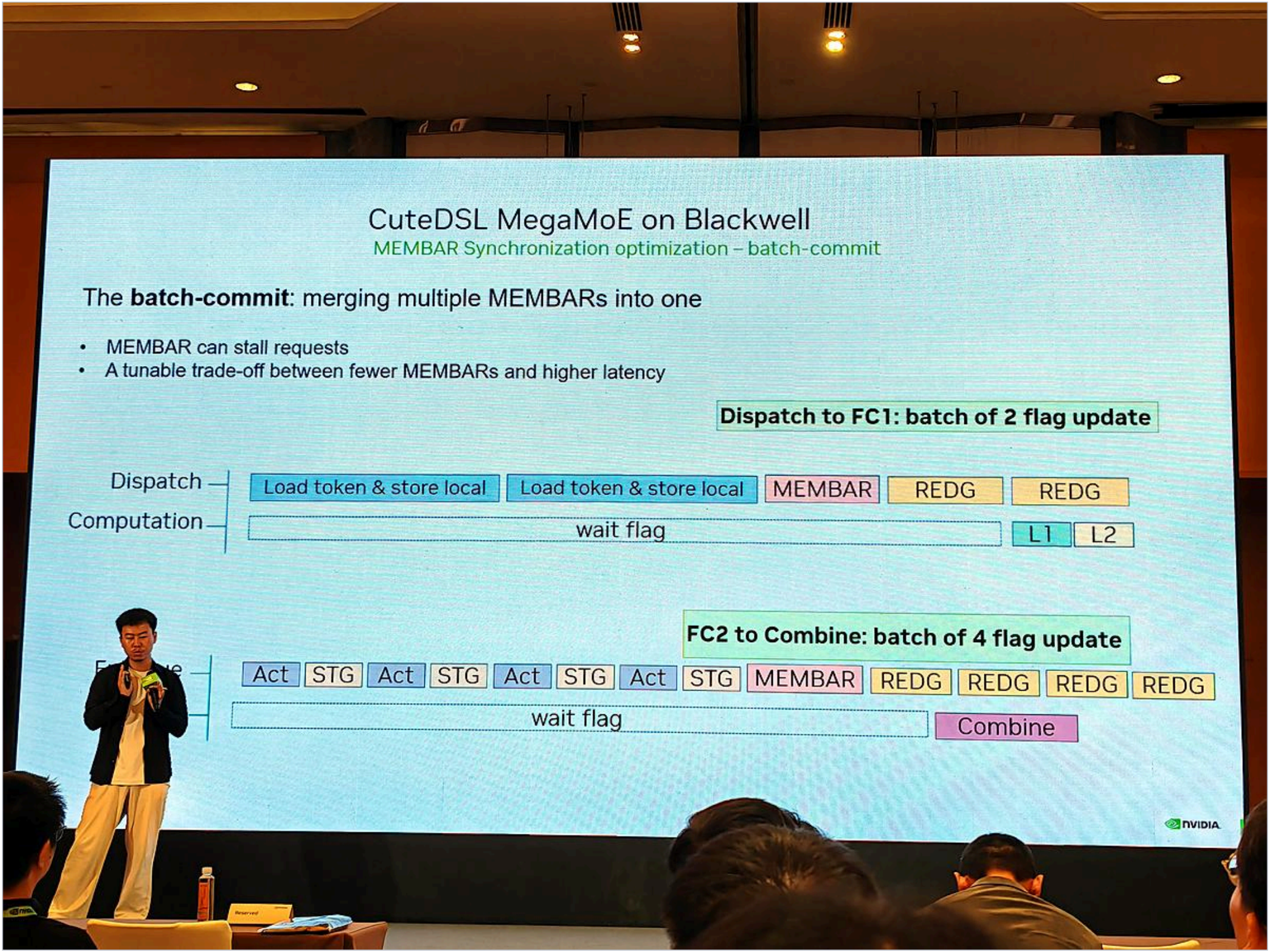

MegaMoE on Blackwell
Optimizations
• GEMM Perf optimizations
• Large tile size (256x256)
• Dynamic tileN with swapA/B
• In-TMEM epilogue transpose
• Reduce interference between communication and compute
• MEMBAR synchronization optimization
o Batch-commit
o Async UREDGR
• Combine perf issues and optimizations
• Three modes for combine traffic
hase interleaving between FC1 and FC2
tive traffic control
ort for low-precision combine types

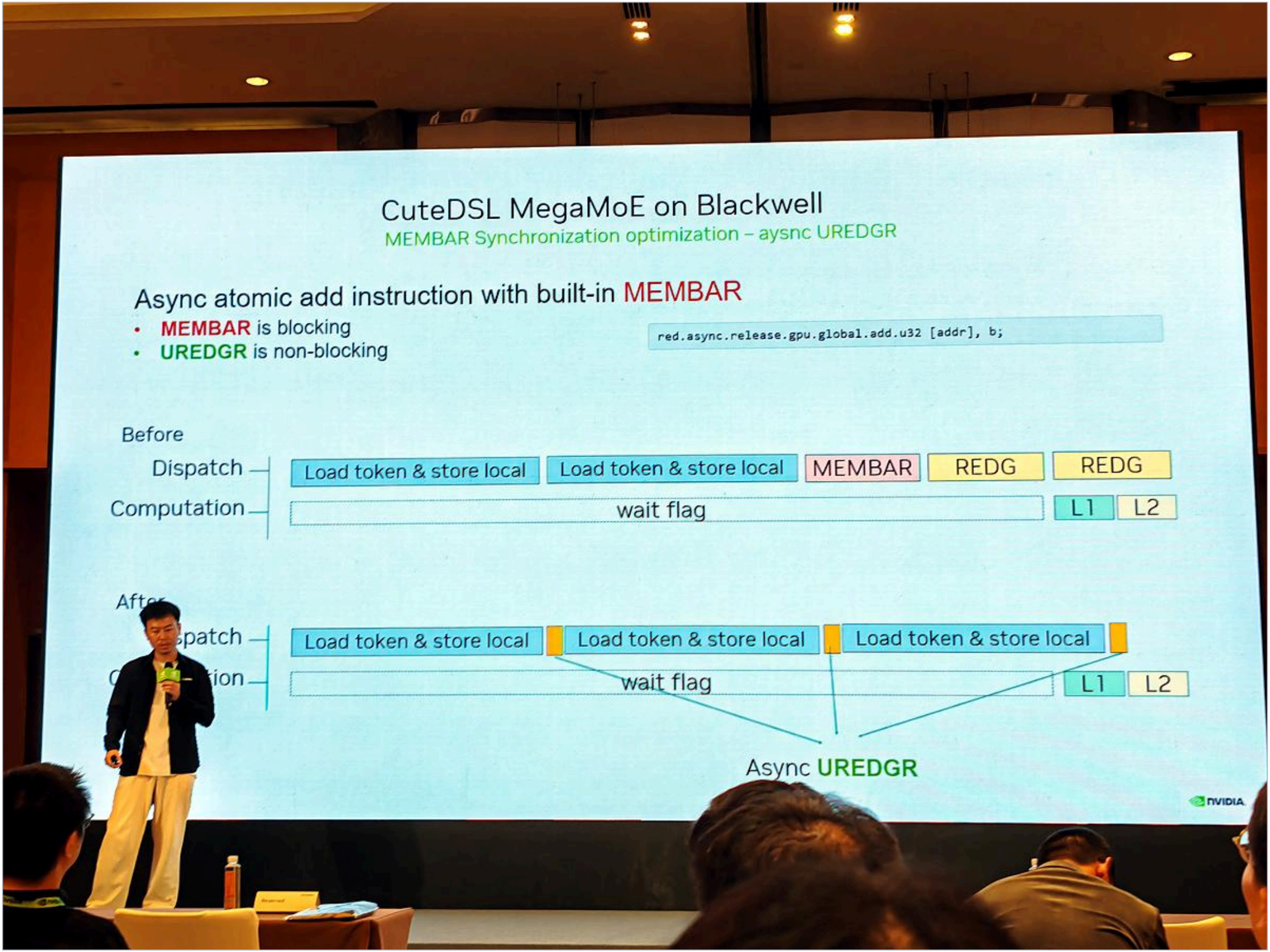

MegaMoE on Blackwell
Large tile size (256x256)
• Larger tile has more memory access budget
• Using 256x256 instead of 256x192
• Overlapping acc trick to achieve nearly 2-stage acc for 256x256
512 columns
48
48 for SF
ACC 0
ACC 1

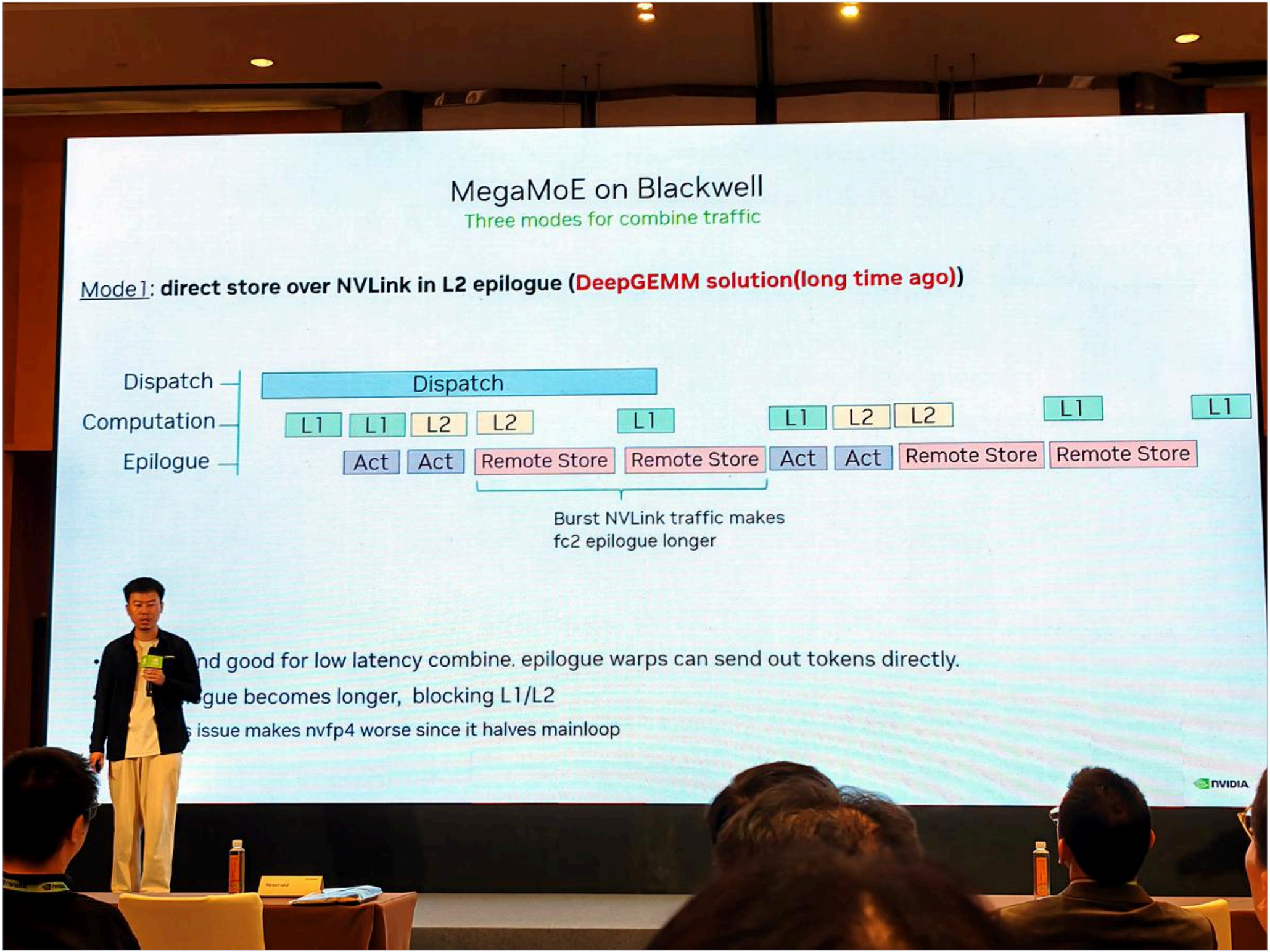

MegaMoE on Blackwell
Dynamic tileN with swapA/B
• Use swapAB to map N onto the token dimension to reduce tile quant
• Dynamically tile N to skip unnecessary loads&computation
• Epilogue transpose
• Fuse TMEM 16dp and 32dp ops to transpose 16x32 → 32x16
• Human&LLM co-work to find the optimal TMEM solution
Valid tokens
Token
tileN
Weight
ACC
tileM
View of one CTA

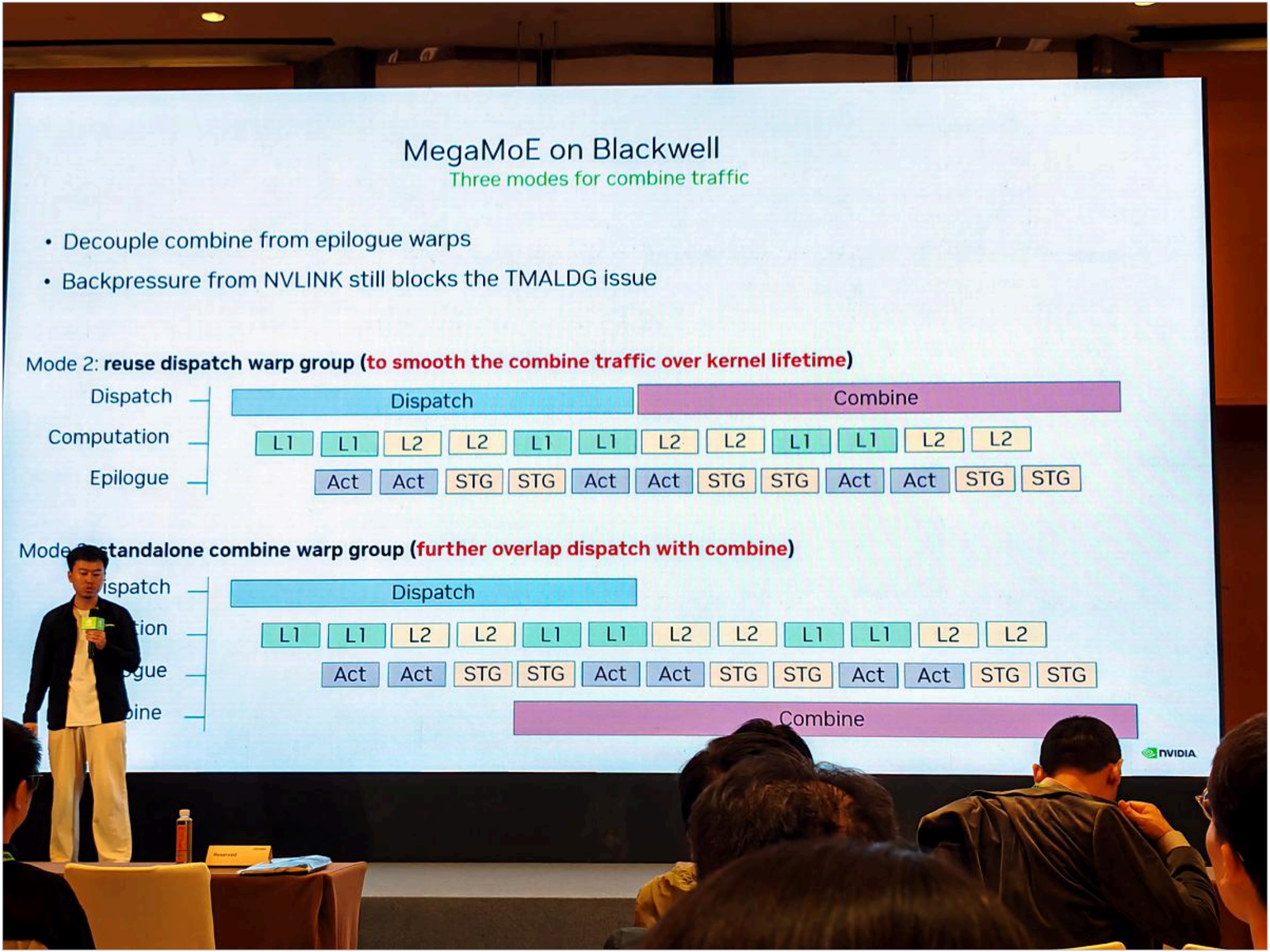

From a 32×4 Trick to a Full TMEM Transpose
Human hardware intuition × LLM-assisted mathematical modeling
The observable starting point
2× LDTM.16dp128b
+ STTM.32dp32b
16×8 → 32×4
The modeling leap — developed through iterative discussion with an LLM
Treat each LDTM / register permutation / STTM as a bit permutation over thread bits, register bits, TMEM-DP bits, and column bits.
Observe
16×8 → 32×4
copy-pattern trick
Model with LLM
bit permutations
+ generator constraints
Search
Cayley / BFS
32×32 and 16×32→32×16
Validate
simulator replay
+ CuTe silicon kernels
Integrate
overlap-safe in-place component
+ FC1 epilogue
hat the LLM contributed
matical abstraction, search-state design, permutation generators,
ematic exploration beyond what one developer could enumerate
What the developer contributed
Hardware intuition, useful constraints, simulator/silicon validation, and the final overlap-aware kernel integration.
result was not “LLM writes a kernel”; it was human–LLM co-design that amplified a single developer’s mathematical reach.
t: 16×32 → 32×16 in 4 rounds / 10 TMEM instructions within the searched generator model.
NVIDIA

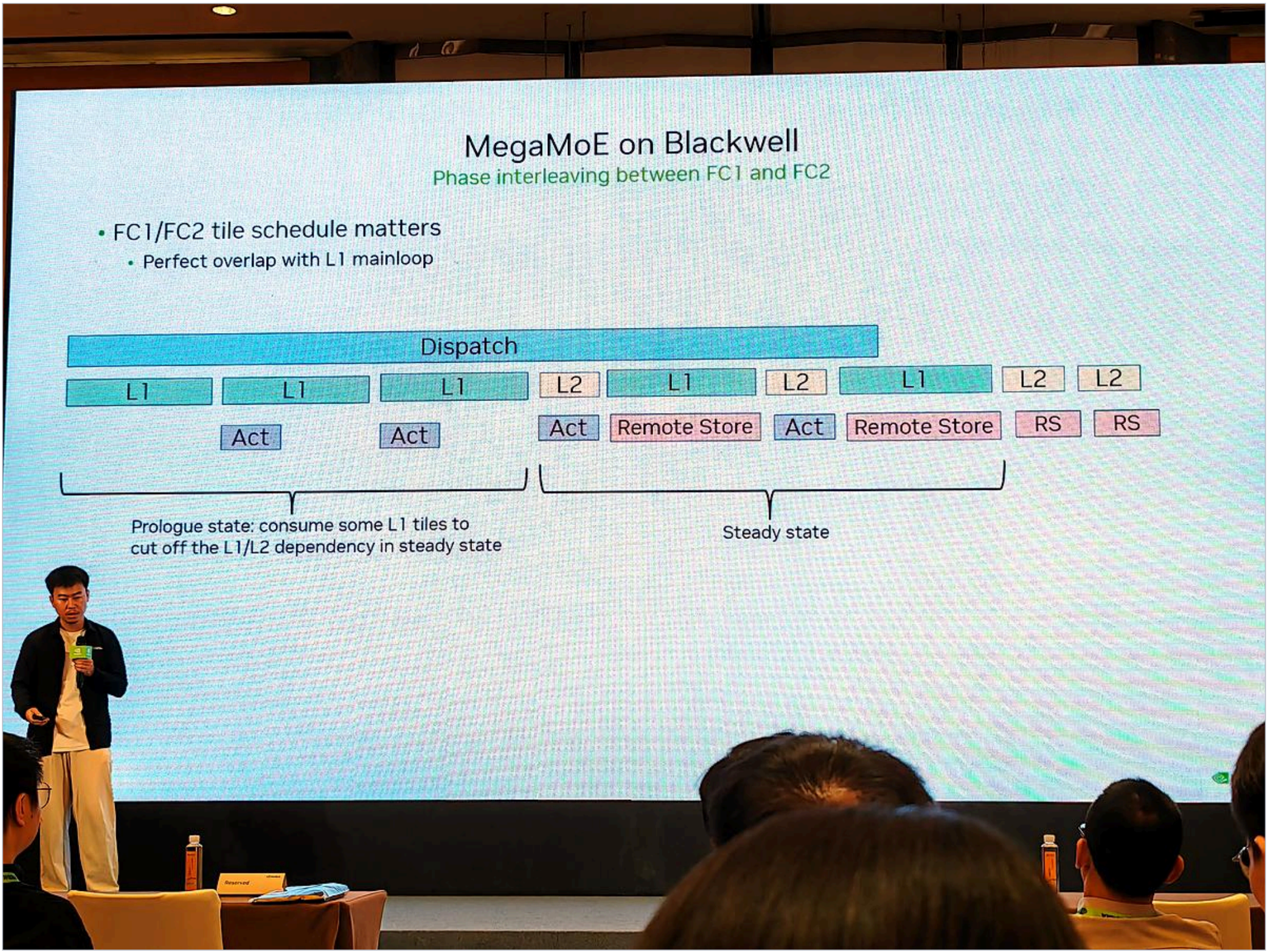

CuteDSL MegaMoE on Blackwell
MEMBAR Synchronization optimization – batch-commit
L2 flag is used for pipeline synchronization
write flag
1. MEMBAR.GPU
Make sure prior in-flight GPU-scope memory traffic to reach POC(L2 in this case)
2. RED.ADD flag 1
Atomic add 1 to flag
spatch
tation
ue
ine
Load token & store local
write flag
Load token & store local
write flag
wait flag
L1
L2
wait flag
L1
L2
Act
STG
write flag
Act
STG
write flag
wait flag
Combine

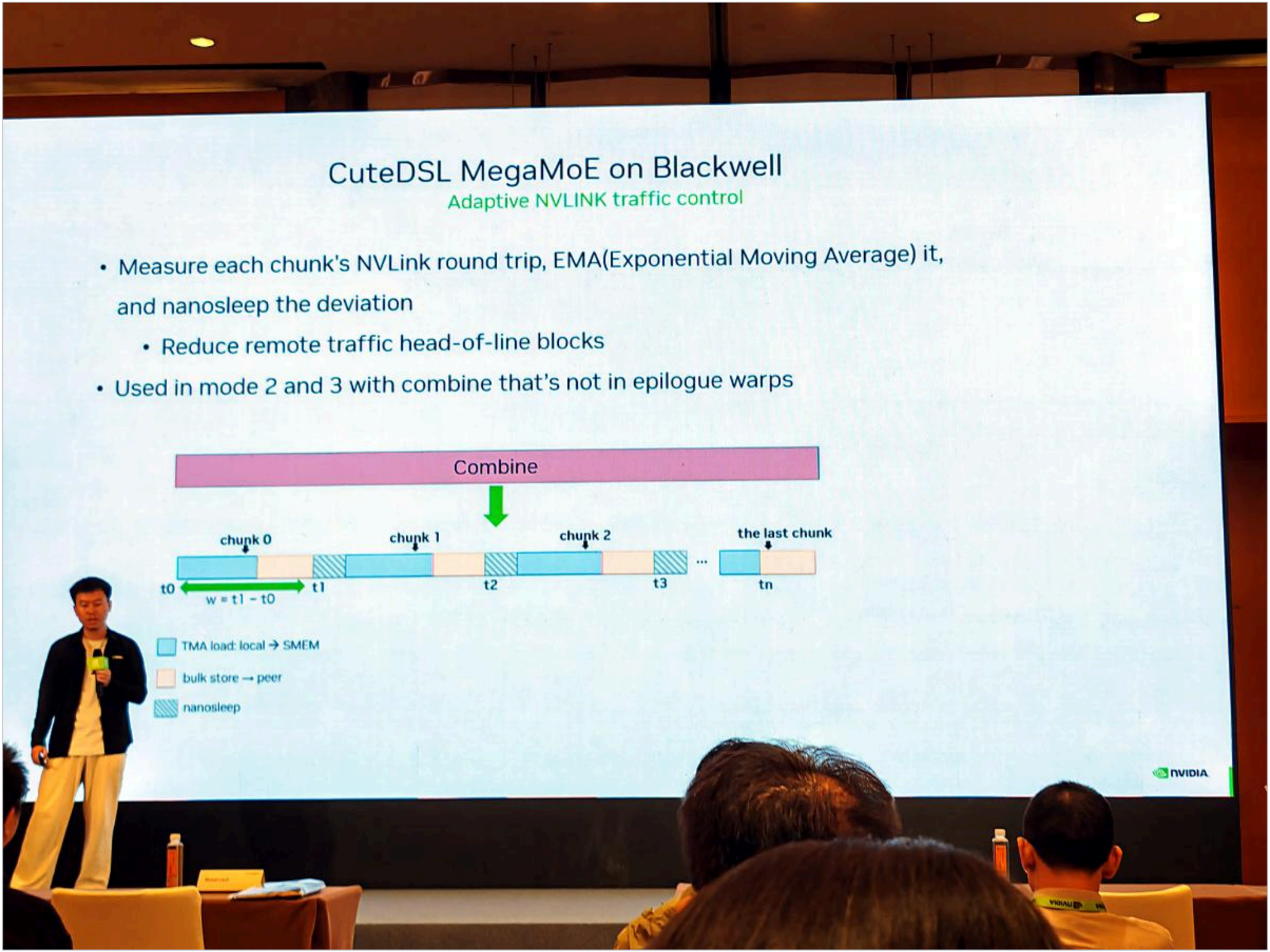

CuteDSL MegaMoE on Blackwell
MEMBAR Synchronization optimization – batch-commit
The batch-commit: merging multiple MEMBARs into one
• MEMBAR can stall requests
• A tunable trade-off between fewer MEMBARs and higher latency
Dispatch to FC1: batch of 2 flag update
Dispatch
Load token & store local
Load token & store local
MEMBAR
REDG
REDG
Computation
wait flag
L1
L2
FC2 to Combine: batch of 4 flag update
Act
STG
Act
STG
Act
STG
Act
STG
MEMBAR
REDG
REDG
REDG
REDG
wait flag
Combine
NVIDIA

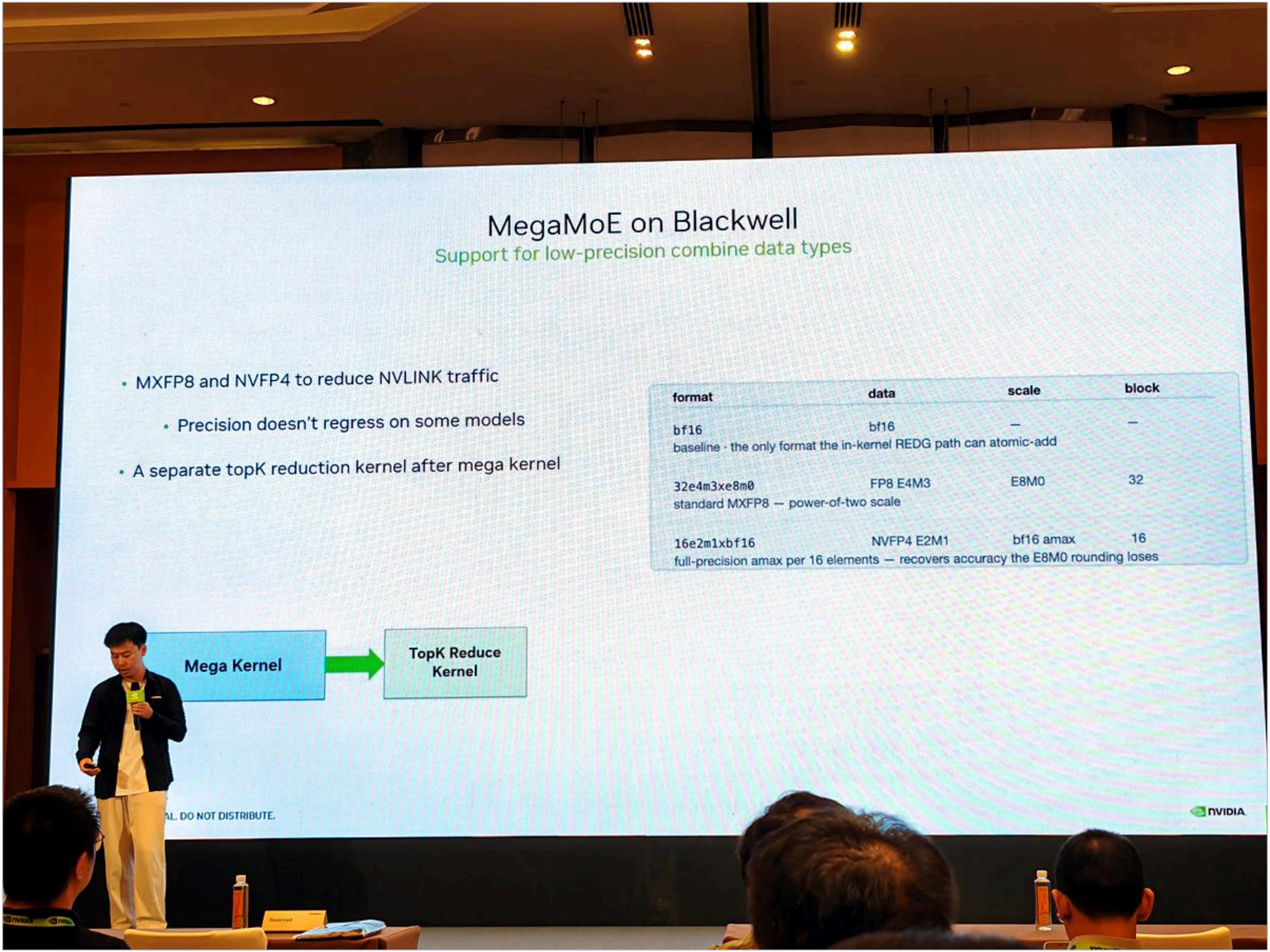

CuteDSL MegaMoE on Blackwell
MEMBAR Synchronization optimization – aysnc UREDGR
Async atomic add instruction with built-in MEMBAR
MEMBAR is blocking
UREDGR is non-blocking
red.async.release.gpu.global.add.u32 [addr], b;
Before
Dispatch
Computation
Load token & store local
Load token & store local
MEMBAR
REDG
REDG
wait flag
L1
L2
Load token & store local
Load token & store local
Load token & store local
wait flag
L1
L2
Async UREDGR
NVIDIA

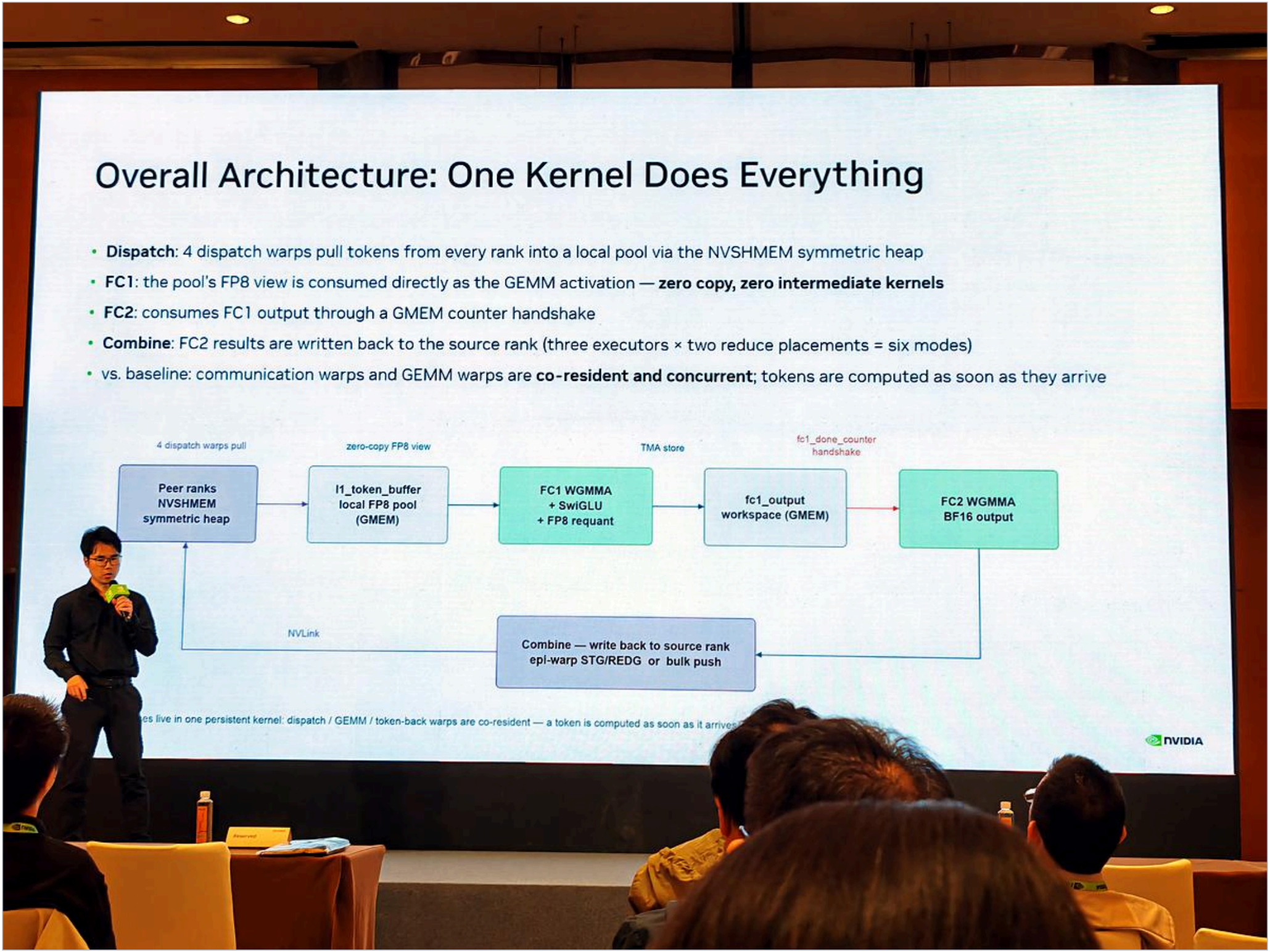

MegaMoE on Blackwell
Three modes for combine traffic
Mode1: direct store over NVLink in L2 epilogue (DeepGEMM solution(long time ago))
Dispatch
Computation
Epilogue
Dispatch
L1 L1 L2 L2 L1 L1 L2 L2 L1 L1
Act Act Remote Store Remote Store Act Act Remote Store Remote Store
Burst NVLink traffic makes
fc2 epilogue longer
nd good for low latency combine. epilogue warps can send out tokens directly.
gue becomes longer, blocking L1/L2
issue makes nvfp4 worse since it halves mainloop
NVIDIA

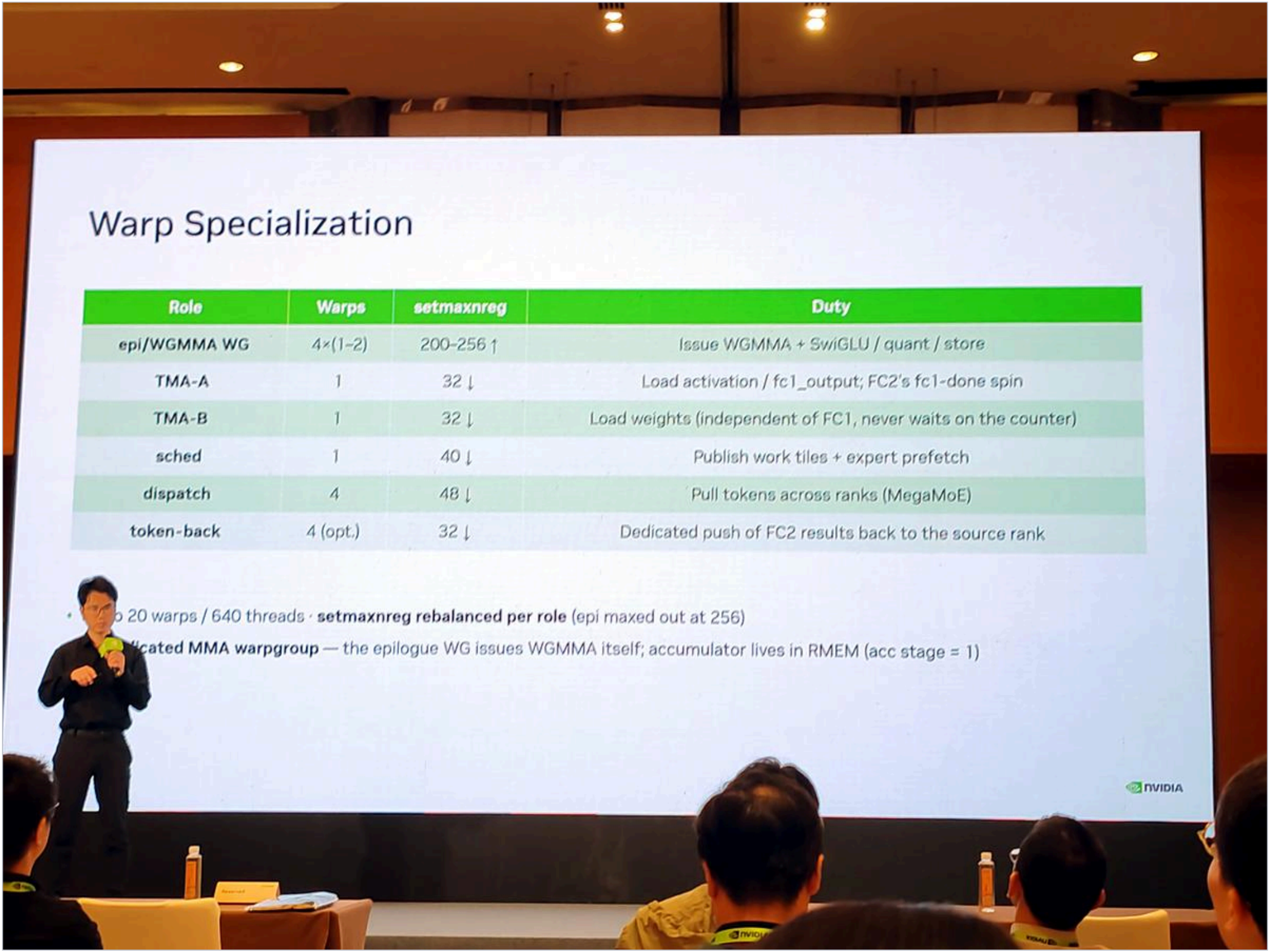

MegaMoE on Blackwell
Three modes for combine traffic
• Decouple combine from epilogue warps
• Backpressure from NVLINK still blocks the TMALDG issue
Mode 2: reuse dispatch warp group (to smooth the combine traffic over kernel lifetime)
Dispatch
Computation
Epilogue
Dispatch
Combine
L1 L1 L2 L2 L1 L1 L2 L2 L1 L1 L2 L2
Act Act STG STG Act Act STG STG Act Act STG STG
standalone combine warp group (further overlap dispatch with combine)
Dispatch
Combine
NVIDIA

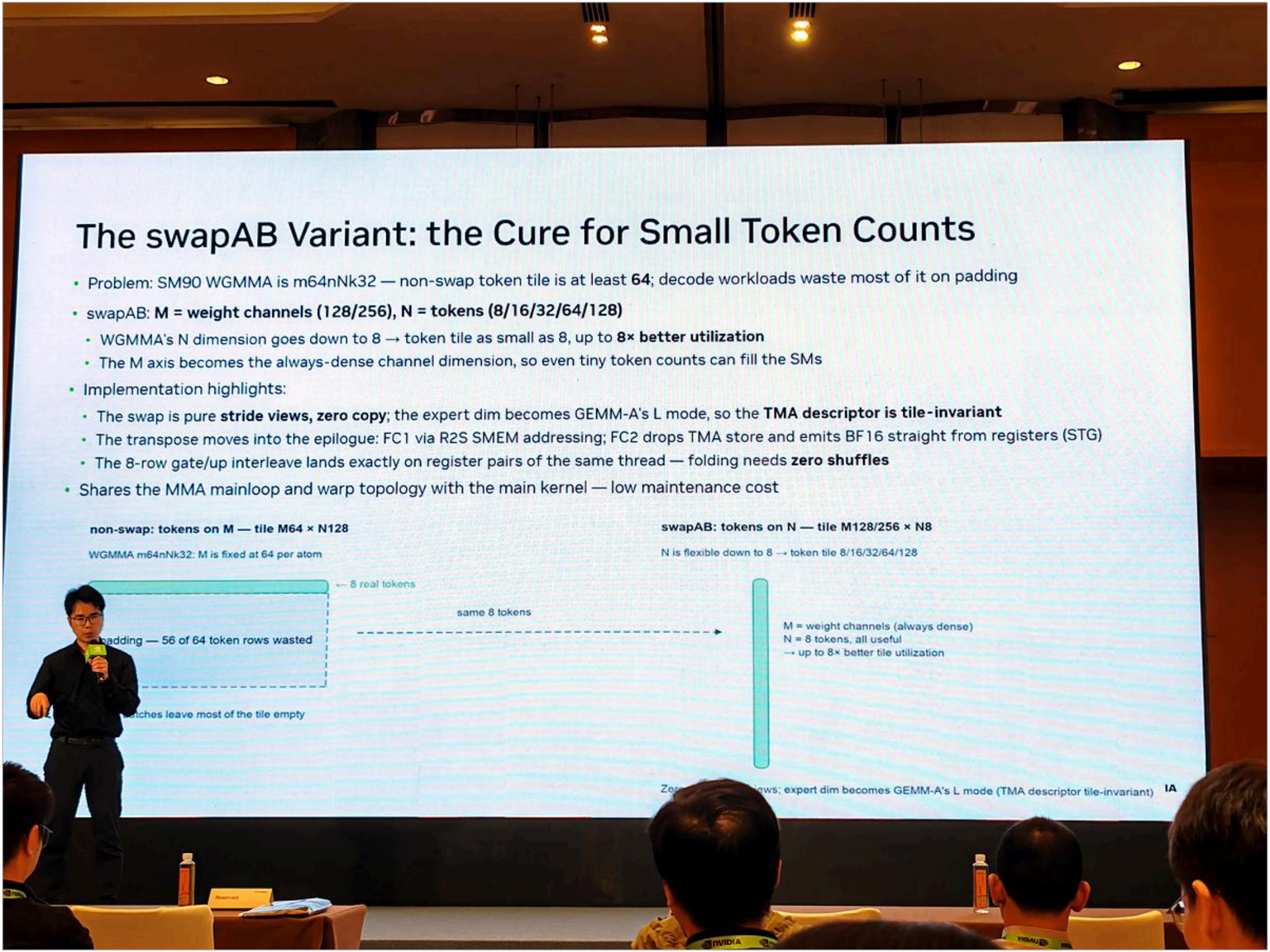

MegaMoE on Blackwell
Phase interleaving between FC1 and FC2
• FC1/FC2 tile schedule matters
• Perfect overlap with L1 mainloop
Dispatch
L1
L1
L1
L2
L1
L2
L1
L2
L2
Act
Act
Act
Remote Store
Act
Remote Store
RS
RS
Prologue state: consume some L1 tiles to cut off the L1/L2 dependency in steady state
Steady state

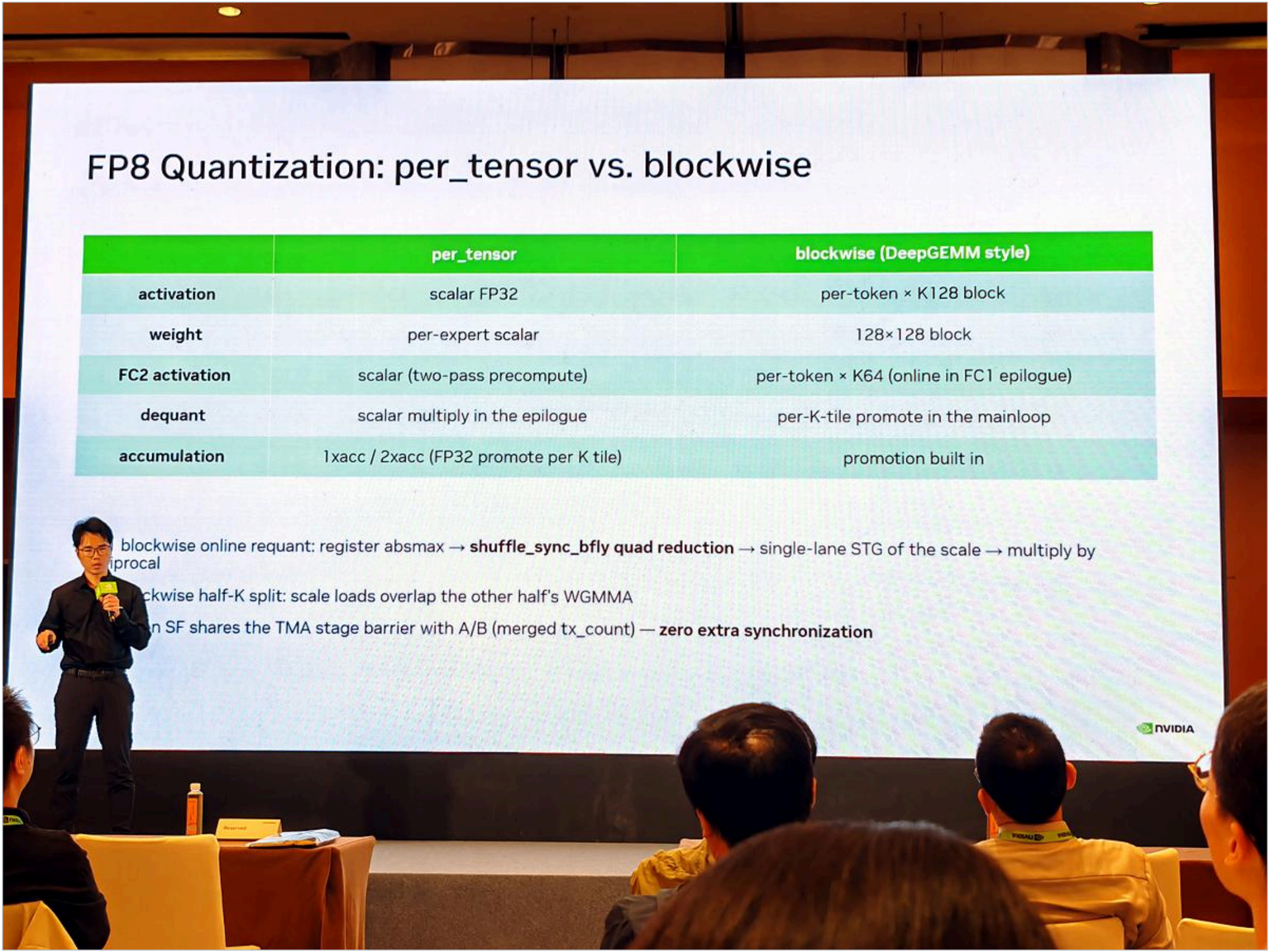

CuteDSL MegaMoE on Blackwell
Adaptive NVLINK traffic control
• Measure each chunk's NVLink round trip, EMA(Exponential Moving Average) it, and nanosleep the deviation
• Reduce remote traffic head-of-line blocks
• Used in mode 2 and 3 with combine that's not in epilogue warps
Combine
chunk 0
chunk 1
chunk 2
the last chunk
t0
t1
t2
t3
tn
w = t1 - t0
TMA load: local → SMEM
bulk store → peer
nanosleep
NVIDIA

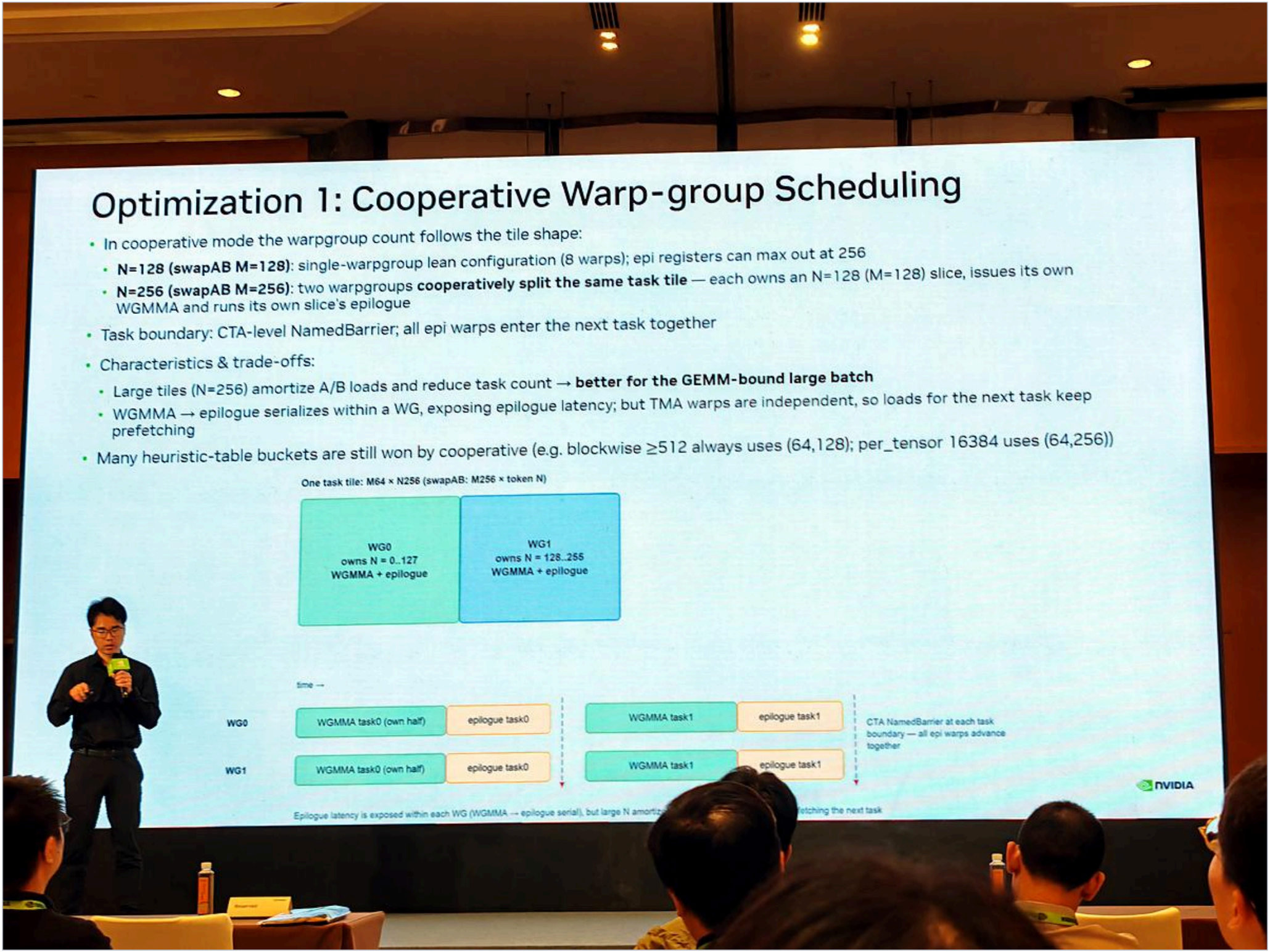

MegaMoE on Blackwell
Support for low-precision combine data types
• MXFP8 and NVFP4 to reduce NVLINK traffic
• Precision doesn't regress on some models
• A separate topK reduction kernel after mega kernel
format data scale block
bf16 bf16 — —
baseline · the only format the in-kernel REDG path can atomic-add
32e4m3xe8m0 FP8 E4M3 E8M0 32
standard MXFP8 — power-of-two scale
16e2m1xbf16 NVFP4 E2M1 bf16 amax 16
full-precision amax per 16 elements — recovers accuracy the E8M0 rounding loses
Mega Kernel
TopK Reduce Kernel
AL. DO NOT DISTRIBUTE.
NVIDIA

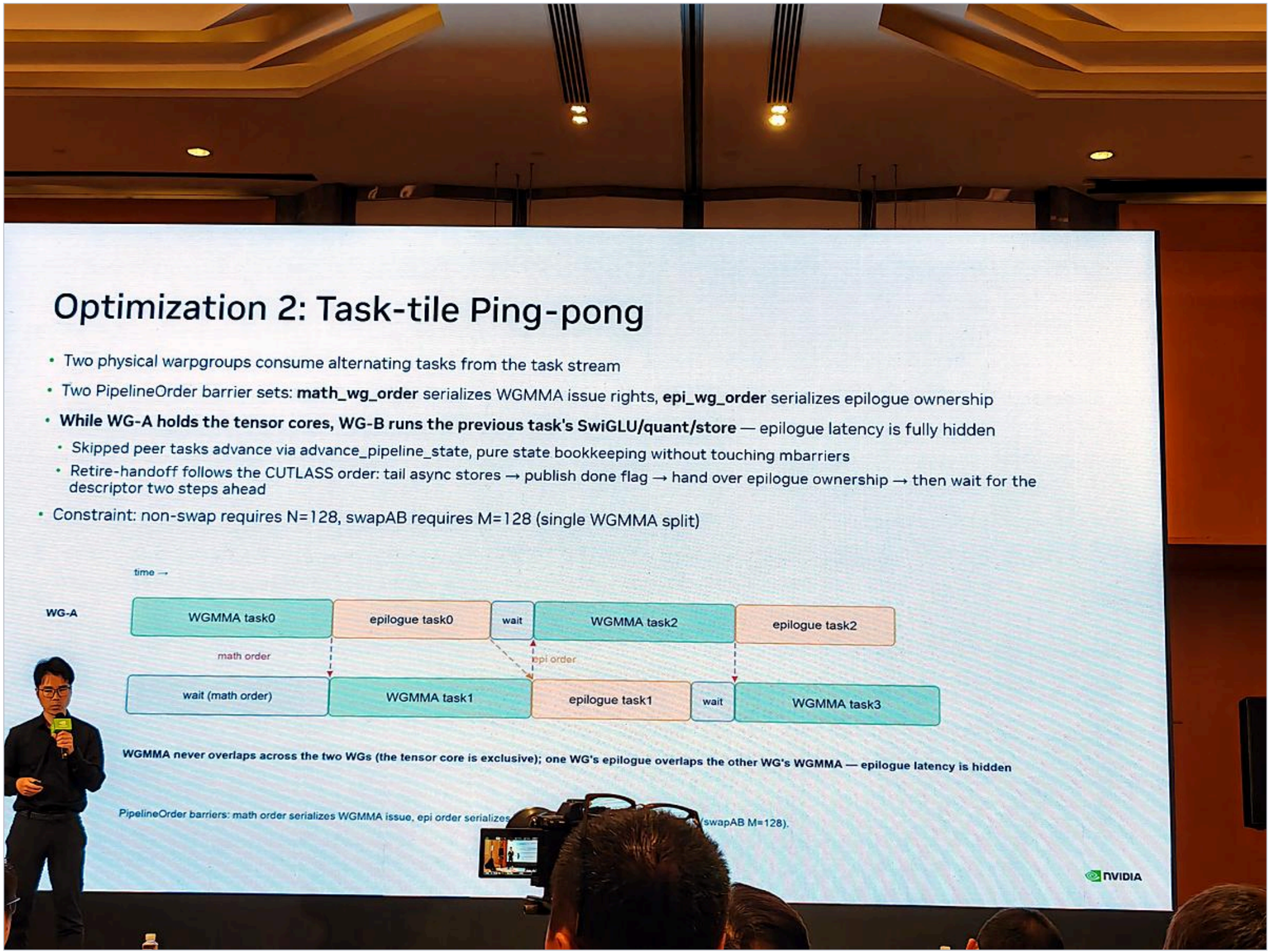

Overall Architecture: One Kernel Does Everything
• Dispatch: 4 dispatch warps pull tokens from every rank into a local pool via the NVSHMEM symmetric heap
• FC1: the pool's FP8 view is consumed directly as the GEMM activation — zero copy, zero intermediate kernels
• FC2: consumes FC1 output through a GMEM counter handshake
• Combine: FC2 results are written back to the source rank (three executors × two reduce placements = six modes)
• vs. baseline: communication warps and GEMM warps are co-resident and concurrent; tokens are computed as soon as they arrive
4 dispatch warps pull
zero-copy FP8 view
TMA store
fc1_done_counter handshake
Peer ranks NVSHMEM symmetric heap
l1_token_buffer local FP8 pool (GMEM)
FC1 WGMMA + SwiGLU + FP8 requant
fc1_output workspace (GMEM)
FC2 WGMMA BF16 output
NVLink
Combine — write back to source rank epi-warp STG/REDG or bulk push
es live in one persistent kernel: dispatch / GEMM / token-back warps are co-resident — a token is computed as soon as it arrives
NVIDIA

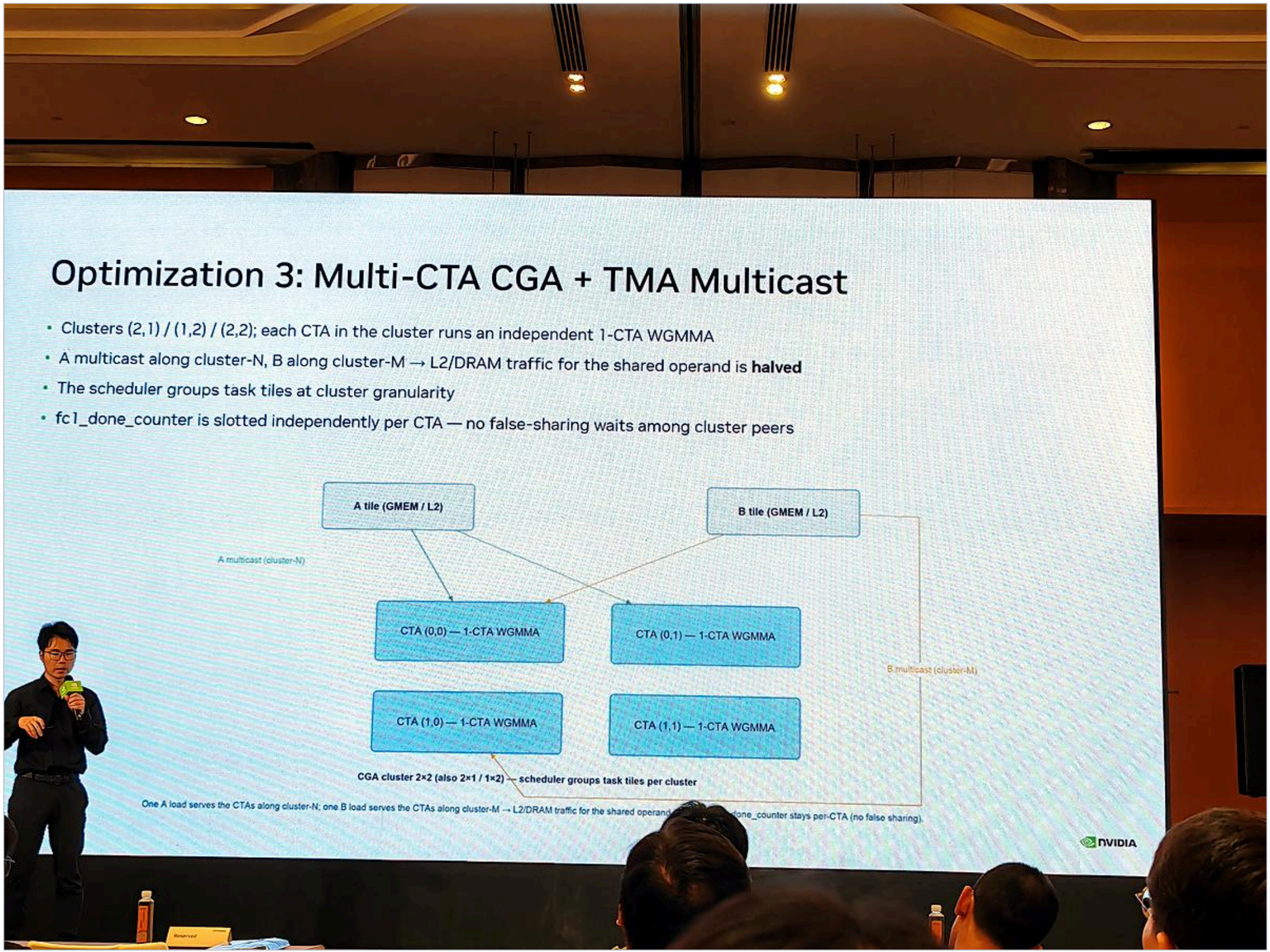

Warp Specialization
Role | Warps | setmaxnreg | Duty
epi/WGMMA WG | 4×(1–2) | 200–256 ↑ | Issue WGMMA + SwiGLU / quant / store
TMA-A | 1 | 32 ↓ | Load activation / fc1_output; FC2's fc1-done spin
TMA-B | 1 | 32 ↓ | Load weights (independent of FC1, never waits on the counter)
sched | 1 | 40 ↓ | Publish work tiles + expert prefetch
dispatch | 4 | 48 ↓ | Pull tokens across ranks (MegaMoE)
token-back | 4 (opt.) | 32 ↓ | Dedicated push of FC2 results back to the source rank
20 warps / 640 threads · setmaxnreg rebalanced per role (epi maxed out at 256)
cated MMA warpgroup — the epilogue WG issues WGMMA itself; accumulator lives in RMEM (acc stage = 1)
NVIDIA

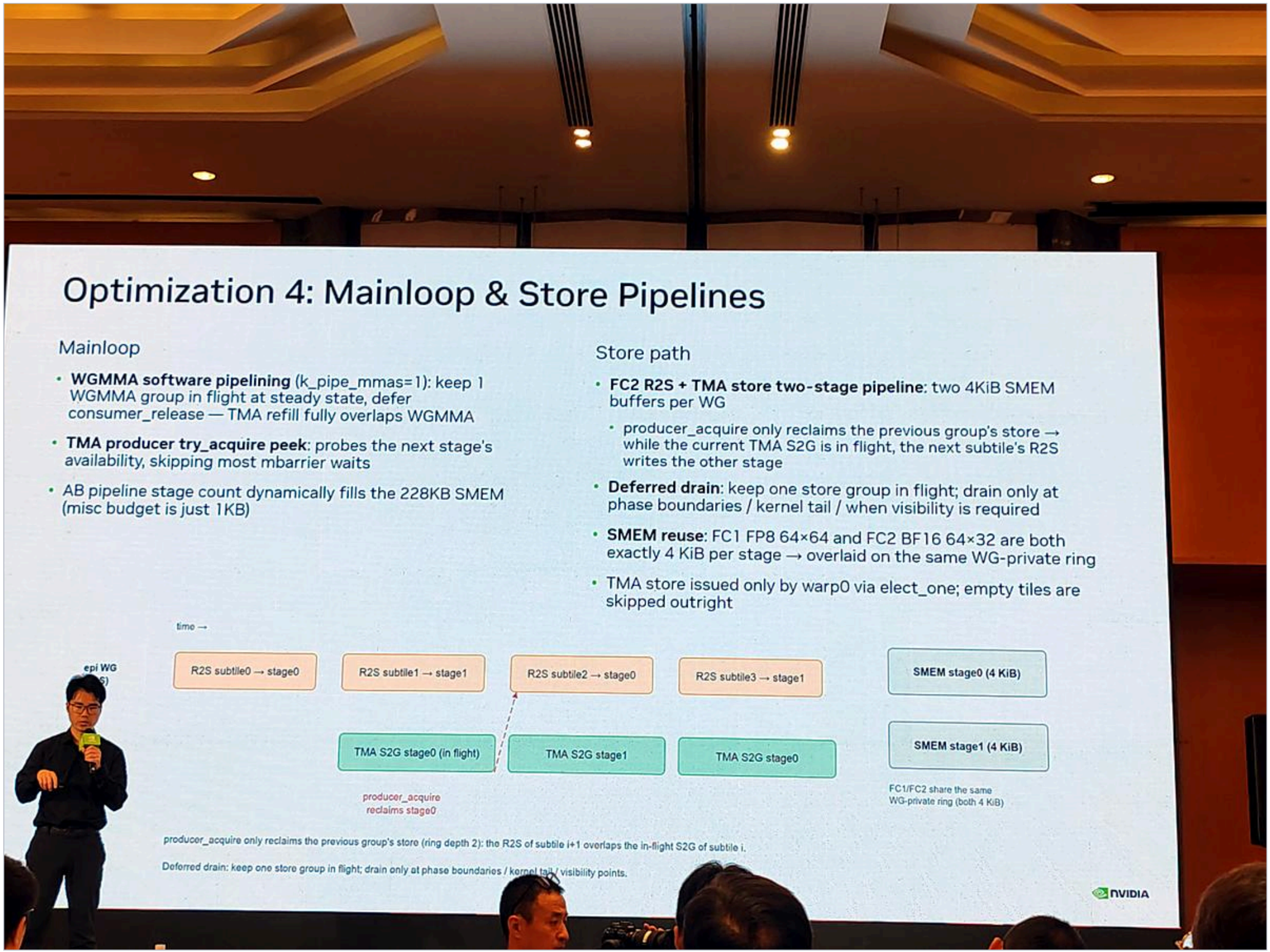

The swapAB Variant: the Cure for Small Token Counts
• Problem: SM90 WGMMA is m64nNk32 — non-swap token tile is at least 64; decode workloads waste most of it on padding
• swapAB: M = weight channels (128/256), N = tokens (8/16/32/64/128)
• WGMMA's N dimension goes down to 8 → token tile as small as 8, up to 8× better utilization
• The M axis becomes the always-dense channel dimension, so even tiny token counts can fill the SMs
• Implementation highlights:
• The swap is pure stride views, zero copy; the expert dim becomes GEMM-A's L mode, so the TMA descriptor is tile-invariant
• The transpose moves into the epilogue: FC1 via R2S SMEM addressing; FC2 drops TMA store and emits BF16 straight from registers (STG)
• The 8-row gate/up interleave lands exactly on register pairs of the same thread — folding needs zero shuffles
• Shares the MMA mainloop and warp topology with the main kernel — low maintenance cost
non-swap: tokens on M — tile M64 × N128
WGMMA m64nNk32: M is fixed at 64 per atom
← 8 real tokens
same 8 tokens
swapAB: tokens on N — tile M128/256 × N8
N is flexible down to 8 → token tile 8/16/32/64/128
M = weight channels (always dense)
N = 8 tokens, all useful
→ up to 8× better tile utilization
ows; expert dim becomes GEMM-A's L mode (TMA descriptor tile-invariant)

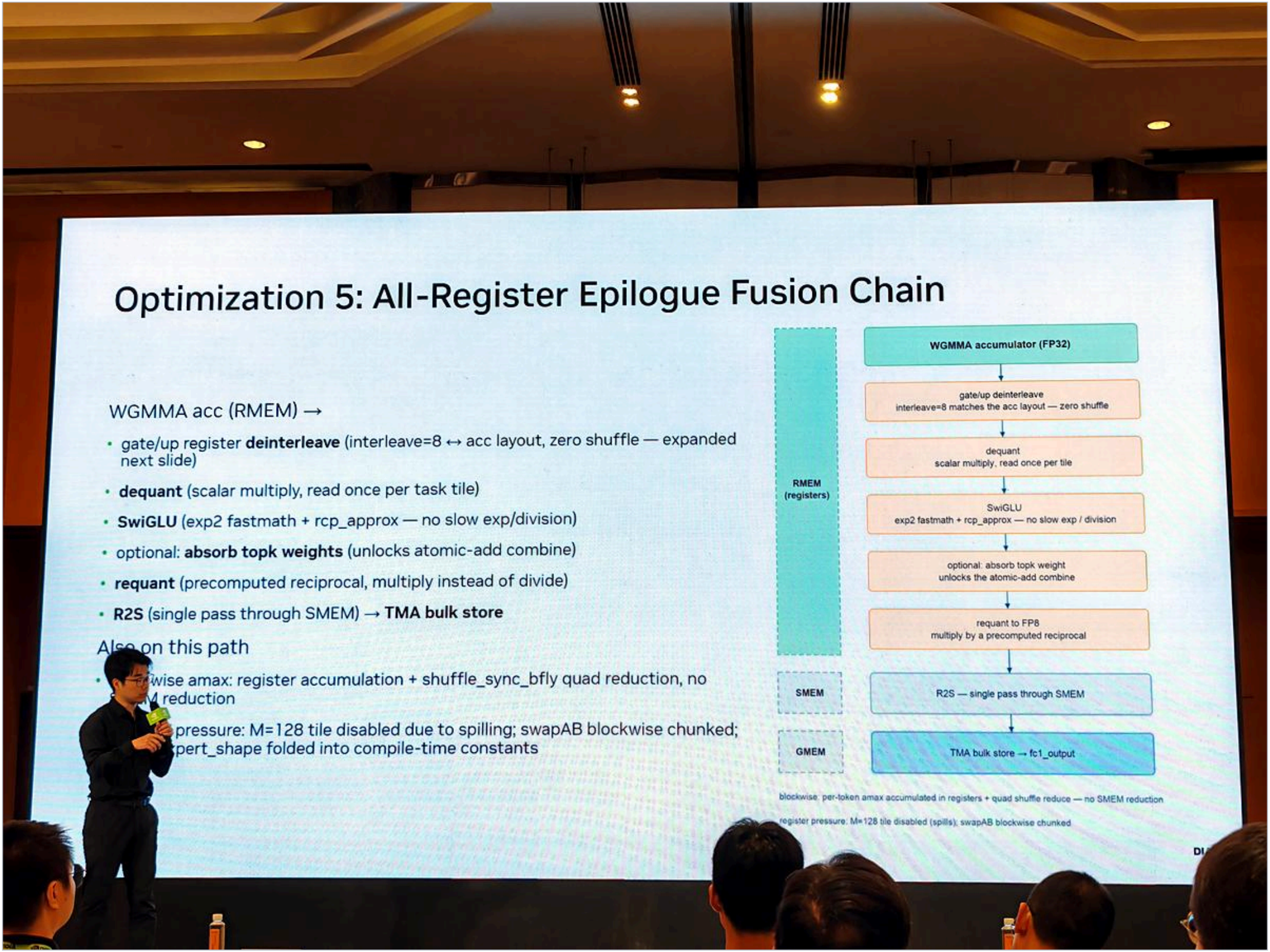

FP8 Quantization: per_tensor vs. blockwise
per_tensor
blockwise (DeepGEMM style)
activation
scalar FP32
per-token × K128 block
weight
per-expert scalar
128×128 block
FC2 activation
scalar (two-pass precompute)
per-token × K64 (online in FC1 epilogue)
dequant
scalar multiply in the epilogue
per-K-tile promote in the mainloop
accumulation
1xacc / 2xacc (FP32 promote per K tile)
promotion built in
blockwise online requant: register absmax → shuffle_sync_bfly quad reduction → single-lane STG of the scale → multiply by
iprocal
ckwise half-K split: scale loads overlap the other half's WGMMA
n SF shares the TMA stage barrier with A/B (merged tx_count) — zero extra synchronization
NVIDIA

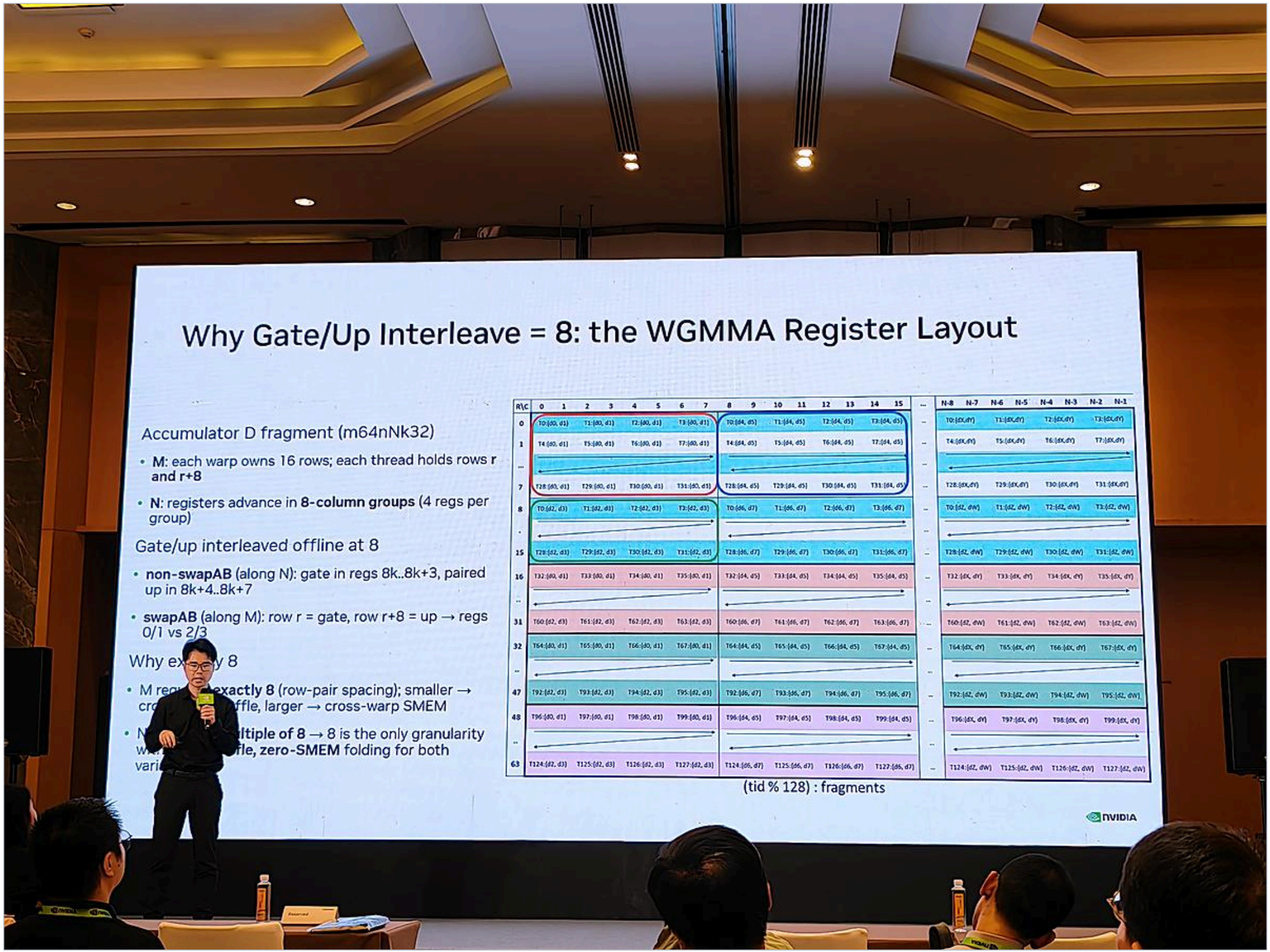

Optimization 1: Cooperative Warp-group Scheduling
• In cooperative mode the warpgroup count follows the tile shape:
• N=128 (swapAB M=128): single-warpgroup lean configuration (8 warps); epi registers can max out at 256
• N=256 (swapAB M=256): two warpgroups cooperatively split the same task tile — each owns an N=128 (M=128) slice, issues its own WGMMA and runs its own slice's epilogue
• Task boundary: CTA-level NamedBarrier; all epi warps enter the next task together
• Characteristics & trade-offs:
• Large tiles (N=256) amortize A/B loads and reduce task count → better for the GEMM-bound large batch
• WGMMA → epilogue serializes within a WG, exposing epilogue latency; but TMA warps are independent, so loads for the next task keep prefetching
• Many heuristic-table buckets are still won by cooperative (e.g. blockwise ≥512 always uses (64,128); per_tensor 16384 uses (64,256))
One task tile: M64 × N256 (swapAB: M256 × token N)
WG0
owns N = 0..127
WGMMA + epilogue
WG1
owns N = 128..255
WGMMA + epilogue
time →
WG0
WG1
WGMMA task0 (own half)
epilogue task0
WGMMA task1
epilogue task1
CTA NamedBarrier at each task boundary — all epi warps advance together
NVIDIA

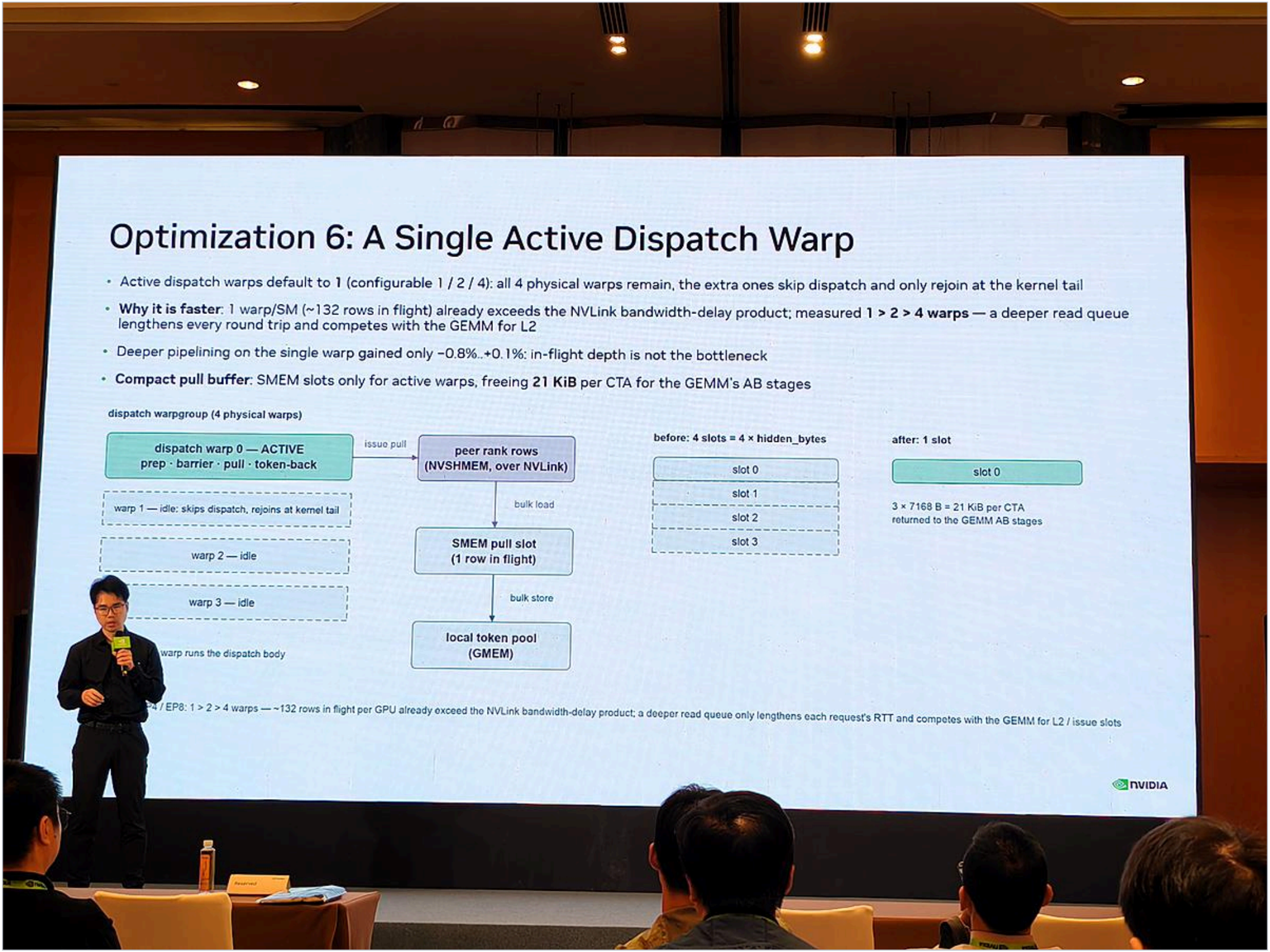

Optimization 2: Task-tile Ping-pong
• Two physical warpgroups consume alternating tasks from the task stream
• Two PipelineOrder barrier sets: math_wg_order serializes WGMMA issue rights, epi_wg_order serializes epilogue ownership
• While WG-A holds the tensor cores, WG-B runs the previous task's SwiGLU/quant/store — epilogue latency is fully hidden
• Skipped peer tasks advance via advance_pipeline_state, pure state bookkeeping without touching mbarriers
• Retire-handoff follows the CUTLASS order: tail async stores → publish done flag → hand over epilogue ownership → then wait for the descriptor two steps ahead
• Constraint: non-swap requires N=128, swapAB requires M=128 (single WGMMA split)
time →
WG-A
WGMMA task0
epilogue task0
wait
WGMMA task2
epilogue task2
math order
epi order
wait (math order)
WGMMA task1
epilogue task1
wait
WGMMA task3
WGMMA never overlaps across the two WGs (the tensor core is exclusive); one WG's epilogue overlaps the other WG's WGMMA — epilogue latency is hidden
PipelineOrder barriers: math order serializes WGMMA issue, epi order serializes
(swapAB M=128).
NVIDIA

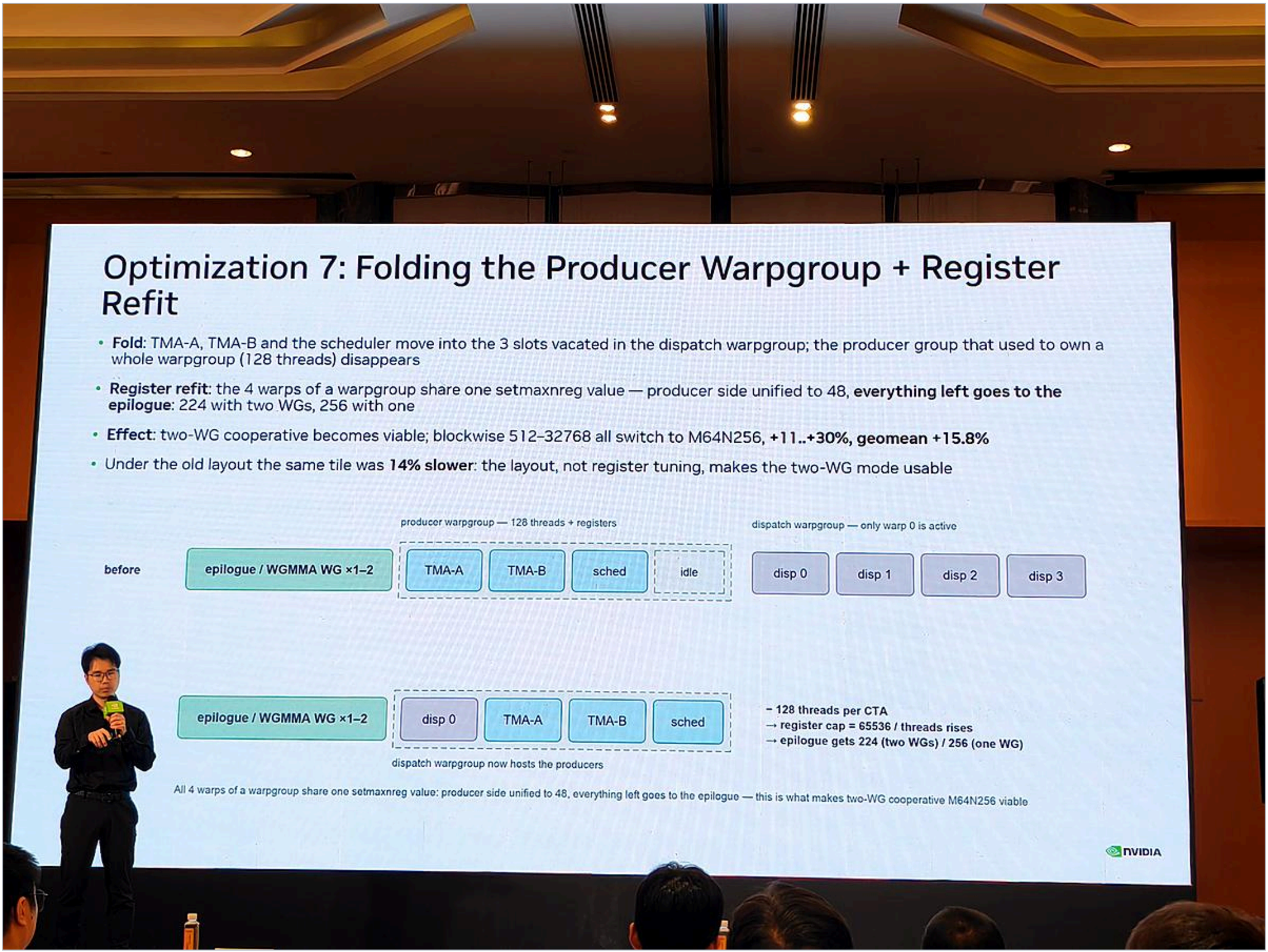

Optimization 3: Multi-CTA CGA + TMA Multicast
• Clusters (2,1) / (1,2) / (2,2); each CTA in the cluster runs an independent 1-CTA WGMMA
• A multicast along cluster-N, B along cluster-M → L2/DRAM traffic for the shared operand is halved
• The scheduler groups task tiles at cluster granularity
• fc1_done_counter is slotted independently per CTA — no false-sharing waits among cluster peers
A tile (GMEM / L2)
B tile (GMEM / L2)
A multicast (cluster-N)
CTA (0,0) — 1-CTA WGMMA
CTA (0,1) — 1-CTA WGMMA
B multicast (cluster-M)
CTA (1,0) — 1-CTA WGMMA
CTA (1,1) — 1-CTA WGMMA
CGA cluster 2×2 (also 2×1 / 1×2) — scheduler groups task tiles per cluster
One A load serves the CTAs along cluster-N; one B load serves the CTAs along cluster-M → L2/DRAM traffic for the shared operand
done_counter stays per-CTA (no false sharing).
NVIDIA

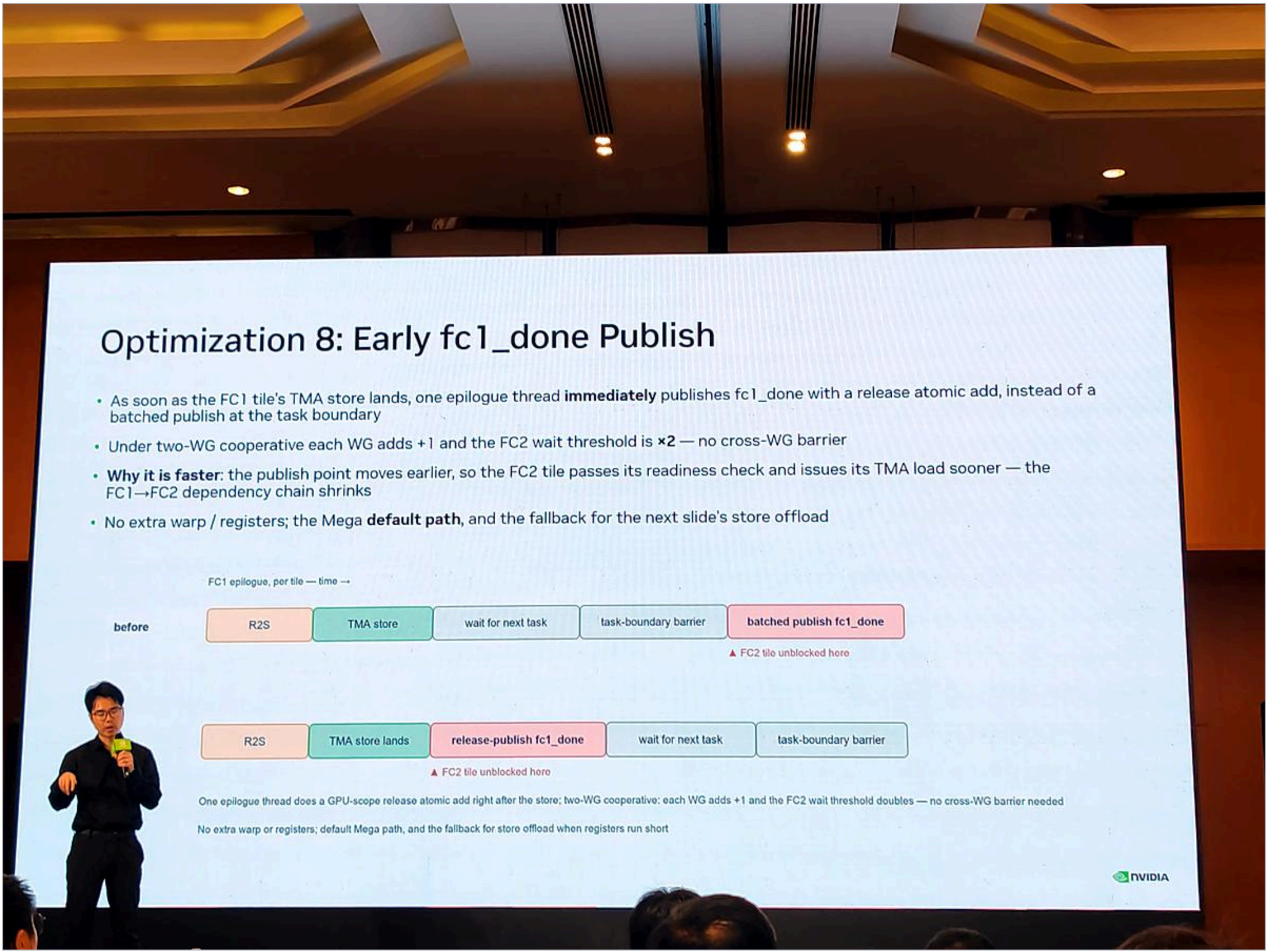

Optimization 4: Mainloop & Store Pipelines
Mainloop
• WGMMA software pipelining (k_pipe_mmas=1): keep 1 WGMMA group in flight at steady state, defer consumer_release — TMA refill fully overlaps WGMMA
• TMA producer try_acquire peek: probes the next stage's availability, skipping most mbarrier waits
• AB pipeline stage count dynamically fills the 228KB SMEM (misc budget is just 1KB)
Store path
• FC2 R2S + TMA store two-stage pipeline: two 4KiB SMEM buffers per WG
• producer_acquire only reclaims the previous group's store → while the current TMA S2G is in flight, the next subtile's R2S writes the other stage
• Deferred drain: keep one store group in flight; drain only at phase boundaries / kernel tail / when visibility is required
• SMEM reuse: FC1 FP8 64×64 and FC2 BF16 64×32 are both exactly 4 KiB per stage → overlaid on the same WG-private ring
• TMA store issued only by warp0 via elect_one; empty tiles are skipped outright
time →
R2S subtile0 → stage0
R2S subtile1 → stage1
R2S subtile2 → stage0
R2S subtile3 → stage1
TMA S2G stage0 (in flight)
TMA S2G stage1
TMA S2G stage0
producer_acquire reclaims stage0
SMEM stage0 (4 KiB)
SMEM stage1 (4 KiB)
FC1/FC2 share the same WG-private ring (both 4 KiB)
producer_acquire only reclaims the previous group's store (ring depth 2): the R2S of subtile i+1 overlaps the in-flight S2G of subtile i.
Deferred drain: keep one store group in flight; drain only at phase boundaries / kernel tail / visibility points.
NVIDIA

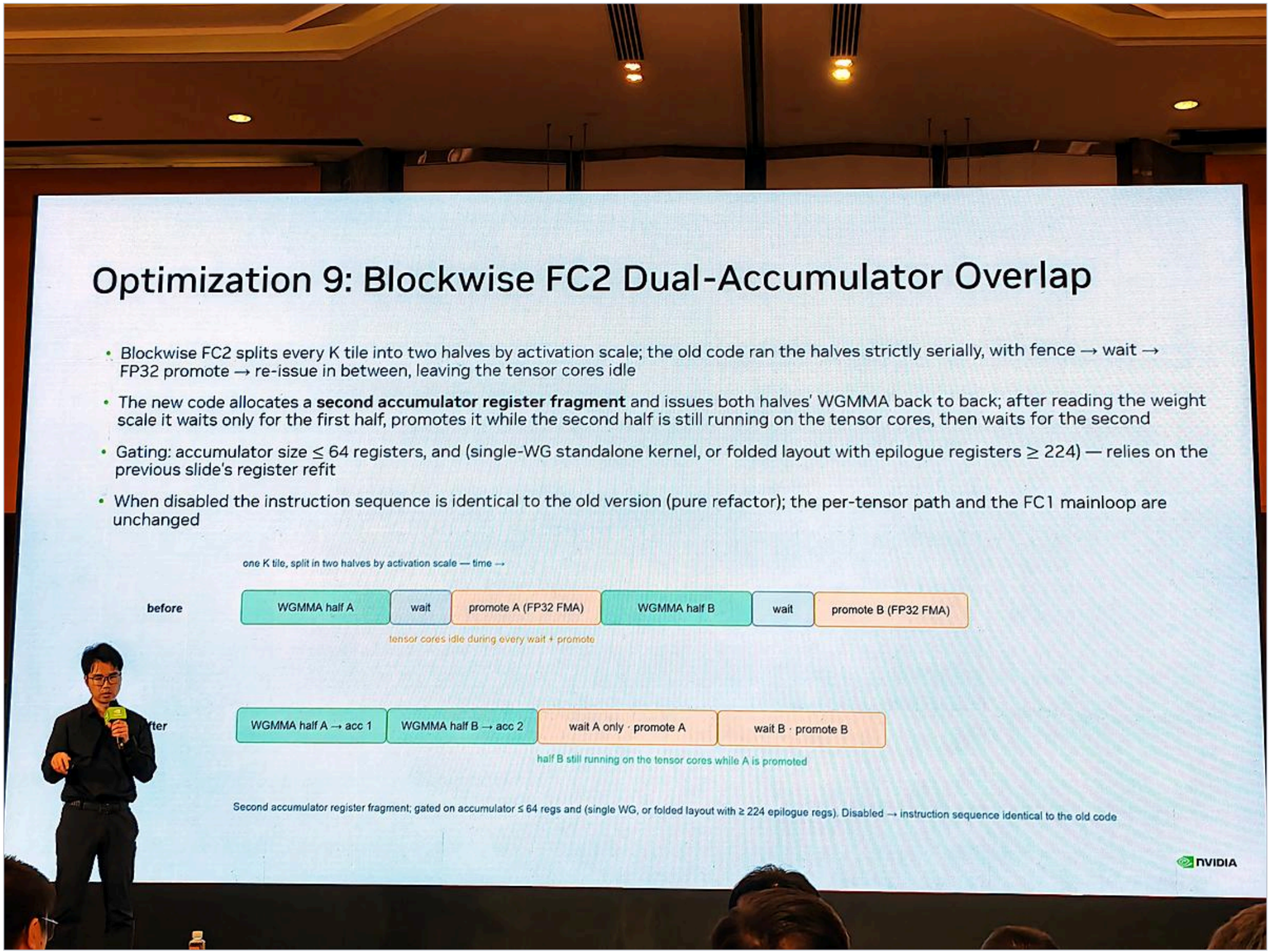

Optimization 5: All-Register Epilogue Fusion Chain
WGMMA acc (RMEM) →
• gate/up register deinterleave (interleave=8 ↔ acc layout, zero shuffle — expanded next slide)
• dequant (scalar multiply, read once per task tile)
• SwiGLU (exp2 fastmath + rcp_approx — no slow exp/division)
• optional: absorb topk weights (unlocks atomic-add combine)
• requant (precomputed reciprocal, multiply instead of divide)
• R2S (single pass through SMEM) → TMA bulk store
Also on this path
• wise amax: register accumulation + shuffle_sync_bfly quad reduction, no M reduction
pressure: M=128 tile disabled due to spilling; swapAB blockwise chunked;
pert_shape folded into compile-time constants
RMEM (registers)
SMEM
GMEM
WGMMA accumulator (FP32)
gate/up deinterleave
interleave=8 matches the acc layout — zero shuffle
dequant
scalar multiply, read once per tile
SwiGLU
exp2 fastmath + rcp_approx — no slow exp / division
optional: absorb topk weight
unlocks the atomic-add combine
requant to FP8
multiply by a precomputed reciprocal
R2S — single pass through SMEM
TMA bulk store → fc1_output
blockwise: per-token amax accumulated in registers + quad shuffle reduce — no SMEM reduction
register pressure: M=128 tile disabled (spills); swapAB blockwise chunked

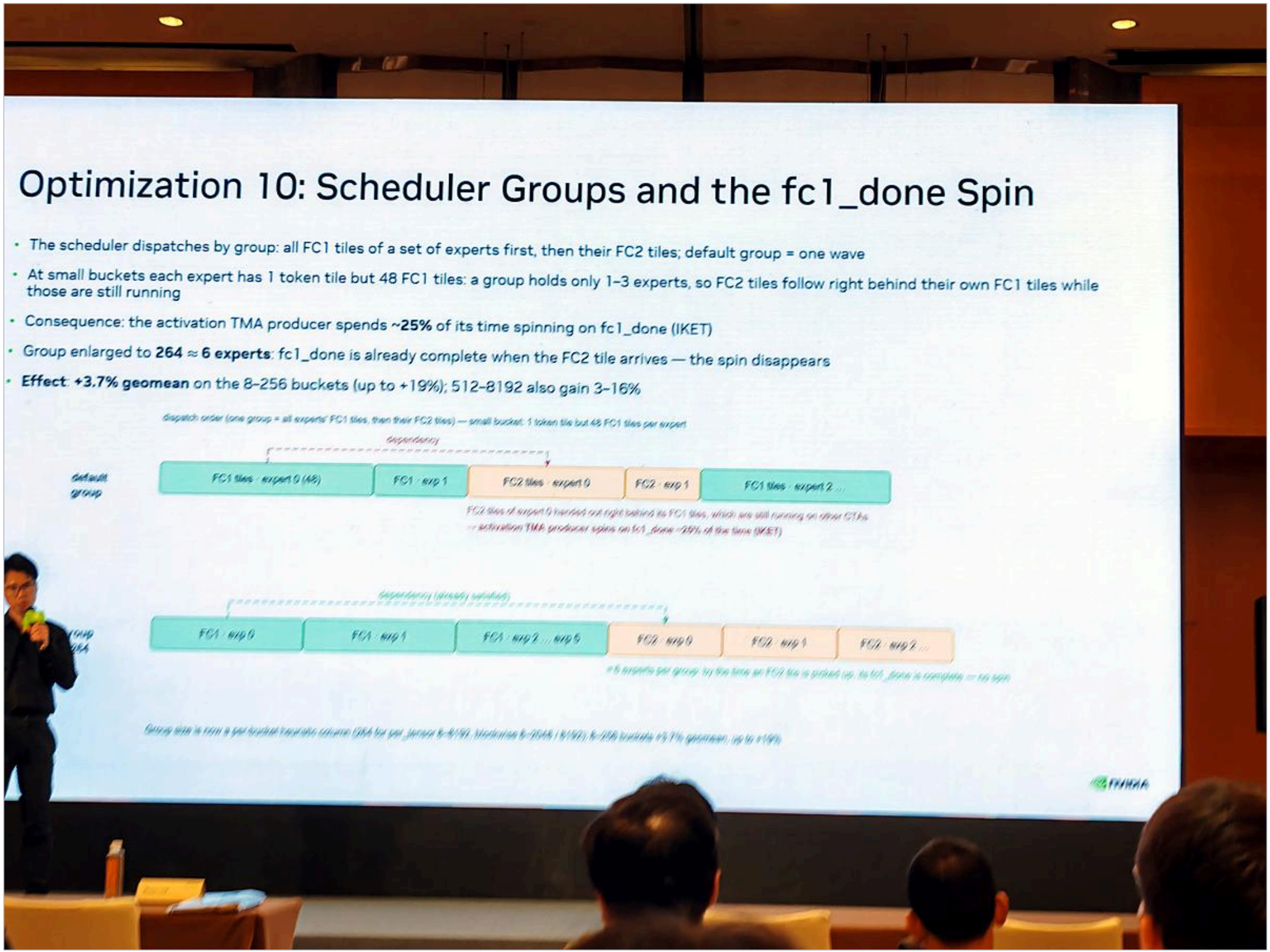

Why Gate/Up Interleave = 8: the WGMMA Register Layout
Accumulator D fragment (m64nNk32)
• M: each warp owns 16 rows; each thread holds rows r and r+8
• N: registers advance in 8-column groups (4 regs per group)
Gate/up interleaved offline at 8
• non-swapAB (along N): gate in regs 8k..8k+3, paired up in 8k+4..8k+7
• swapAB (along M): row r = gate, row r+8 = up → regs 0/1 vs 2/3
Why ex y 8
• M req exactly 8 (row-pair spacing); smaller → cr ffle, larger → cross-warp SMEM
• N ltiple of 8 → 8 is the only granularity w fle, zero-SMEM folding for both vari
R\C
(tid % 128) : fragments
NVIDIA

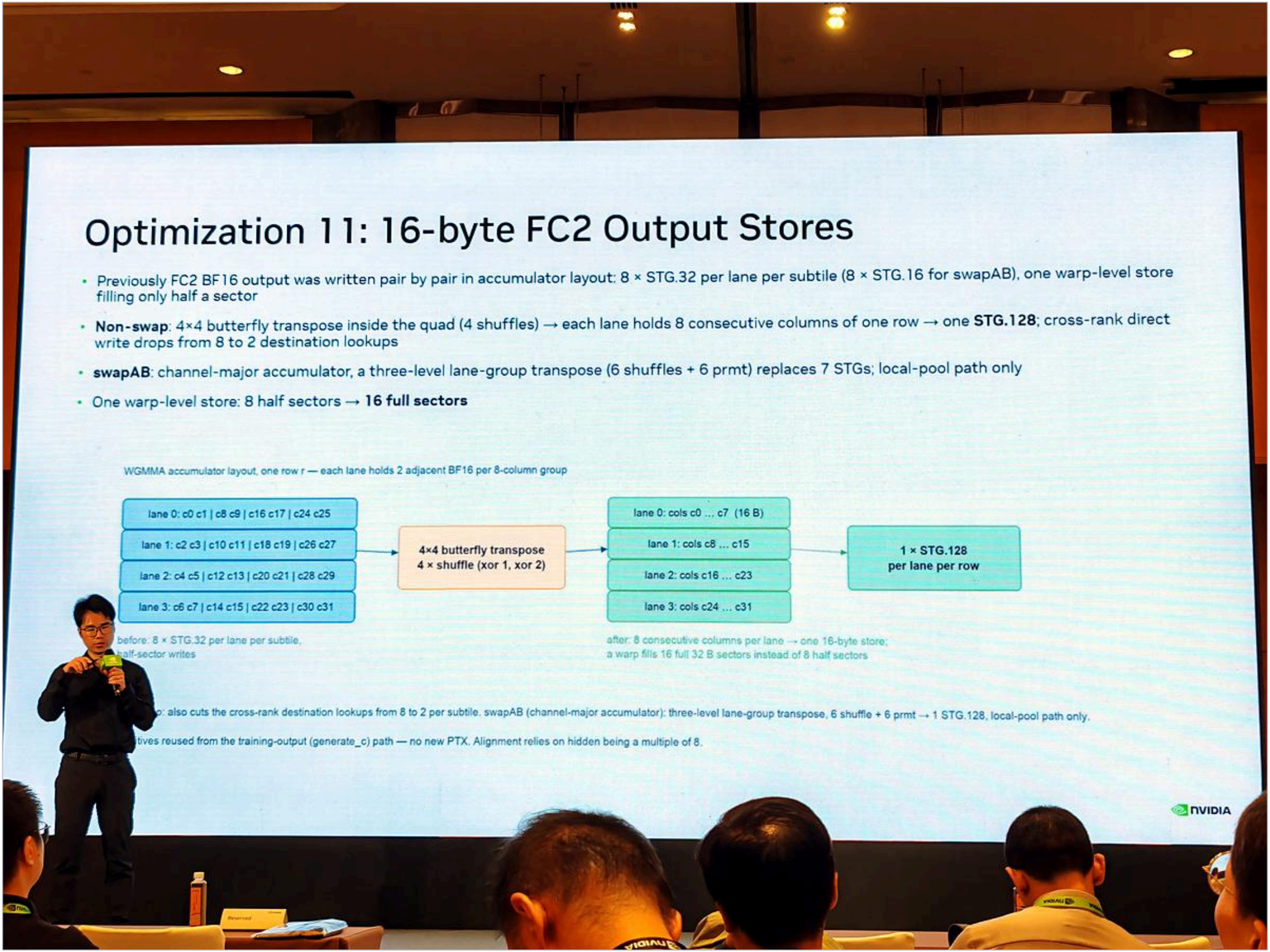

Optimization 6: A Single Active Dispatch Warp
• Active dispatch warps default to 1 (configurable 1 / 2 / 4): all 4 physical warps remain, the extra ones skip dispatch and only rejoin at the kernel tail
• Why it is faster: 1 warp/SM (~132 rows in flight) already exceeds the NVLink bandwidth-delay product; measured 1 > 2 > 4 warps — a deeper read queue lengthens every round trip and competes with the GEMM for L2
• Deeper pipelining on the single warp gained only −0.8%..+0.1%: in-flight depth is not the bottleneck
• Compact pull buffer: SMEM slots only for active warps, freeing 21 KiB per CTA for the GEMM's AB stages
dispatch warpgroup (4 physical warps)
dispatch warp 0 — ACTIVE
prep · barrier · pull · token-back
issue pull
peer rank rows
(NVSHMEM, over NVLink)
warp 1 — idle: skips dispatch, rejoins at kernel tail
bulk load
warp 2 — idle
SMEM pull slot
(1 row in flight)
warp 3 — idle
bulk store
local token pool
(GMEM)
warp runs the dispatch body
before: 4 slots = 4 × hidden_bytes
slot 0
slot 1
slot 2
slot 3
after: 1 slot
slot 0
3 × 7168 B = 21 KiB per CTA
returned to the GEMM AB stages
4 / EP8: 1 > 2 > 4 warps — ~132 rows in flight per GPU already exceed the NVLink bandwidth-delay product; a deeper read queue only lengthens each request's RTT and competes with the GEMM for L2 / issue slots
NVIDIA


Optimization 7: Folding the Producer Warpgroup + Register Refit
• Fold: TMA-A, TMA-B and the scheduler move into the 3 slots vacated in the dispatch warpgroup; the producer group that used to own a whole warpgroup (128 threads) disappears
• Register refit: the 4 warps of a warpgroup share one setmaxnreg value — producer side unified to 48, everything left goes to the epilogue: 224 with two WGs, 256 with one
• Effect: two-WG cooperative becomes viable; blockwise 512–32768 all switch to M64N256, +11..+30%, geomean +15.8%
• Under the old layout the same tile was 14% slower: the layout, not register tuning, makes the two-WG mode usable
producer warpgroup — 128 threads + registers
dispatch warpgroup — only warp 0 is active
before
epilogue / WGMMA WG ×1–2
TMA-A
TMA-B
sched
idle
disp 0
disp 1
disp 2
disp 3
epilogue / WGMMA WG ×1–2
disp 0
TMA-A
TMA-B
sched
dispatch warpgroup now hosts the producers
– 128 threads per CTA
→ register cap = 65536 / threads rises
→ epilogue gets 224 (two WGs) / 256 (one WG)
All 4 warps of a warpgroup share one setmaxnreg value: producer side unified to 48, everything left goes to the epilogue — this is what makes two-WG cooperative M64N256 viable
NVIDIA


Optimization 8: Early fc1_done Publish
As soon as the FC1 tile's TMA store lands, one epilogue thread immediately publishes fc1_done with a release atomic add, instead of a batched publish at the task boundary
Under two-WG cooperative each WG adds +1 and the FC2 wait threshold is ×2 — no cross-WG barrier
Why it is faster: the publish point moves earlier, so the FC2 tile passes its readiness check and issues its TMA load sooner — the FC1→FC2 dependency chain shrinks
No extra warp / registers; the Mega default path, and the fallback for the next slide's store offload
FC1 epilogue, per tile — time →
before
R2S
TMA store
wait for next task
task-boundary barrier
batched publish fc1_done
▲ FC2 tile unblocked here
R2S
TMA store lands
release-publish fc1_done
wait for next task
task-boundary barrier
▲ FC2 tile unblocked here
One epilogue thread does a GPU-scope release atomic add right after the store; two-WG cooperative: each WG adds +1 and the FC2 wait threshold doubles — no cross-WG barrier needed
No extra warp or registers; default Mega path, and the fallback for store offload when registers run short
NVIDIA


Optimization 9: Blockwise FC2 Dual-Accumulator Overlap
• Blockwise FC2 splits every K tile into two halves by activation scale; the old code ran the halves strictly serially, with fence → wait → FP32 promote → re-issue in between, leaving the tensor cores idle
• The new code allocates a second accumulator register fragment and issues both halves' WGMMA back to back; after reading the weight scale it waits only for the first half, promotes it while the second half is still running on the tensor cores, then waits for the second
• Gating: accumulator size ≤ 64 registers, and (single-WG standalone kernel, or folded layout with epilogue registers ≥ 224) — relies on the previous slide's register refit
• When disabled the instruction sequence is identical to the old version (pure refactor); the per-tensor path and the FC1 mainloop are unchanged
one K tile, split in two halves by activation scale — time →
before
WGMMA half A
wait
promote A (FP32 FMA)
WGMMA half B
wait
promote B (FP32 FMA)
tensor cores idle during every wait + promote
WGMMA half A → acc 1
WGMMA half B → acc 2
wait A only · promote A
wait B · promote B
half B still running on the tensor cores while A is promoted
Second accumulator register fragment; gated on accumulator ≤ 64 regs and (single WG, or folded layout with ≥ 224 epilogue regs). Disabled → instruction sequence identical to the old code
NVIDIA


Optimization 10: Scheduler Groups and the fc1_done Spin
The scheduler dispatches by group: all FC1 tiles of a set of experts first, then their FC2 tiles; default group = one wave
At small buckets each expert has 1 token tile but 48 FC1 tiles: a group holds only 1–3 experts, so FC2 tiles follow right behind their own FC1 tiles while those are still running
Consequence: the activation TMA producer spends ~25% of its time spinning on fc1_done (IKET)
Group enlarged to 264 ≈ 6 experts: fc1_done is already complete when the FC2 tile arrives — the spin disappears
Effect: +3.7% geomean on the 8–256 buckets (up to +19%); 512–8192 also gain 3–16%

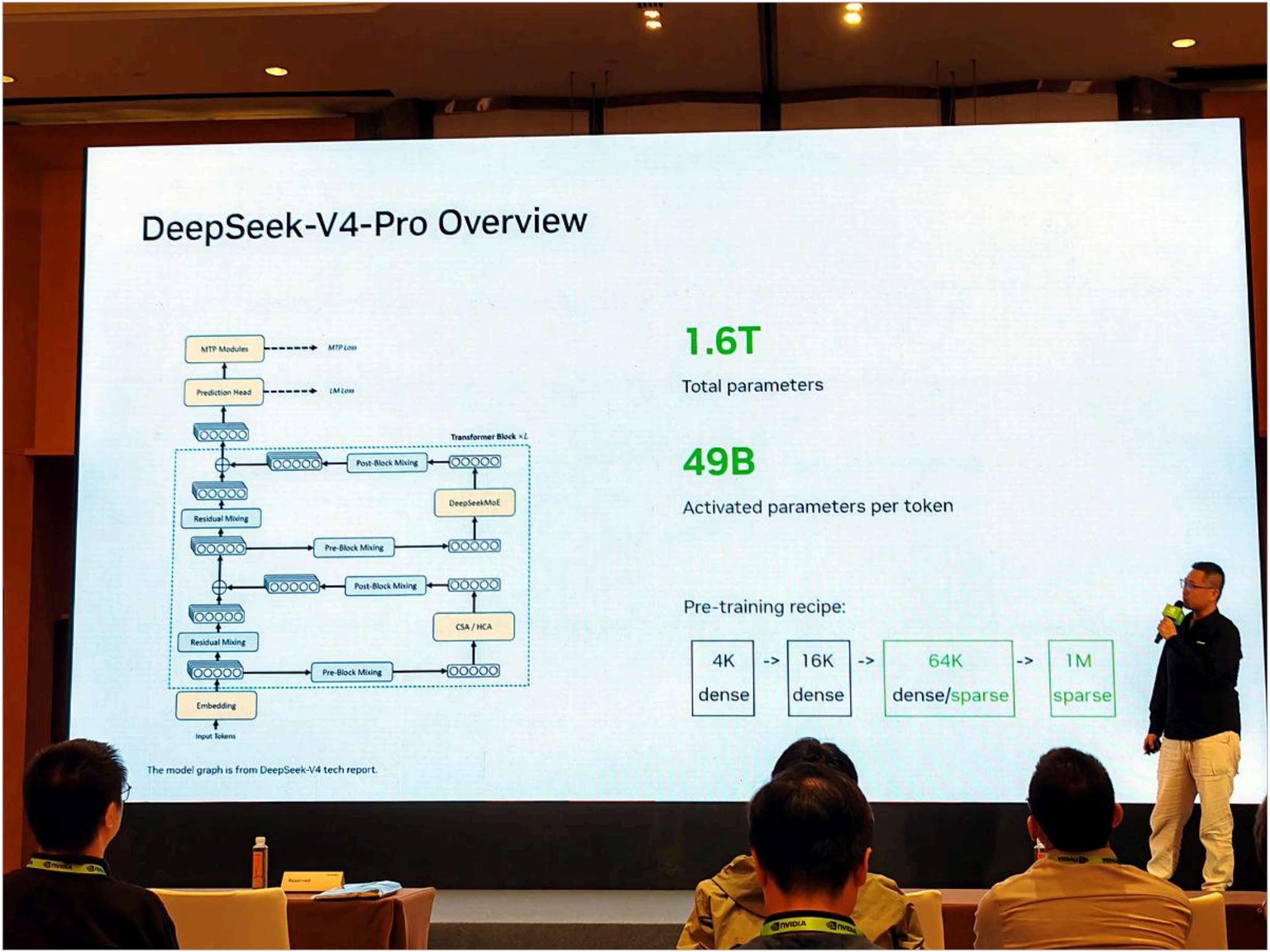

Optimization 11: 16-byte FC2 Output Stores
Previously FC2 BF16 output was written pair by pair in accumulator layout: 8 × STG.32 per lane per subtile (8 × STG.16 for swapAB), one warp-level store filling only half a sector
Non-swap: 4×4 butterfly transpose inside the quad (4 shuffles) → each lane holds 8 consecutive columns of one row → one STG.128; cross-rank direct write drops from 8 to 2 destination lookups
swapAB: channel-major accumulator, a three-level lane-group transpose (6 shuffles + 6 prmt) replaces 7 STGs; local-pool path only
One warp-level store: 8 half sectors → 16 full sectors
WGMMA accumulator layout, one row r — each lane holds 2 adjacent BF16 per 8-column group
lane 0: c0 c1 | c8 c9 | c16 c17 | c24 c25
lane 1: c2 c3 | c10 c11 | c18 c19 | c26 c27
lane 2: c4 c5 | c12 c13 | c20 c21 | c28 c29
lane 3: c6 c7 | c14 c15 | c22 c23 | c30 c31
4×4 butterfly transpose
4 × shuffle (xor 1, xor 2)
lane 0: cols c0 … c7 (16 B)
lane 1: cols c8 … c15
lane 2: cols c16 … c23
lane 3: cols c24 … c31
1 × STG.128
per lane per row
before: 8 × STG.32 per lane per subtile,
half-sector writes
after: 8 consecutive columns per lane → one 16-byte store;
a warp fills 16 full 32 B sectors instead of 8 half sectors
: also cuts the cross-rank destination lookups from 8 to 2 per subtile. swapAB (channel-major accumulator): three-level lane-group transpose, 6 shuffle + 6 prmt → 1 STG.128, local-pool path only.
tives reused from the training-output (generate_c) path — no new PTX. Alignment relies on hidden being a multiple of 8.
NVIDIA

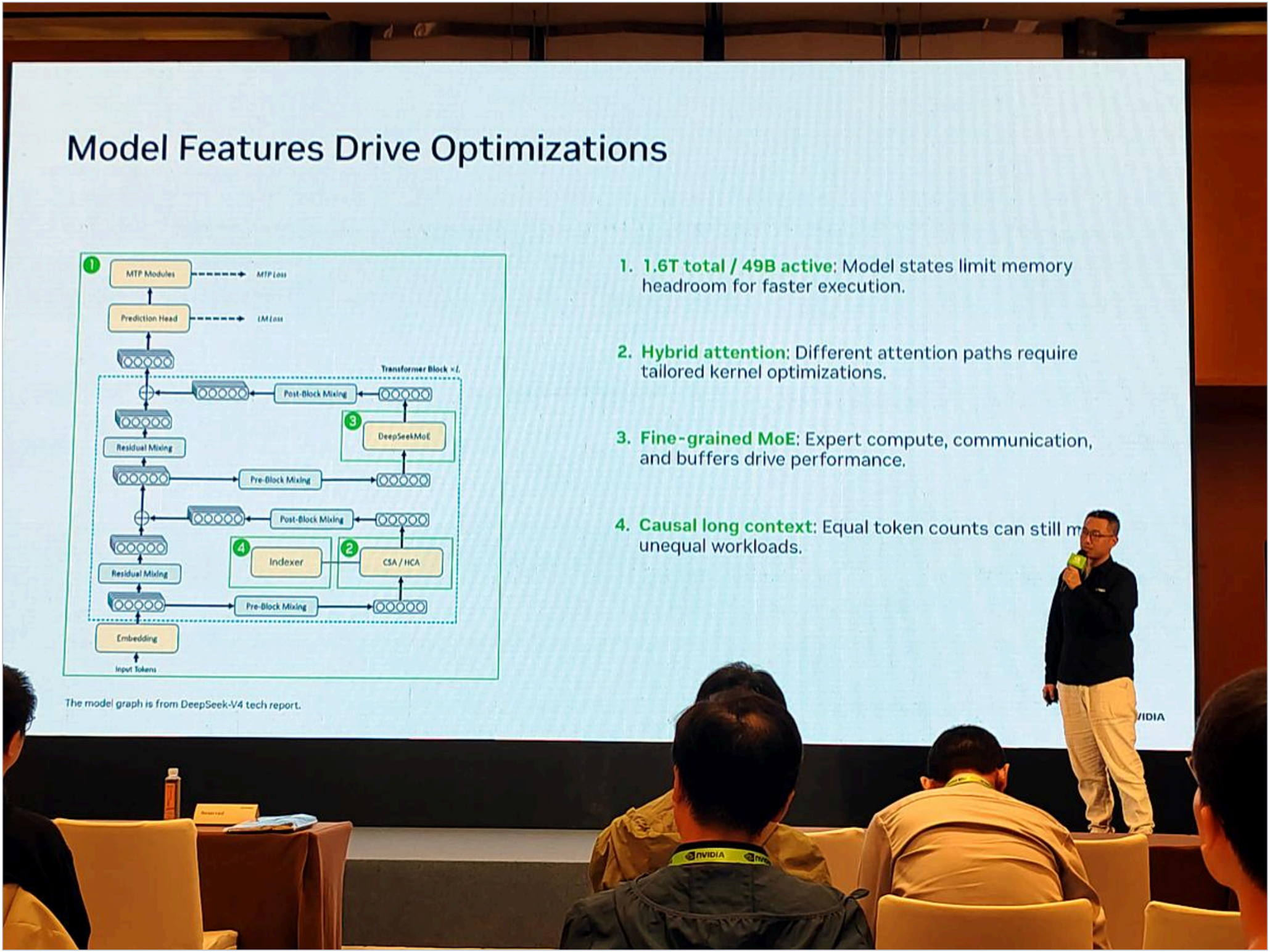

Token Sweep Results
4× H200, EP4, DeepSeek-V4-Pro shape, full MoE-layer forward; the number above each group = speedup (MegaMoE ÷ baseline)
Baseline = the conventional four-stage flow: NCCL all-gather / reduce-scatter + CUTLASS grouped GEMM (fused finalize)
Per-Tensor: MegaMoE leads at all 13 buckets — 1.02–1.11× up to 512, 1.24–1.35× from 1024 up (peak 1.35× at 8192), geomean 1.16×
Blockwise: leads at all 13 buckets — ≥1.16× from 32 up, widening to 1.33–1.72× from 1024 (peak 1.72× at 32768), geomean 1.30×
Both are close at small token counts — launch / communication dominated; the single-kernel advantage shows once the GEMM share grows
MegaMoE — fused single kernel (NVSHMEM dispatch/combine)
all-gather / reduce-scatter + CUTLASS grouped GEMM
4× H200 · EP4 · DeepSeek-V4-Pro shape · full MoE-layer forward, p50 over the slowest rank
Per-Tensor FP8
Blockwise FP8 (DeepGEMM-style)
tokens per rank
NVIDIA

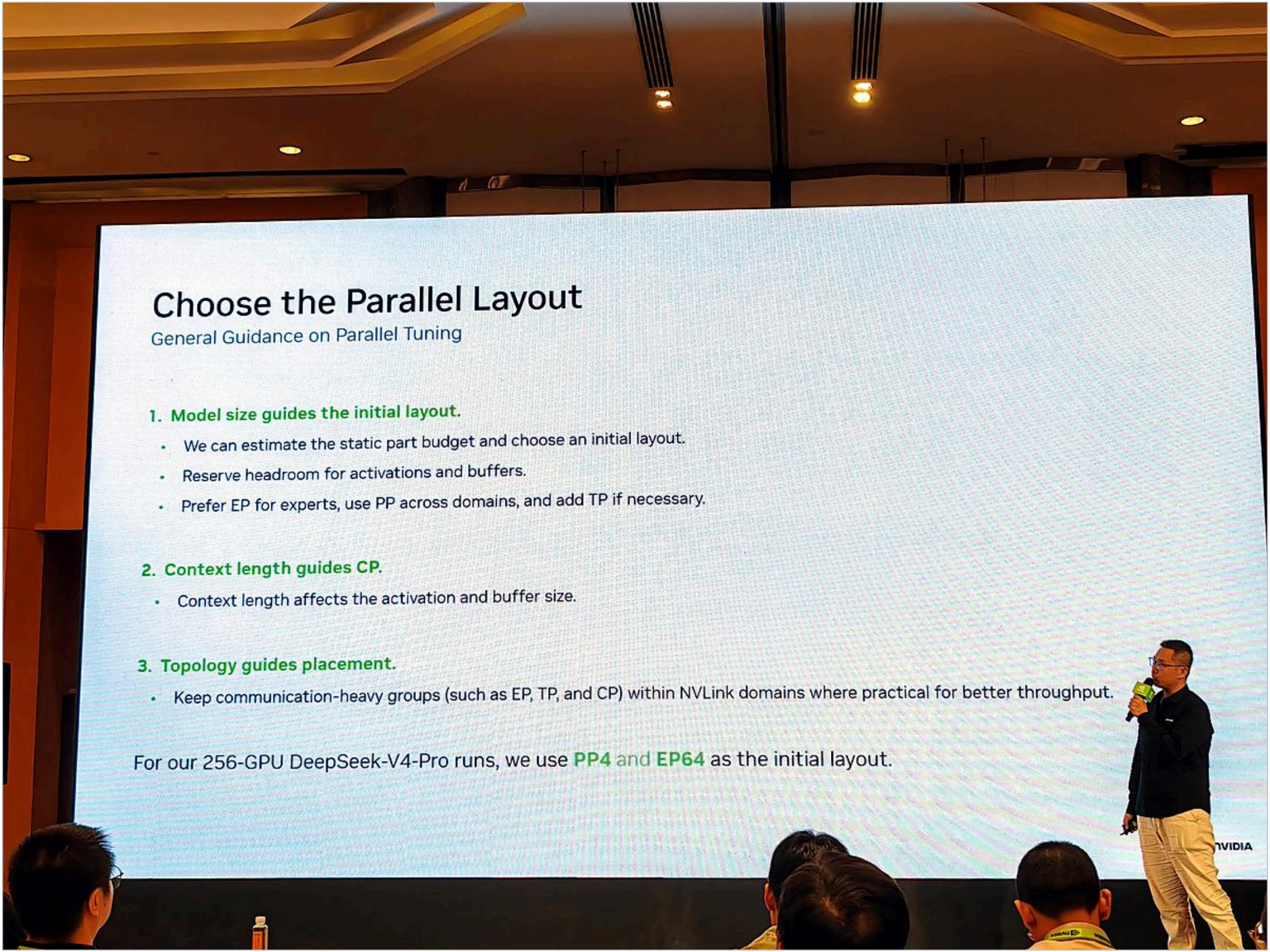

Summary
What was built
• One persistent kernel fusing dispatch + FC1/SwiGLU + FC2 + combine (six modes)
• Two complementary kernel variants (non-swap / swapAB) + two quantization modes (per_tensor / blockwise)
• 30+ kernel/communication/scheduling optimizations, with token-bucket heuristics freezing the sweep winners

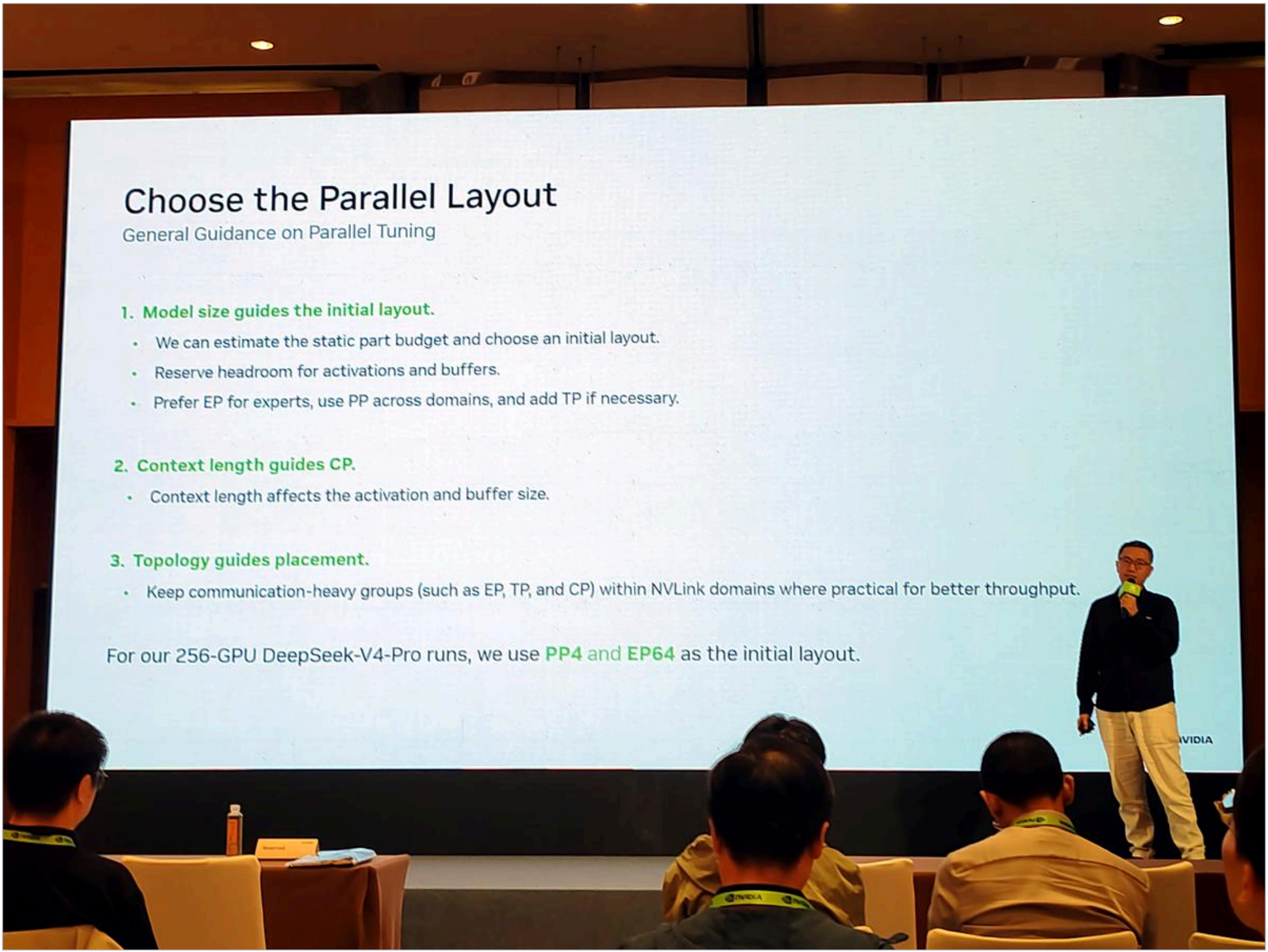

Two Challenges Define Training at Scale
Make it fit:
Model states and HBM capacity
Parallel layout and topology
Feature compatibility
Make it fast:
Kernel and launch overhead
Recompute and offload costs
Communication and load imbalance
Frontier Models
Massive parameter counts
Long context windows
This talk is based on
Public DeepSeek-V4-Pro model archite
Implementations in Megatron Core
NVIDIA Blackwell GPUs
NVIDIA

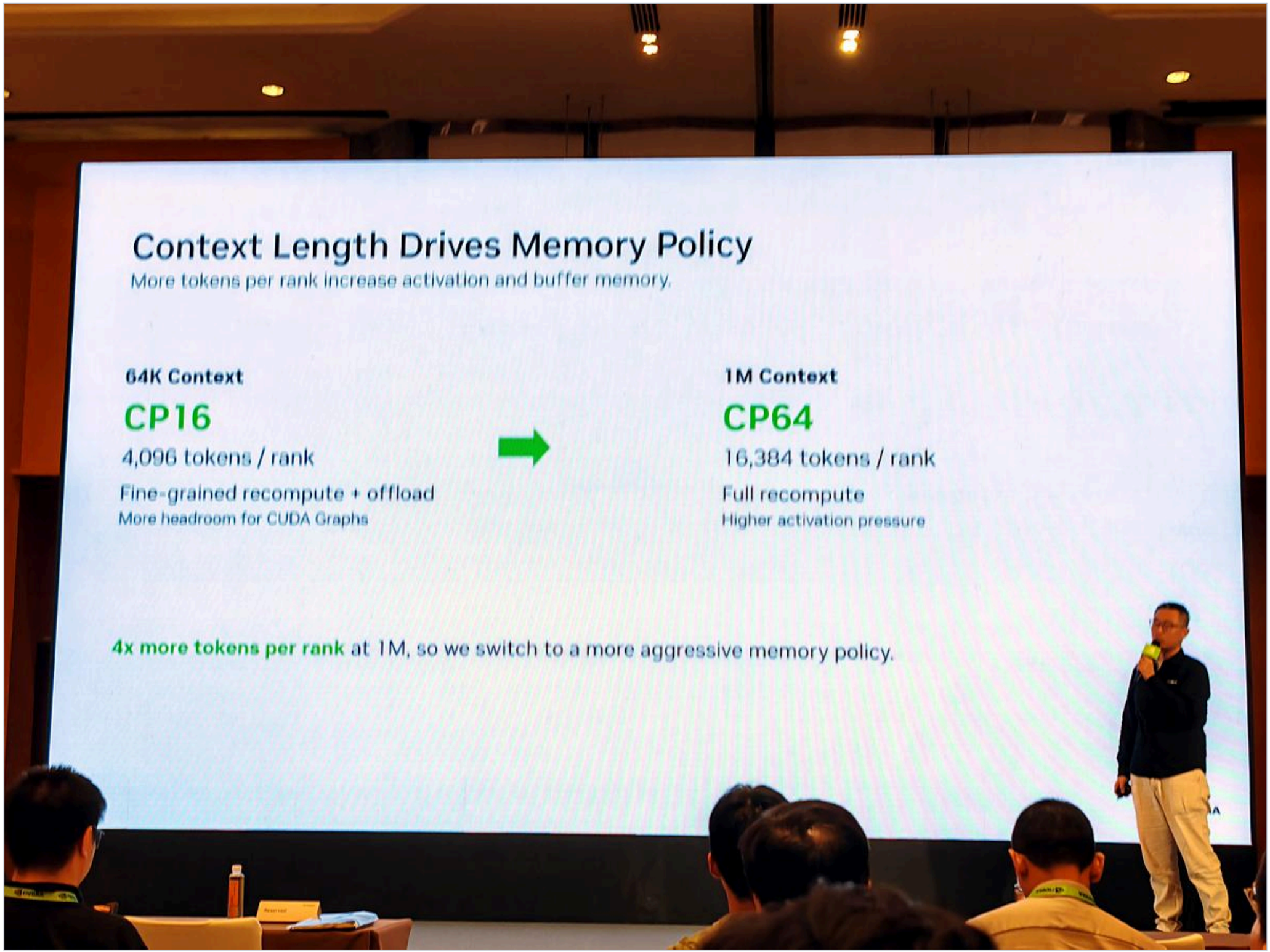

One Playbook, Two Workloads
One Optimization Playbook
1. Runtime Overhead and Memory Trade-offs
2. Attention Compute Efficiency
3. Expert Execution and Memory Footprint
4. Context-Parallel Load Balance
Measured throughput gains from the playbook
+ 59%
at 64K context length
+ 32%
at 1M context length

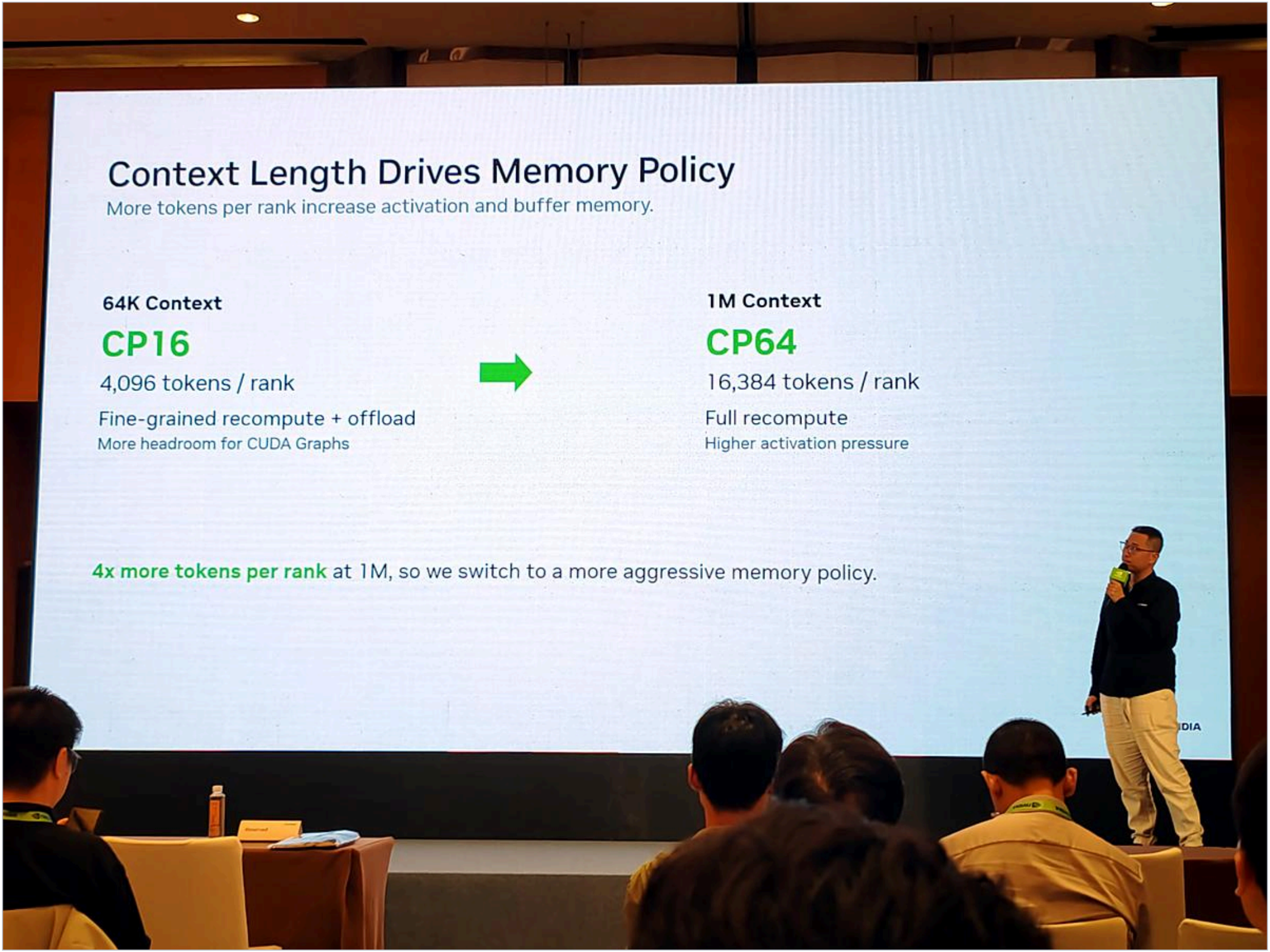

DeepSeek-V4-Pro Overview
MTP Modules
MTP Loss
Prediction Head
LM Loss
Transformer Block ×L
Post-Block Mixing
DeepSeekMoE
Residual Mixing
Pre-Block Mixing
Post-Block Mixing
CSA / HCA
Residual Mixing
Pre-Block Mixing
Embedding
Input Tokens
The model graph is from DeepSeek-V4 tech report.
1.6T
Total parameters
49B
Activated parameters per token
Pre-training recipe:
4K dense
->
16K dense
->
64K dense/sparse
->
1M sparse


Model Features Drive Optimizations
MTP Modules
MTP Loss
Prediction Head
LM Loss
Transformer Block ×L
Post-Block Mixing
DeepSeekMoE
Residual Mixing
Pre-Block Mixing
Post-Block Mixing
Indexer
CSA / HCA
Residual Mixing
Pre-Block Mixing
Embedding
Input Tokens
The model graph is from DeepSeek-V4 tech report.
1. 1.6T total / 49B active: Model states limit memory headroom for faster execution.
2. Hybrid attention: Different attention paths require tailored kernel optimizations.
3. Fine-grained MoE: Expert compute, communication, and buffers drive performance.
4. Causal long context: Equal token counts can still m
unequal workloads.
NVIDIA

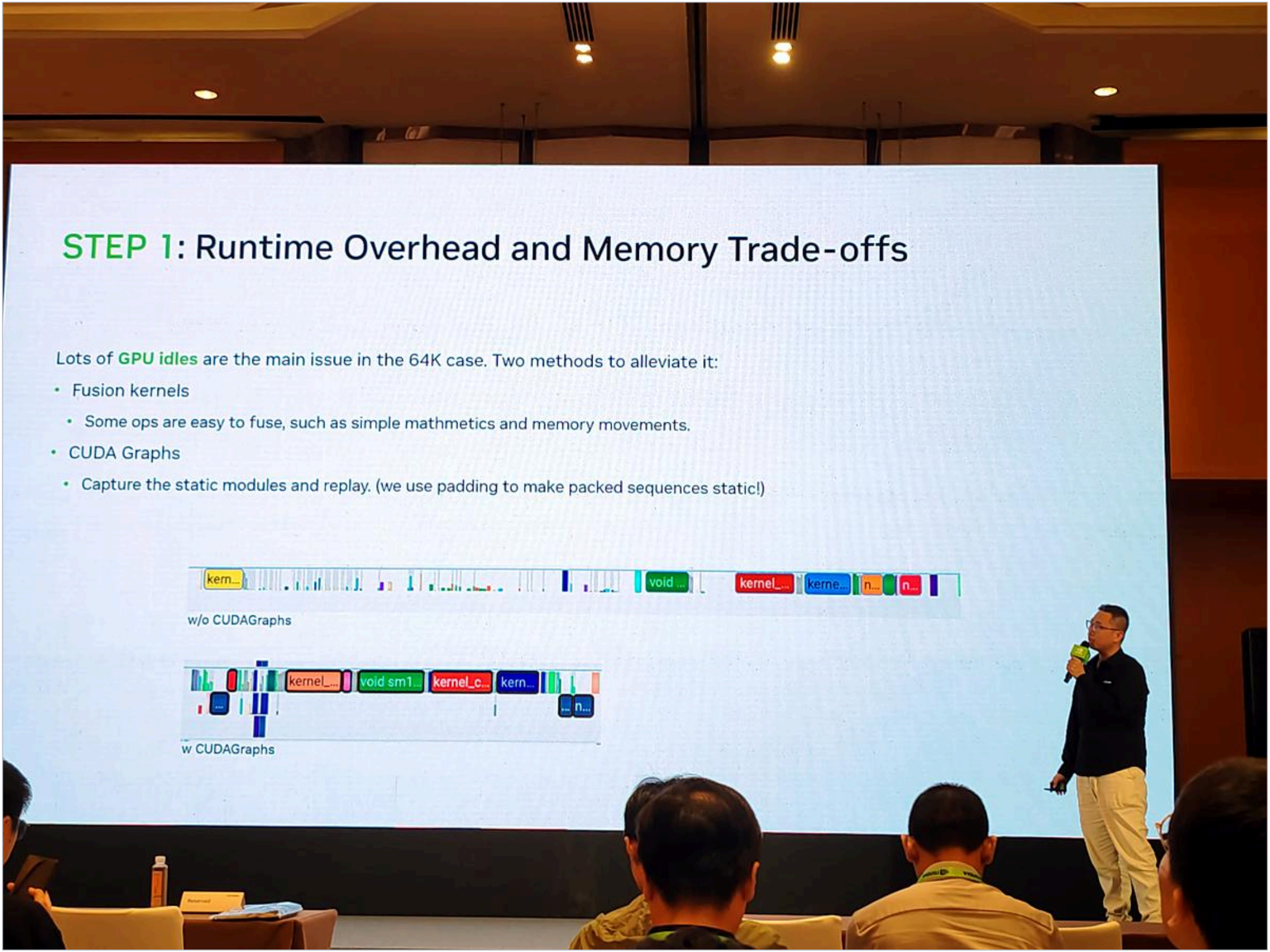

Choose the Parallel Layout
General Guidance on Parallel Tuning
1. Model size guides the initial layout.
• We can estimate the static part budget and choose an initial layout.
• Reserve headroom for activations and buffers.
• Prefer EP for experts, use PP across domains, and add TP if necessary.
2. Context length guides CP.
• Context length affects the activation and buffer size.
3. Topology guides placement.
• Keep communication-heavy groups (such as EP, TP, and CP) within NVLink domains where practical for better throughput.
For our 256-GPU DeepSeek-V4-Pro runs, we use PP4 and EP64 as the initial layout.


Choose the Parallel Layout
General Guidance on Parallel Tuning
1. Model size guides the initial layout.
• We can estimate the static part budget and choose an initial layout.
• Reserve headroom for activations and buffers.
• Prefer EP for experts, use PP across domains, and add TP if necessary.
2. Context length guides CP.
• Context length affects the activation and buffer size.
3. Topology guides placement.
• Keep communication-heavy groups (such as EP, TP, and CP) within NVLink domains where practical for better throughput.
For our 256-GPU DeepSeek-V4-Pro runs, we use PP4 and EP64 as the initial layout.

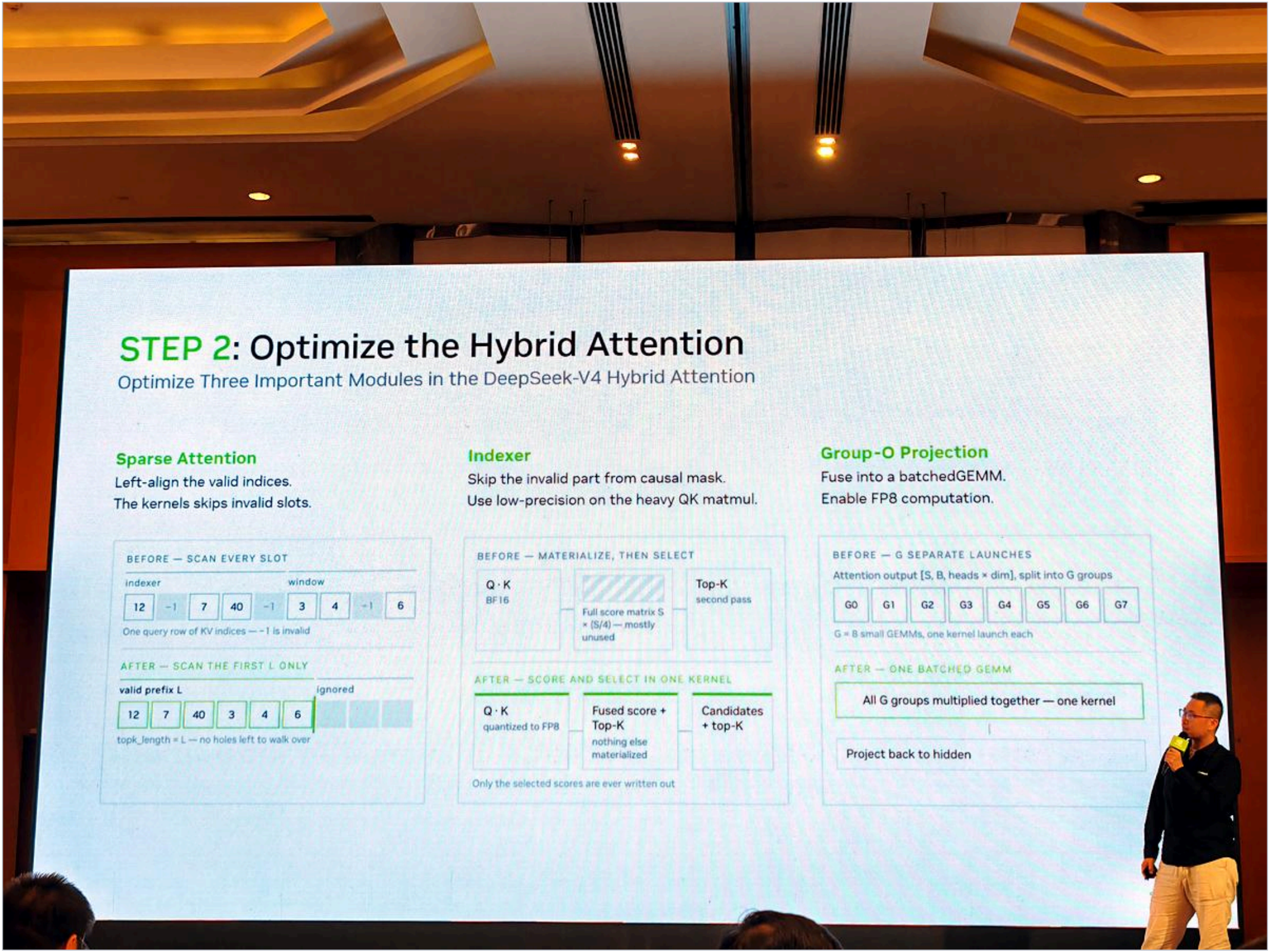

Context Length Drives Memory Policy
More tokens per rank increase activation and buffer memory.
64K Context
CP16
4,096 tokens / rank
Fine-grained recompute + offload
More headroom for CUDA Graphs
1M Context
CP64
16,384 tokens / rank
Full recompute
Higher activation pressure
4x more tokens per rank at 1M, so we switch to a more aggressive memory policy.

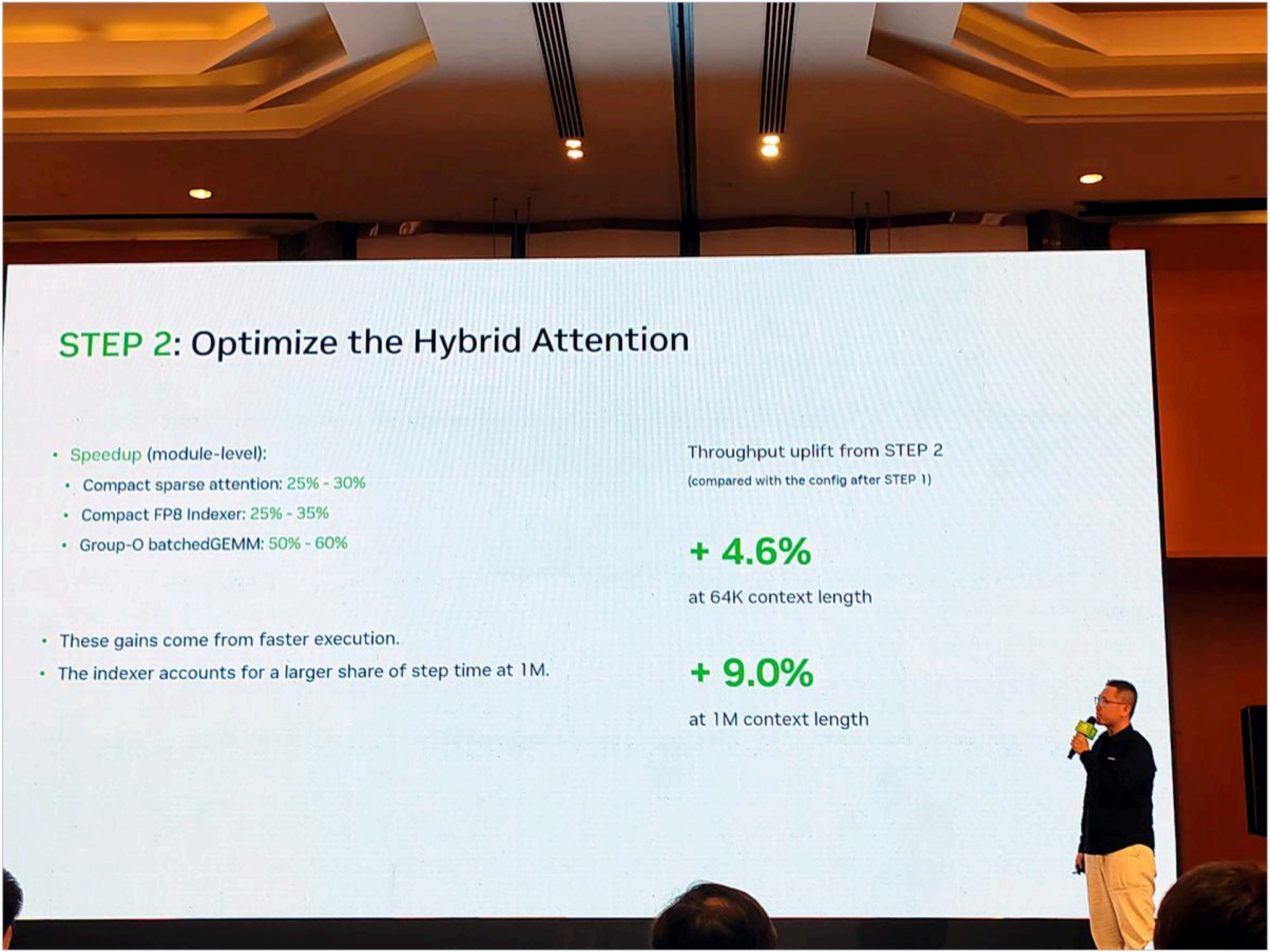

Context Length Drives Memory Policy
More tokens per rank increase activation and buffer memory.
64K Context
CP16
4,096 tokens / rank
Fine-grained recompute + offload
More headroom for CUDA Graphs
1M Context
CP64
16,384 tokens / rank
Full recompute
Higher activation pressure
4x more tokens per rank at 1M, so we switch to a more aggressive memory policy.


The Starting Points
We use a 4K-context sparse proxy model to basically verify the needed features, including:
• Kernel fusions, packed sequence support, CUDA Graphs support, and others
Use profiles to guide the following optimizations (both are within a 150ms window)
64K
CUDA HW (0008:06:00.0 - N
Kernel
Memory
[All Streams]
1M
CUDA HW (0008:06:00.0 - N
Kernel
Memory
nvjet_sm103_tst_
nvjet_sm10...
nvjet_sm103_tss_
[All Streams]


STEP 1: Runtime Overhead and Memory Trade-offs
Lots of GPU idles are the main issue in the 64K case. Two methods to alleviate it:
• Fusion kernels
• Some ops are easy to fuse, such as simple mathmetics and memory movements.
• CUDA Graphs
• Capture the static modules and replay. (we use padding to make packed sequences static!)
kern...
void ...
kernel_...
kerne...
n...
n...
w/o CUDAGraphs
kernel_...
void sm1...
kernel_c...
kern...
n...
w CUDAGraphs


Where the Step Time Goes
Layer time decomposition
Percent %
100
90
80
70
60
50
40
30
20
10
0
Attn
MoE
Attn
MoE
Attn
MoE
4K
4K is a sparse proxy case for comparison
64K
1M
Context Length
MoE experts
EP comms
Indexer + CP comm
Sparse Attn
Group-O Proj
Others


STEP 2: Optimize the Hybrid Attention
Optimize Three Important Modules in the DeepSeek-V4 Hybrid Attention
Sparse Attention
Left-align the valid indices.
The kernels skips invalid slots.
BEFORE — SCAN EVERY SLOT
indexer
window
12 -1 7 40 -1 3 4 -1 6
One query row of KV indices — -1 is invalid
AFTER — SCAN THE FIRST L ONLY
valid prefix L
ignored
12 7 40 3 4 6
topk_length = L — no holes left to walk over
Indexer
Skip the invalid part from causal mask.
Use low-precision on the heavy QK matmul.
BEFORE — MATERIALIZE, THEN SELECT
Q · K
BF16
Full score matrix S × (S/4) — mostly unused
Top-K
second pass
AFTER — SCORE AND SELECT IN ONE KERNEL
Q · K
quantized to FP8
Fused score + Top-K
nothing else materialized
Candidates + top-K
Only the selected scores are ever written out
Group-O Projection
Fuse into a batchedGEMM.
Enable FP8 computation.
BEFORE — G SEPARATE LAUNCHES
Attention output [S, B, heads × dim], split into G groups
G0 G1 G2 G3 G4 G5 G6 G7
G = 8 small GEMMs, one kernel launch each
AFTER — ONE BATCHED GEMM
All G groups multiplied together — one kernel
Project back to hidden


STEP 2: Optimize the Hybrid Attention
• Speedup (module-level):
• Compact sparse attention: 25% - 30%
• Compact FP8 Indexer: 25% - 35%
• Group-O batchedGEMM: 50% - 60%
• These gains come from faster execution.
• The indexer accounts for a larger share of step time at 1M.
Throughput uplift from STEP 2
(compared with the config after STEP 1)
+ 4.6%
at 64K context length
+ 9.0%
at 1M context length

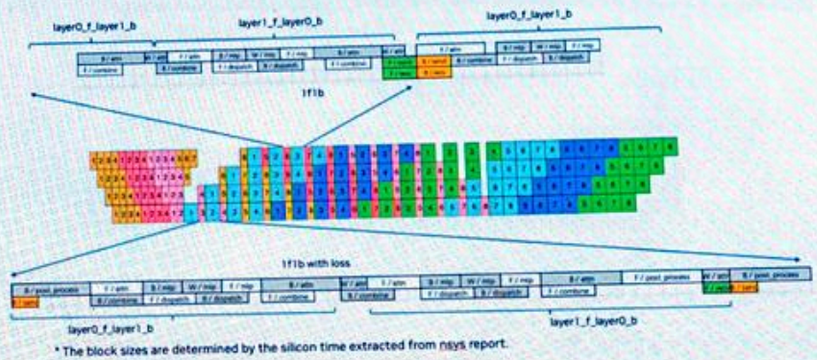
STEP 3: Optimize the MoE

About 30% - 50% of the MoE part is EP communication.

We want to hide the communication behind computation.

Two solutions now:

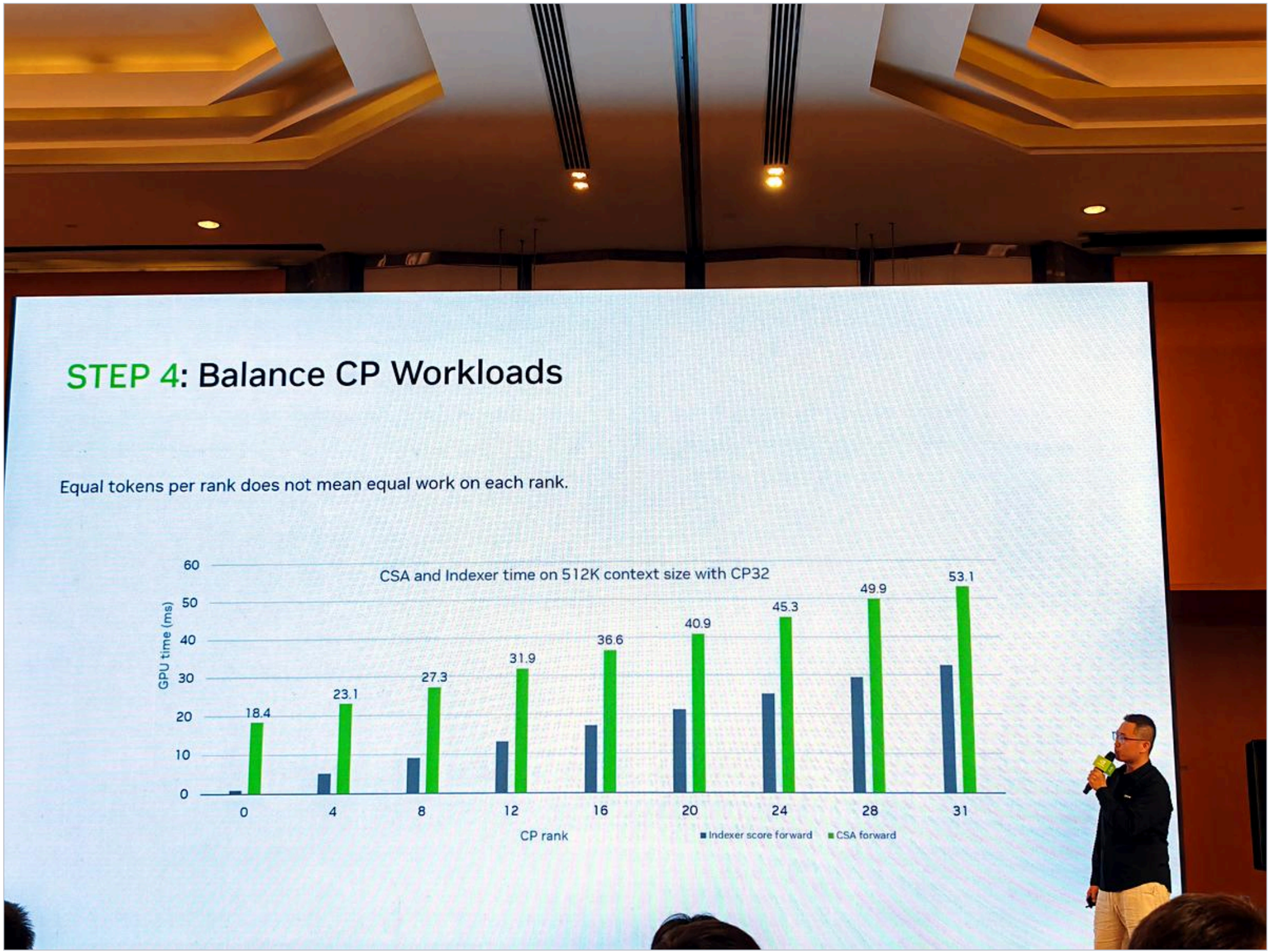
1. Cross-batch EP overlapping
   - More suitable for cross-NVLink domain scenarios
   - Increase memory overhead
2. Fine-grained overlapping with MegaKernel
   1. Relatively easier to fuse inside NVLink domain
   2. Better utilize the memory

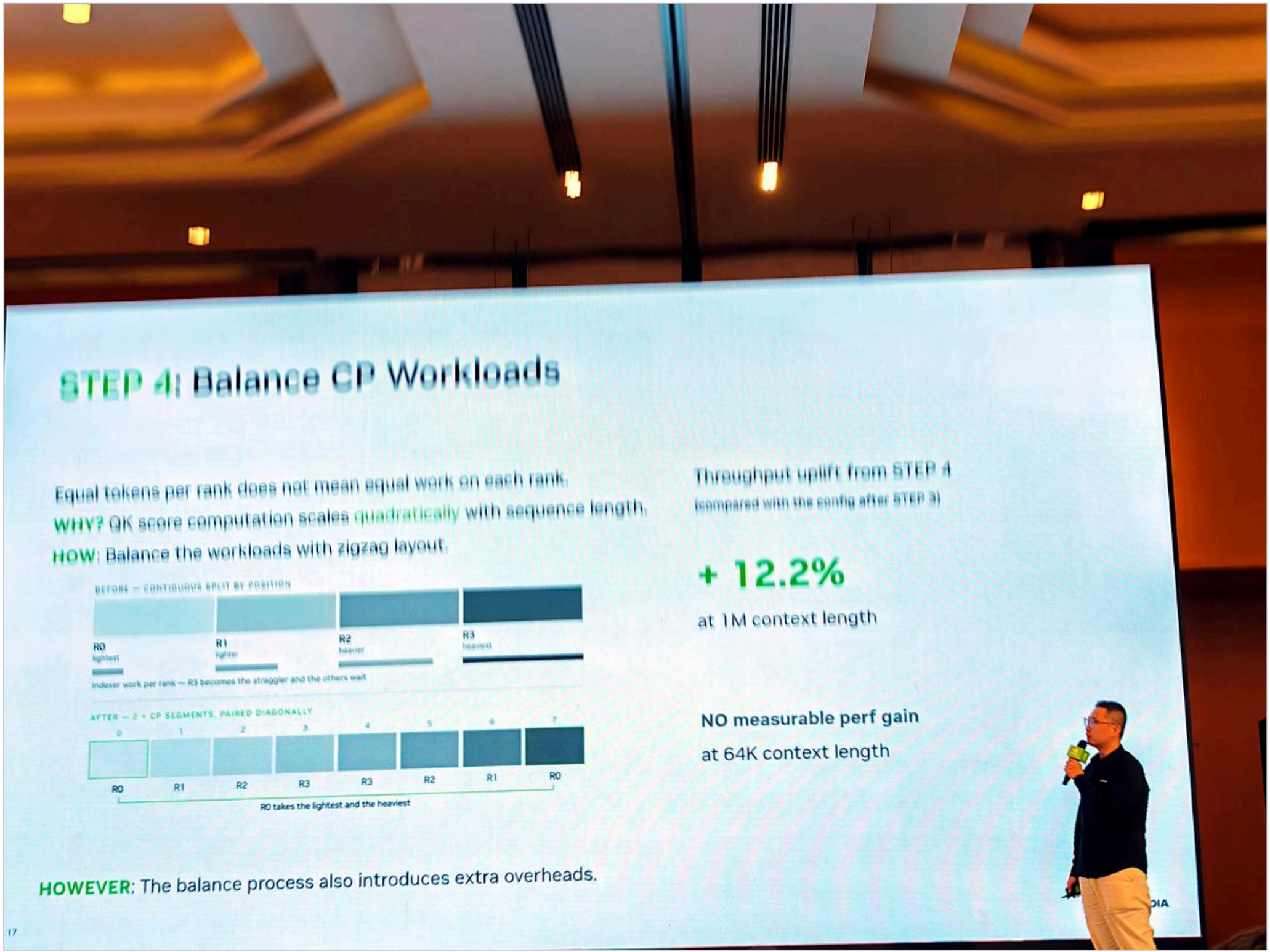


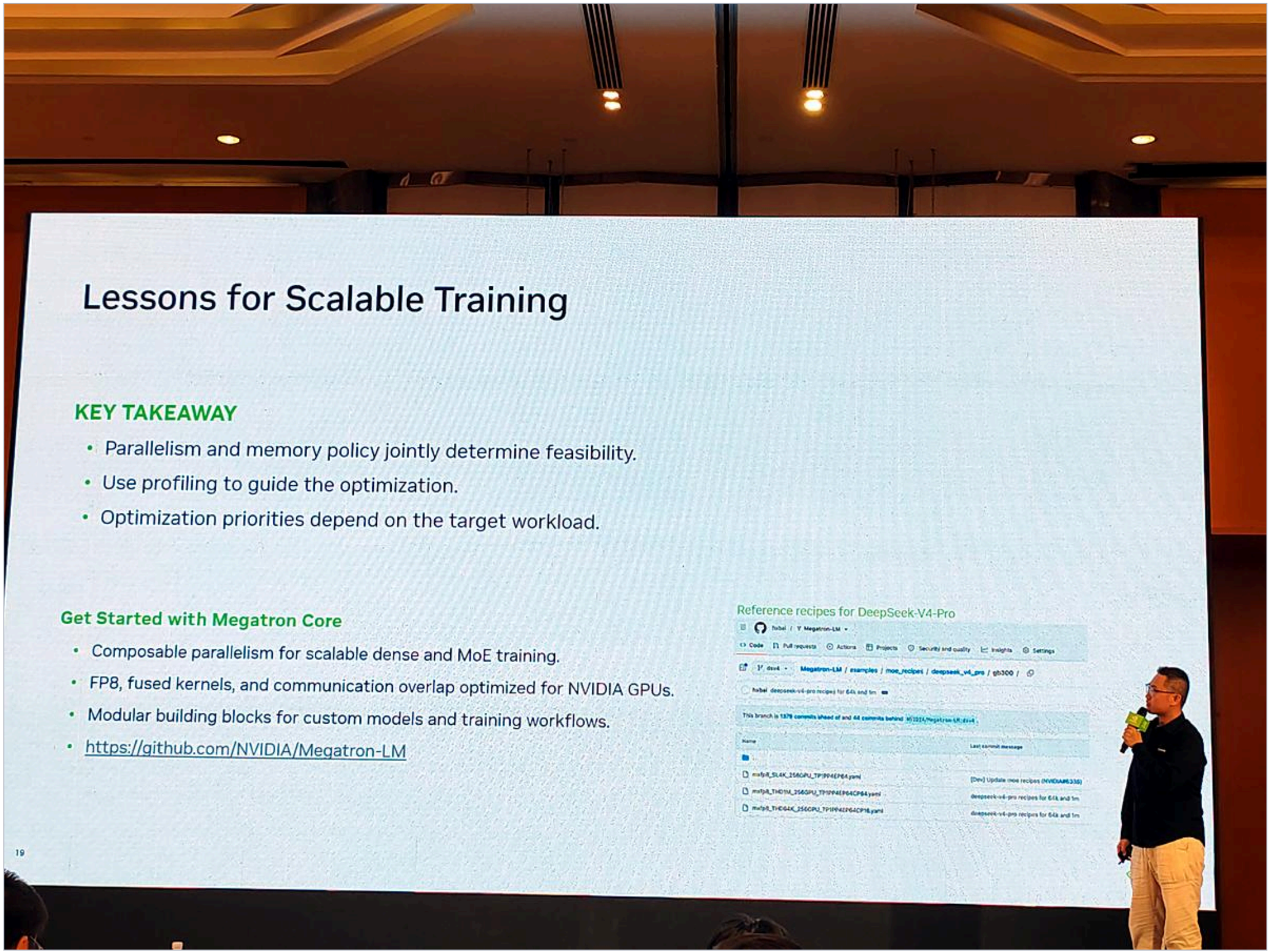


The graph below is from the MOK blog from Cursor.

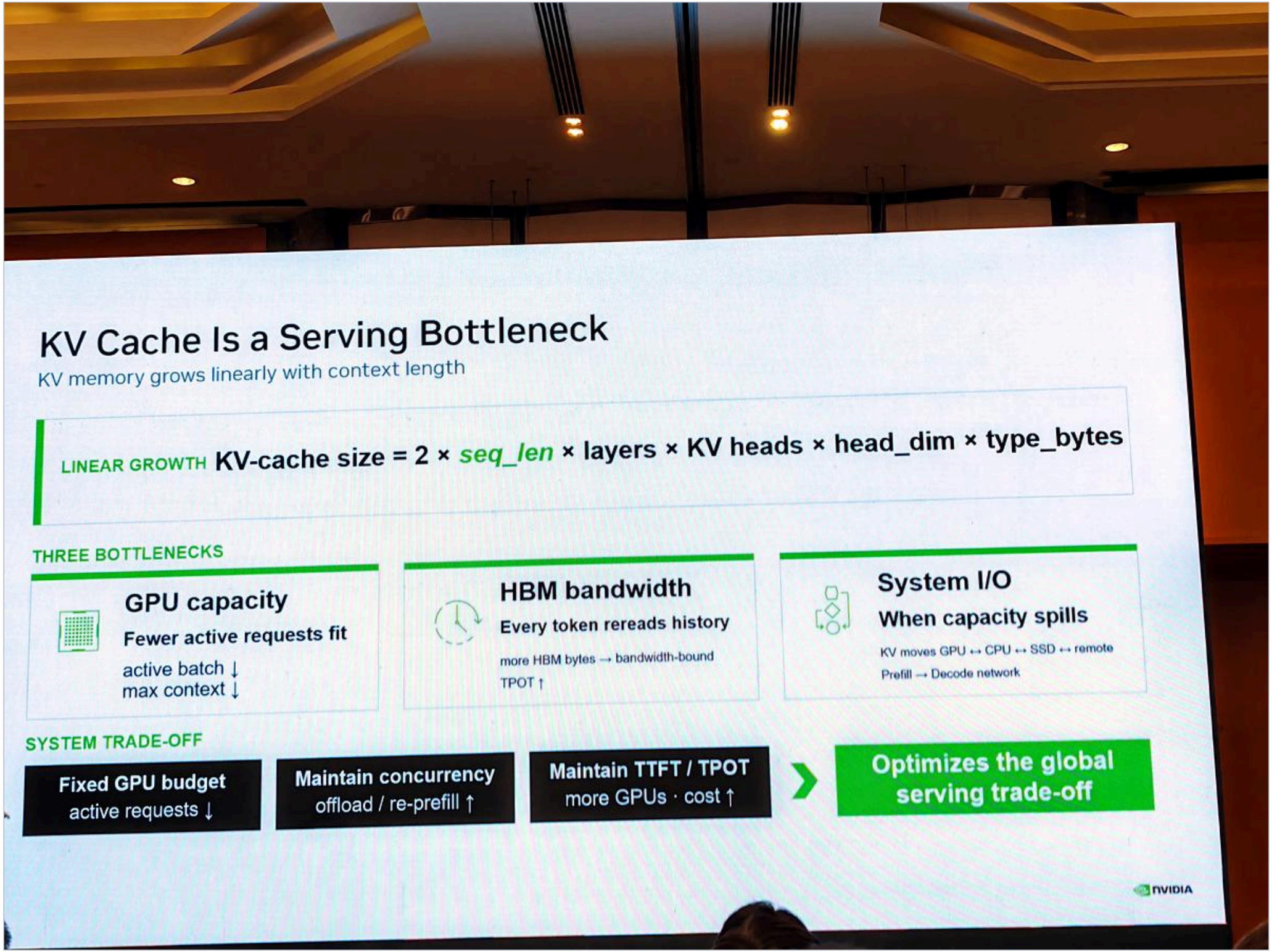

STEP 3: Optimize the MoE
Mixture-of-Kittens (MOK) from Cursor
1. Fused execution
• Combine expert computation and communication in one kernel.
2. Communication overlap
• Pipeline token transfers with expert computation.
3. Buffer reuse
• Reuse fixed-size ring buffers to reduce memory usage and avoid CPU synchronization for buffer allocation.
MOK can bring 30% -40% speedup on the MoE part in the DeepSeek-V4-Pro setting.
Throughput uplift from STEP 3
(compared with the config after STEP 2)
+ 14.0%
at 64K context length
+ 4.6%
at 1M context length

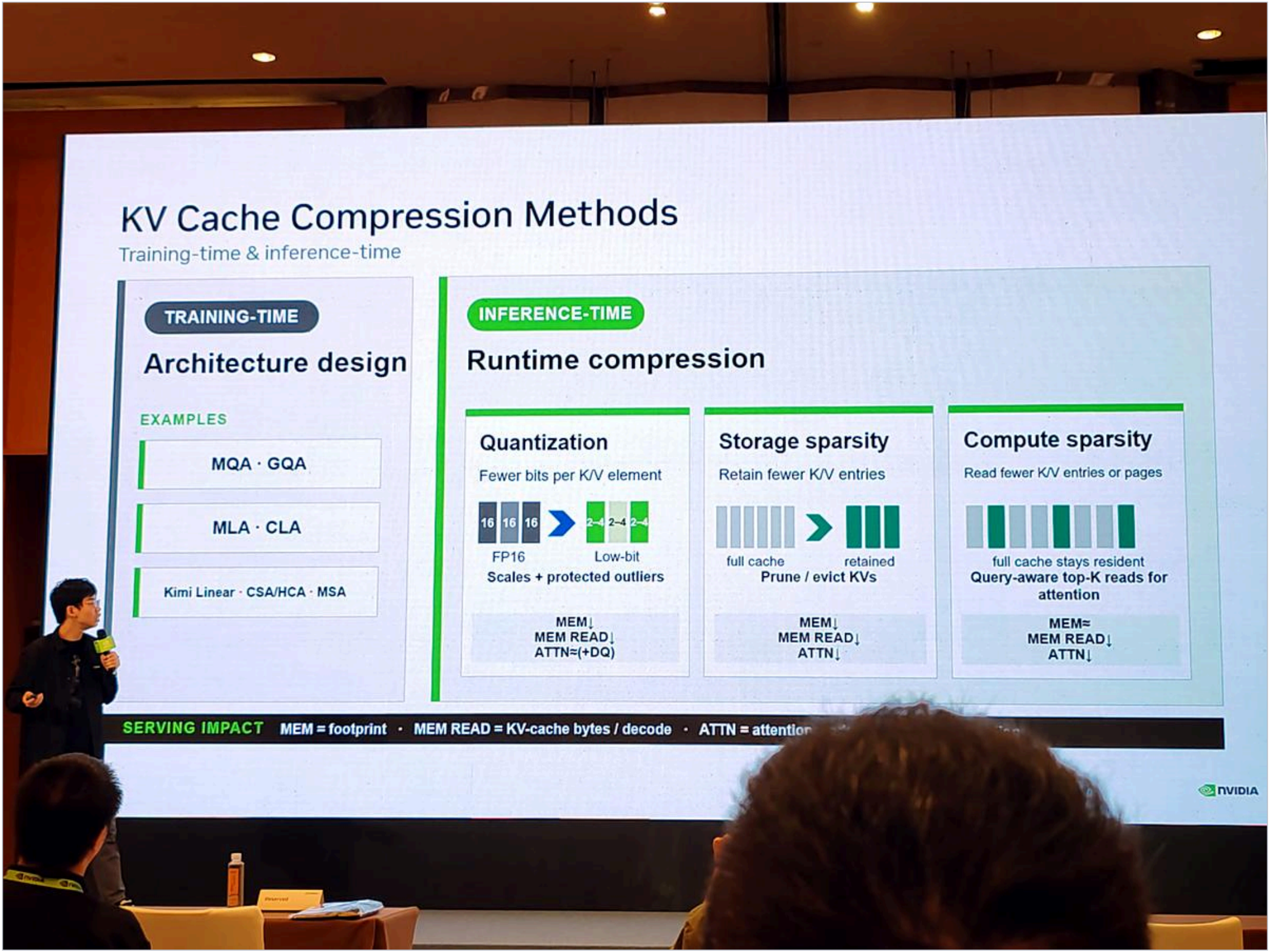

STEP 4: Balance CP Workloads
Equal tokens per rank does not mean equal work on each rank.
CSA and Indexer time on 512K context size with CP32
GPU time (ms)
60
50
40
30
20
10
0
18.4
23.1
27.3
31.9
36.6
40.9
45.3
49.9
53.1
0
4
8
12
16
20
24
28
31
CP rank
Indexer score forward
CSA forward

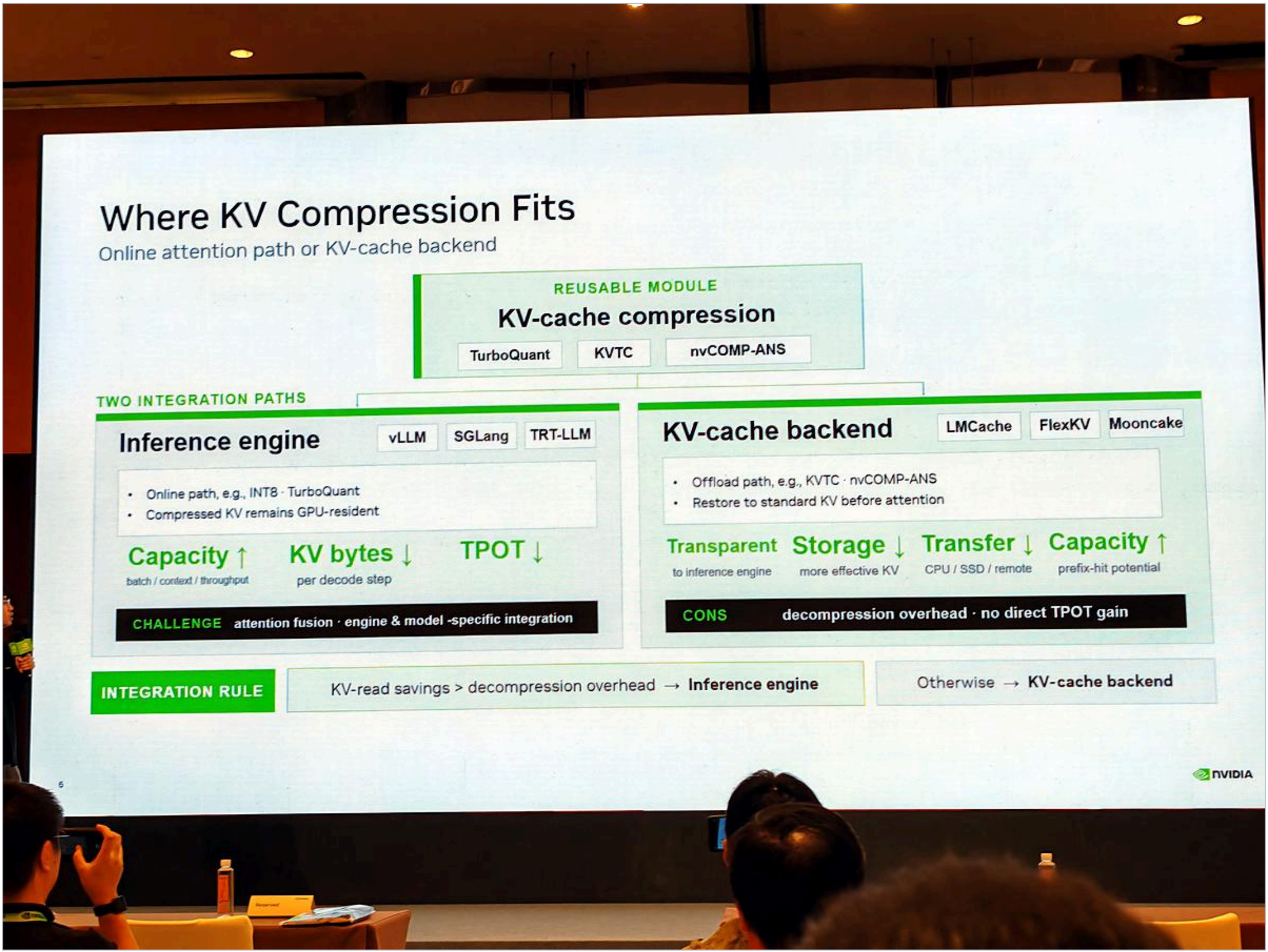

STEP 4: Balance CP Workloads
Equal tokens per rank does not mean equal work on each rank.
WHY? QK score computation scales quadratically with sequence length.
HOW: Balance the workloads with zigzag layout.
BEFORE — CONTIGUOUS SPLIT BY POSITION
R0 lightest
R1 lighter
R2 heavier
R3 heaviest
Indexer work per rank — R3 becomes the straggler and the others wait
AFTER — 2 × CP SEGMENTS, PAIRED DIAGONALLY
0 1 2 3 4 5 6 7
R0 R1 R2 R3 R3 R2 R1 R0
R0 takes the lightest and the heaviest
Throughput uplift from STEP 4
(compared with the config after STEP 3)
+ 12.2%
at 1M context length
NO measurable perf gain
at 64K context length
HOWEVER: The balance process also introduces extra overheads.
17

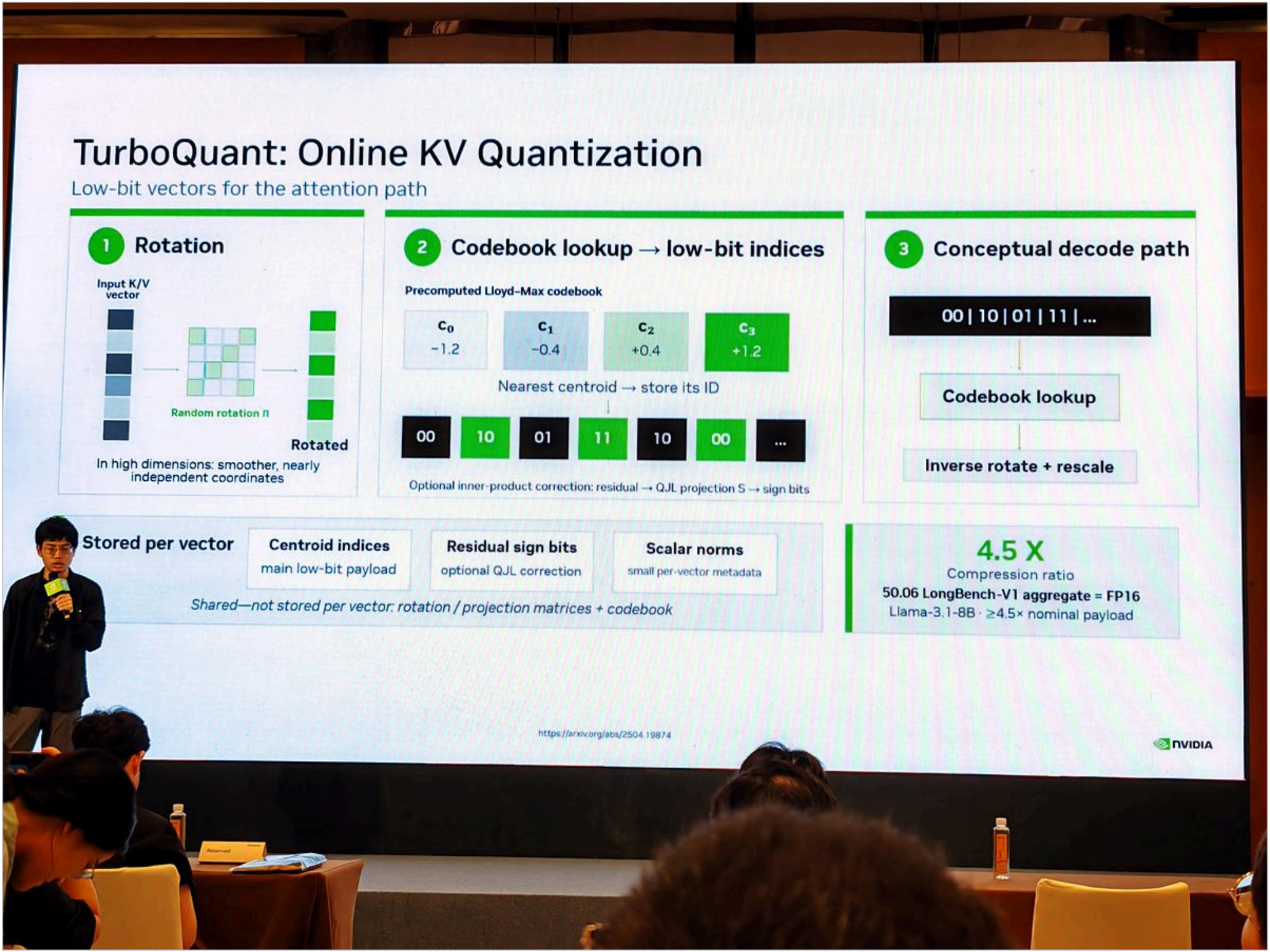

Lessons for Scalable Training
KEY TAKEAWAY
• Parallelism and memory policy jointly determine feasibility.
• Use profiling to guide the optimization.
• Optimization priorities depend on the target workload.
Get Started with Megatron Core
• Composable parallelism for scalable dense and MoE training.
• FP8, fused kernels, and communication overlap optimized for NVIDIA GPUs.
• Modular building blocks for custom models and training workflows.
• https://github.com/NVIDIA/Megatron-LM
Reference recipes for DeepSeek-V4-Pro
19

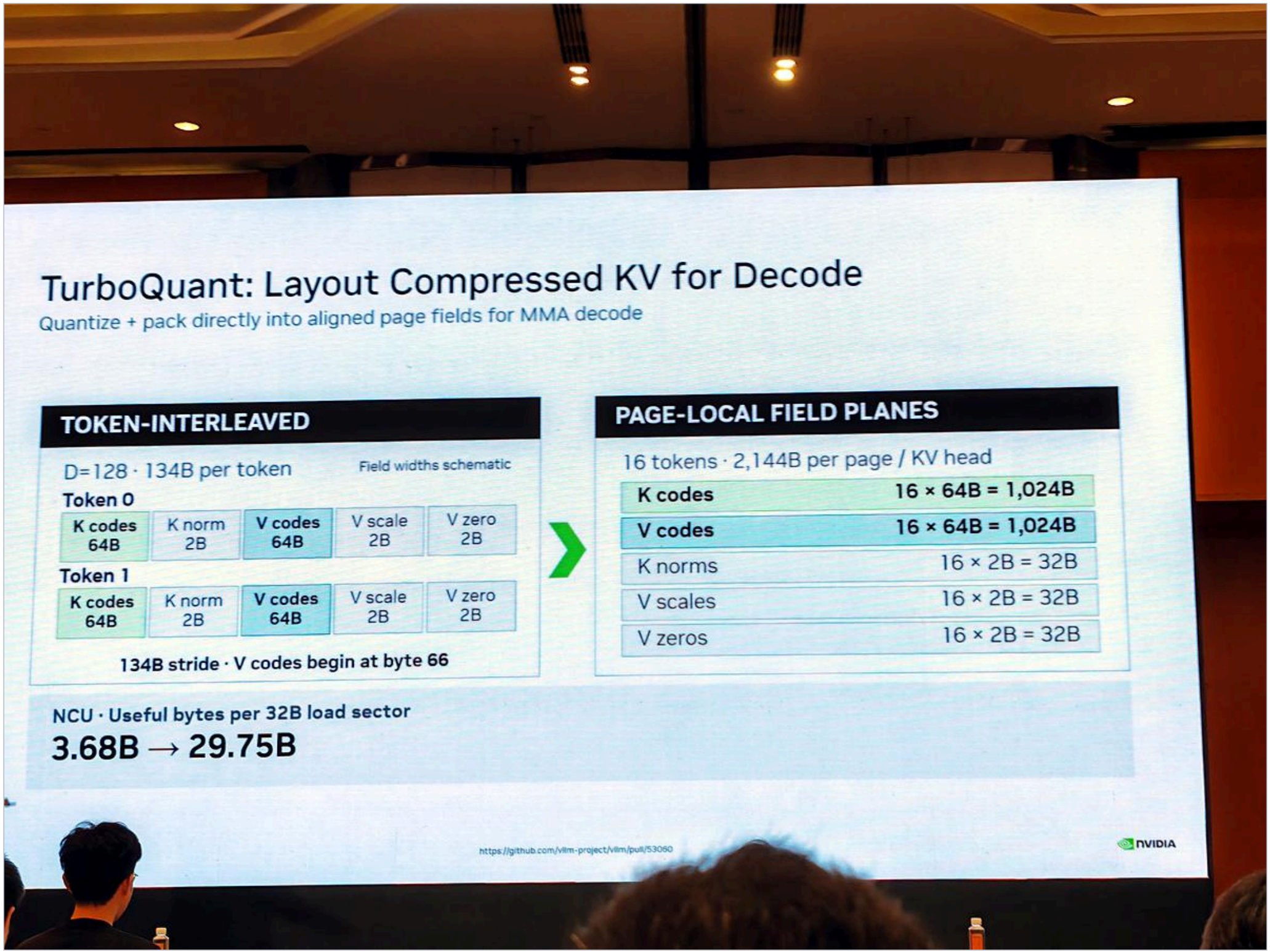

Agenda
Motivation: KV Cache as a Serving Bottleneck
Taxonomy: Inference-time KV Cache Compression Patterns
Performance Analysis: TurboQuant & KVTC & nvCOMP
Takeaways

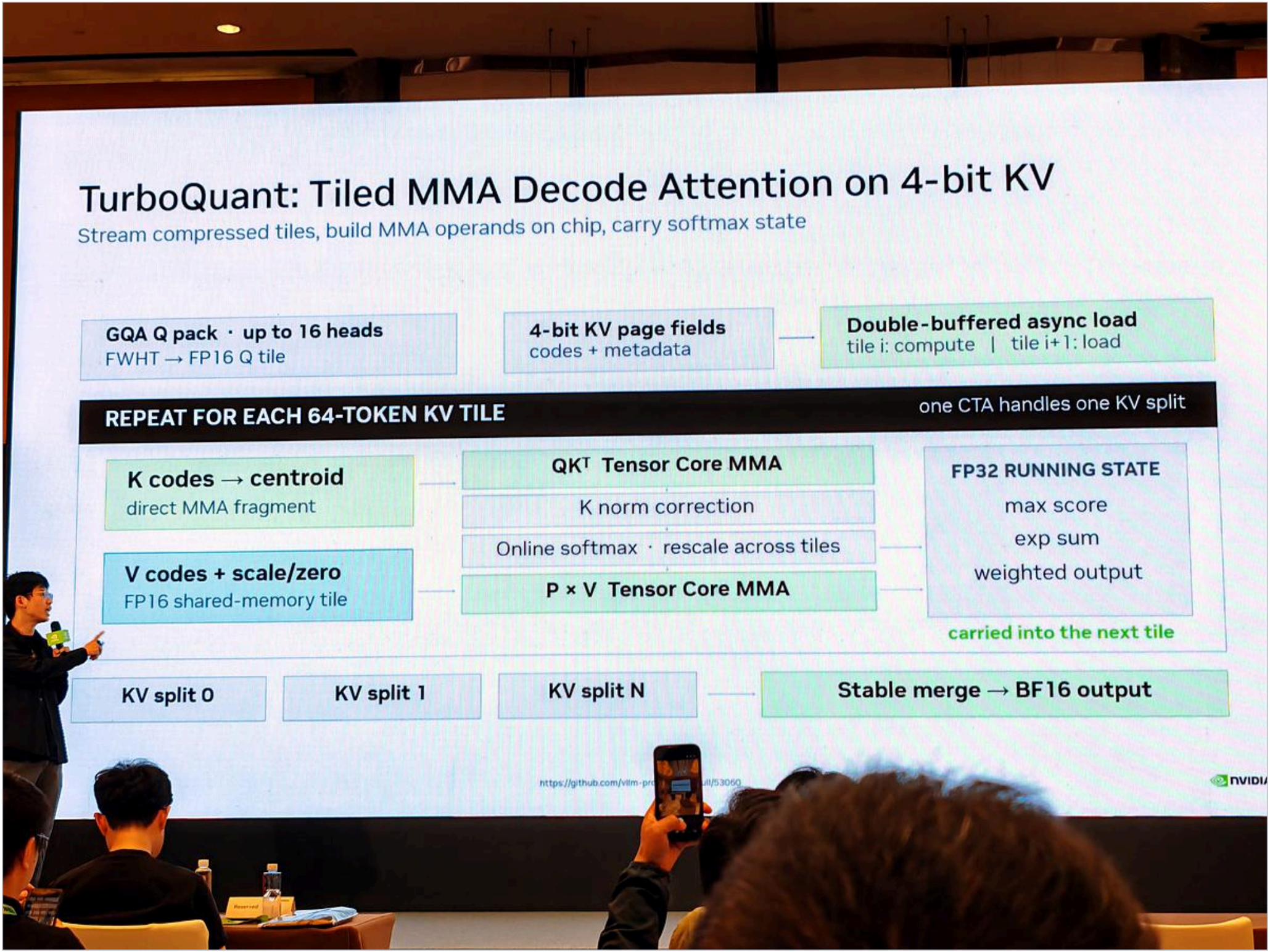

KV Cache Is a Serving Bottleneck
KV memory grows linearly with context length
LINEAR GROWTH KV-cache size = 2 × seq_len × layers × KV heads × head_dim × type_bytes
THREE BOTTLENECKS
GPU capacity
Fewer active requests fit
active batch ↓
max context ↓
HBM bandwidth
Every token rereads history
more HBM bytes → bandwidth-bound
TPOT ↑
System I/O
When capacity spills
KV moves GPU ↔ CPU ↔ SSD ↔ remote
Prefill → Decode network
SYSTEM TRADE-OFF
Fixed GPU budget
active requests ↓
Maintain concurrency
offload / re-prefill ↑
Maintain TTFT / TPOT
more GPUs · cost ↑
Optimizes the global serving trade-off
NVIDIA

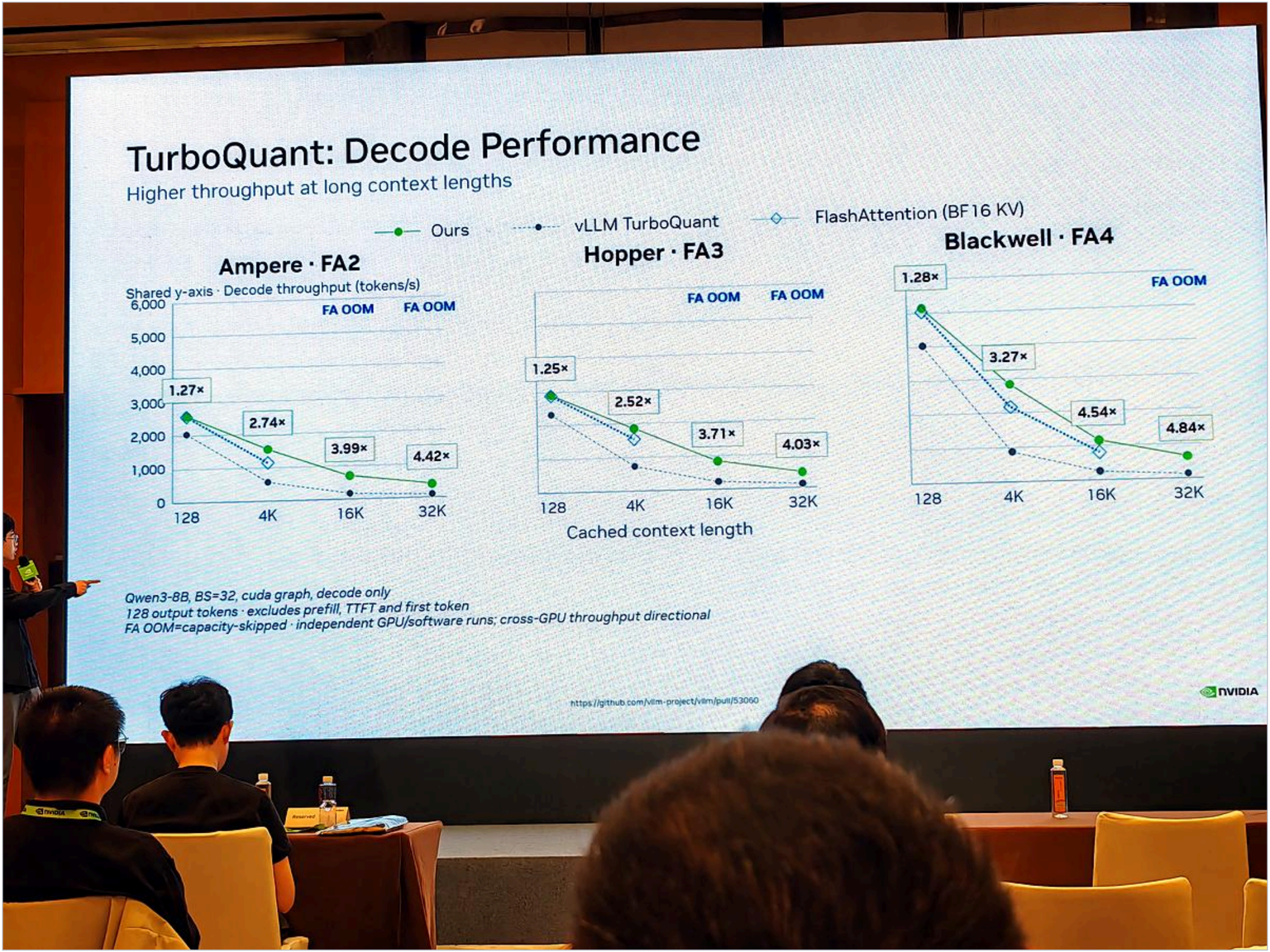

KV Cache Compression Methods
Training-time & inference-time
TRAINING-TIME
Architecture design
EXAMPLES
MQA · GQA
MLA · CLA
Kimi Linear · CSA/HCA · MSA
INFERENCE-TIME
Runtime compression
Quantization
Fewer bits per K/V element
16 16 16
2–4 2–4 2–4
FP16
Low-bit
Scales + protected outliers
MEM↓
MEM READ↓
ATTN≈(+DQ)
Storage sparsity
Retain fewer K/V entries
full cache
retained
Prune / evict KVs
MEM↓
MEM READ↓
ATTN↓
Compute sparsity
Read fewer K/V entries or pages
full cache stays resident
Query-aware top-K reads for attention
MEM≈
MEM READ↓
ATTN↓
SERVING IMPACT MEM = footprint · MEM READ = KV-cache bytes / decode · ATTN = attention
NVIDIA

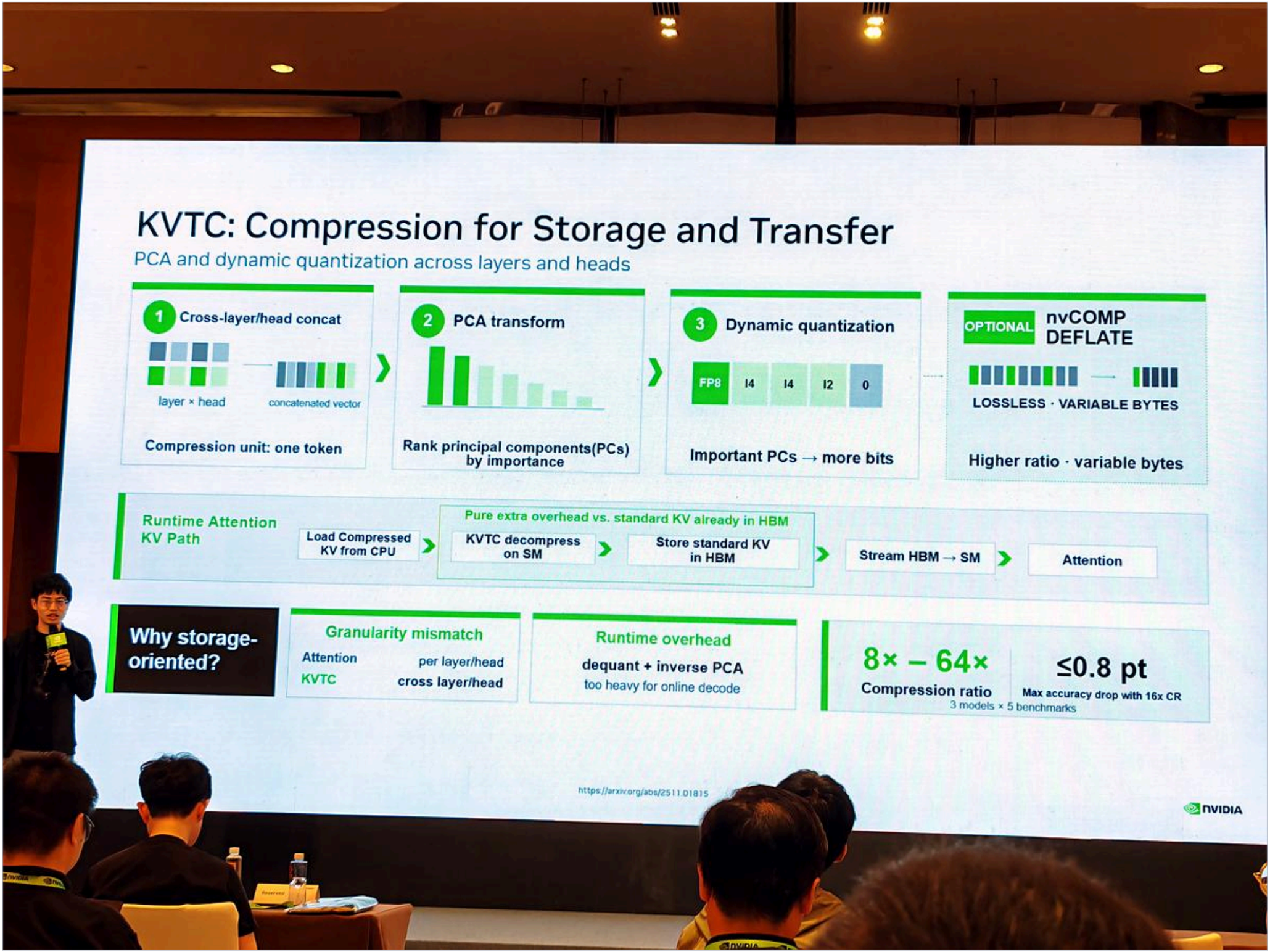

Where KV Compression Fits
Online attention path or KV-cache backend
REUSABLE MODULE
KV-cache compression
TurboQuant
KVTC
nvCOMP-ANS
TWO INTEGRATION PATHS
Inference engine
vLLM
SGLang
TRT-LLM
Online path, e.g., INT8 · TurboQuant
Compressed KV remains GPU-resident
Capacity ↑
batch / context / throughput
KV bytes ↓
per decode step
TPOT ↓
CHALLENGE attention fusion · engine & model -specific integration
KV-cache backend
LMCache
FlexKV
Mooncake
Offload path, e.g., KVTC · nvCOMP-ANS
Restore to standard KV before attention
Transparent
to inference engine
Storage ↓
more effective KV
Transfer ↓
CPU / SSD / remote
Capacity ↑
prefix-hit potential
CONS decompression overhead · no direct TPOT gain
INTEGRATION RULE
KV-read savings > decompression overhead → Inference engine
Otherwise → KV-cache backend
6
NVIDIA

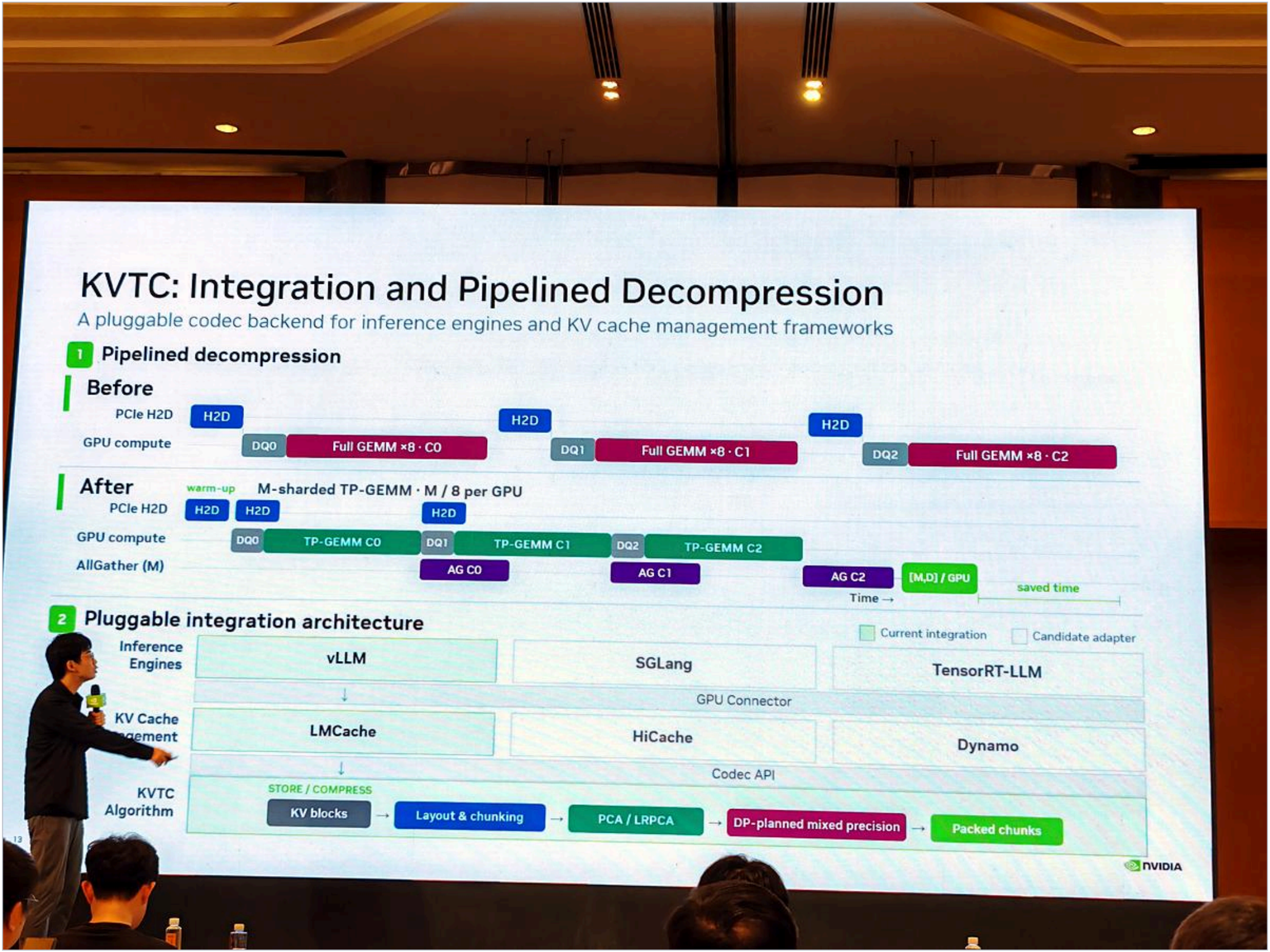

TurboQuant: Online KV Quantization
Low-bit vectors for the attention path
1 Rotation
Input K/V vector
Random rotation Π
Rotated
In high dimensions: smoother, nearly independent coordinates
2 Codebook lookup → low-bit indices
Precomputed Lloyd-Max codebook
c0 −1.2
c1 −0.4
c2 +0.4
c3 +1.2
Nearest centroid → store its ID
00 10 01 11 10 00 ...
Optional inner-product correction: residual → QJL projection S → sign bits
3 Conceptual decode path
00 | 10 | 01 | 11 | ...
Codebook lookup
Inverse rotate + rescale
Stored per vector
Centroid indices
main low-bit payload
Residual sign bits
optional QJL correction
Scalar norms
small per-vector metadata
Shared—not stored per vector: rotation / projection matrices + codebook
4.5 X
Compression ratio
50.06 LongBench-V1 aggregate = FP16
Llama-3.1-8B · ≥4.5× nominal payload
https://arxiv.org/abs/2504.19874
NVIDIA

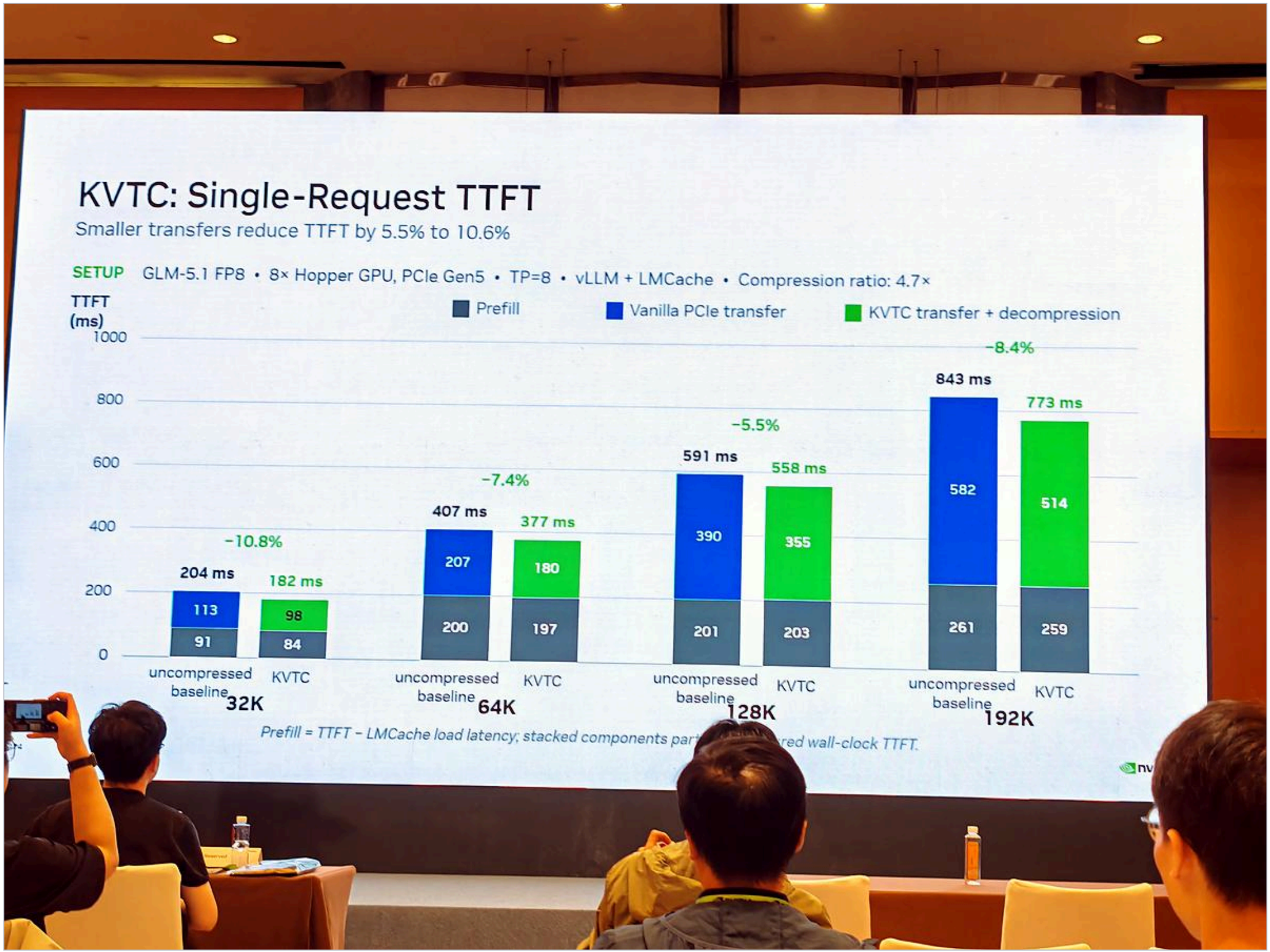

TurboQuant: Layout Compressed KV for Decode
Quantize + pack directly into aligned page fields for MMA decode
TOKEN-INTERLEAVED
D=128 · 134B per token
Field widths schematic
Token 0
K codes 64B
K norm 2B
V codes 64B
V scale 2B
V zero 2B
Token 1
K codes 64B
K norm 2B
V codes 64B
V scale 2B
V zero 2B
134B stride · V codes begin at byte 66
PAGE-LOCAL FIELD PLANES
16 tokens · 2,144B per page / KV head
K codes 16 × 64B = 1,024B
V codes 16 × 64B = 1,024B
K norms 16 × 2B = 32B
V scales 16 × 2B = 32B
V zeros 16 × 2B = 32B
NCU · Useful bytes per 32B load sector
3.68B → 29.75B
https://github.com/vllm-project/vllm/pull/53060
NVIDIA

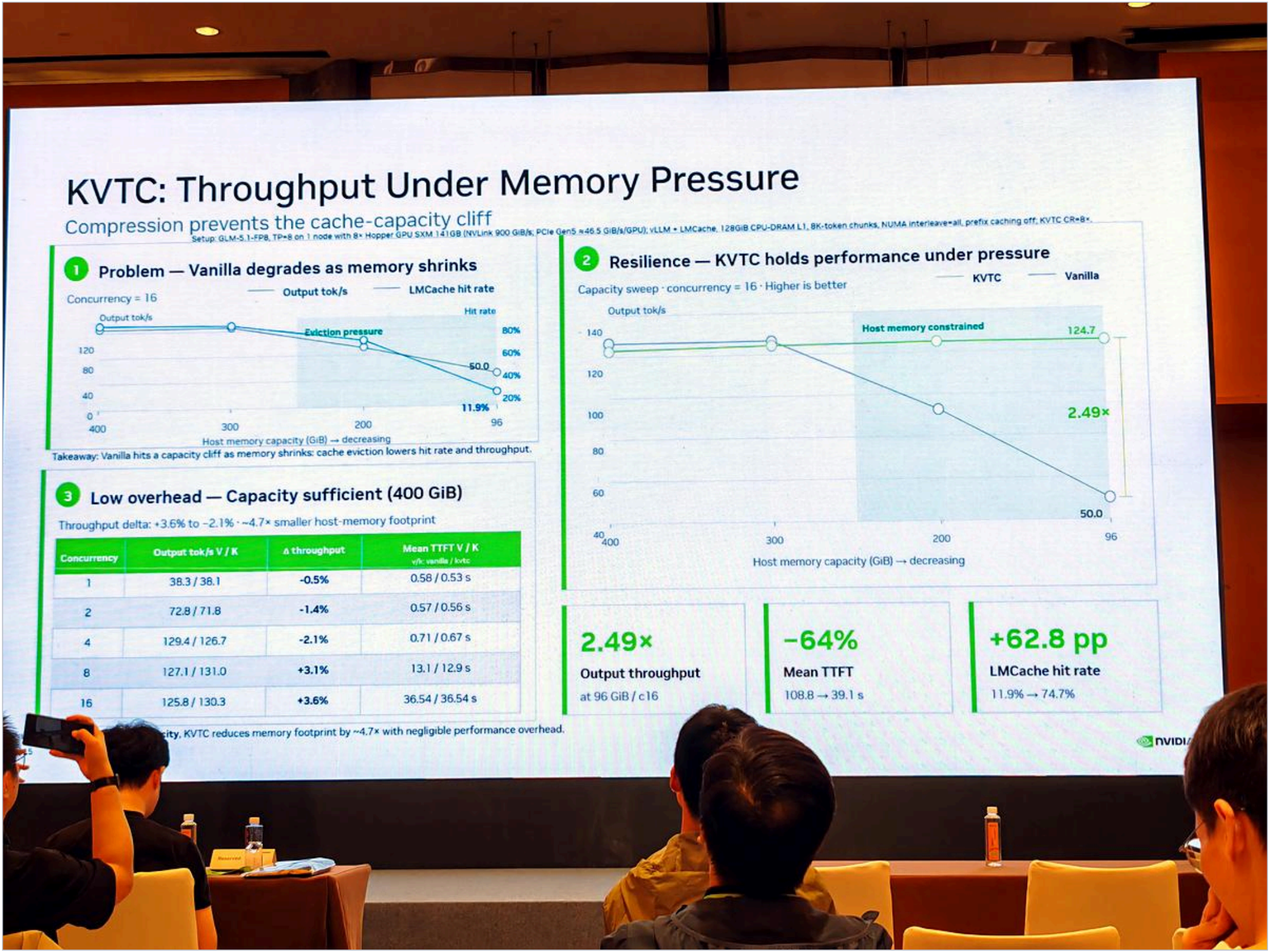

TurboQuant: Tiled MMA Decode Attention on 4-bit KV
Stream compressed tiles, build MMA operands on chip, carry softmax state
GQA Q pack · up to 16 heads
FWHT → FP16 Q tile
4-bit KV page fields
codes + metadata
Double-buffered async load
tile i: compute | tile i+1: load
REPEAT FOR EACH 64-TOKEN KV TILE
one CTA handles one KV split
K codes → centroid
direct MMA fragment
V codes + scale/zero
FP16 shared-memory tile
QKᵀ Tensor Core MMA
K norm correction
Online softmax · rescale across tiles
P × V Tensor Core MMA
FP32 RUNNING STATE
max score
exp sum
weighted output
carried into the next tile
KV split 0
KV split 1
KV split N
Stable merge → BF16 output
https://github.com/vllm-pr
ull/53060
NVIDIA

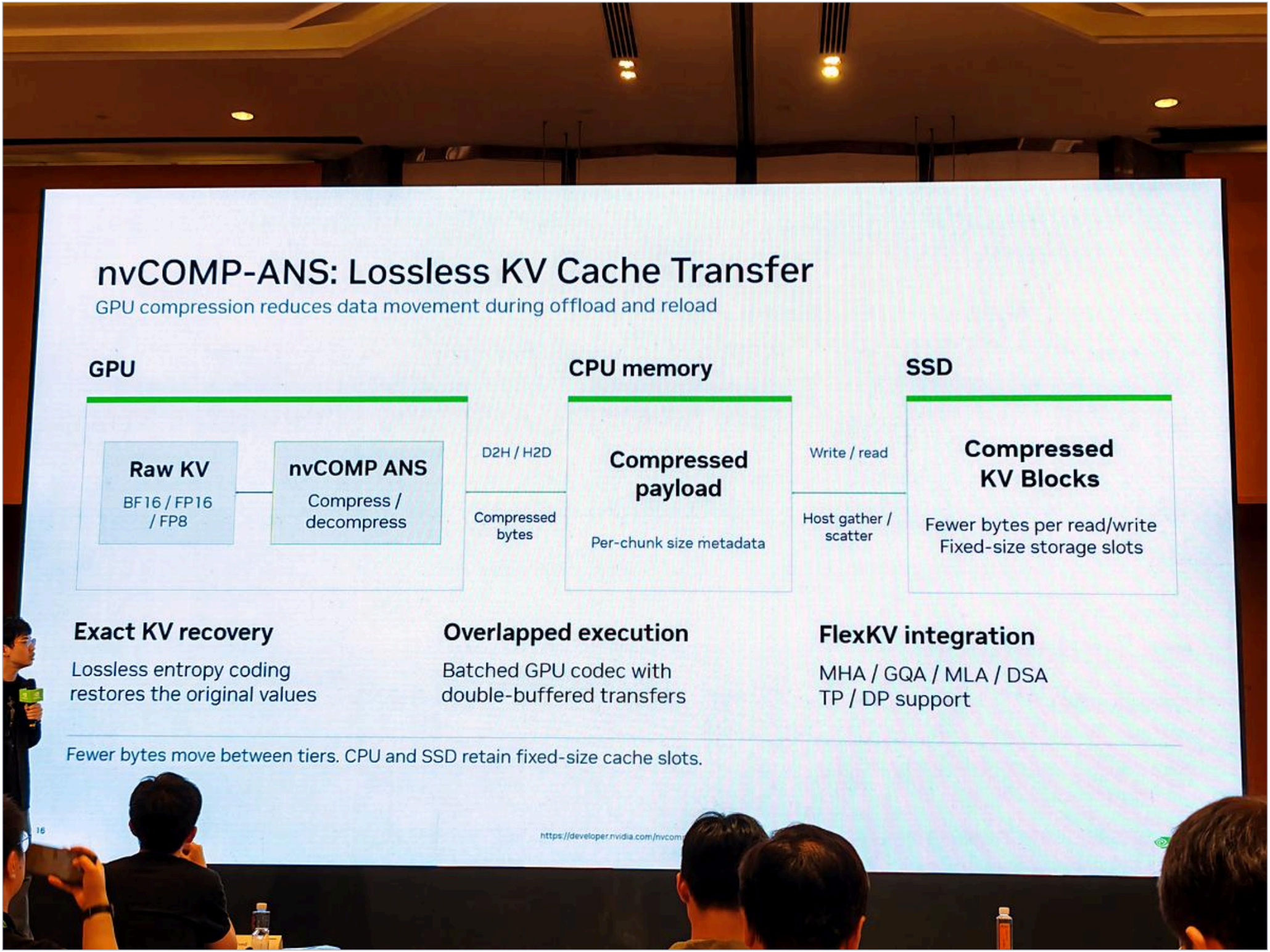

TurboQuant: Decode Performance
Higher throughput at long context lengths
Ours
vLLM TurboQuant
FlashAttention (BF16 KV)
Ampere · FA2
Shared y-axis · Decode throughput (tokens/s)
6,000
5,000
4,000
3,000
2,000
1,000
0
FA OOM
FA OOM
1.27×
2.74×
3.99×
4.42×
128
4K
16K
32K
Hopper · FA3
FA OOM
FA OOM
1.25×
2.52×
3.71×
4.03×
128
4K
16K
32K
Cached context length
Blackwell · FA4
1.28×
FA OOM
3.27×
4.54×
4.84×
128
4K
16K
32K
Qwen3-8B, BS=32, cuda graph, decode only
128 output tokens · excludes prefill, TTFT and first token
FA OOM=capacity-skipped · independent GPU/software runs; cross-GPU throughput directional
https://github.com/vllm-project/vllm/pull/53060
NVIDIA

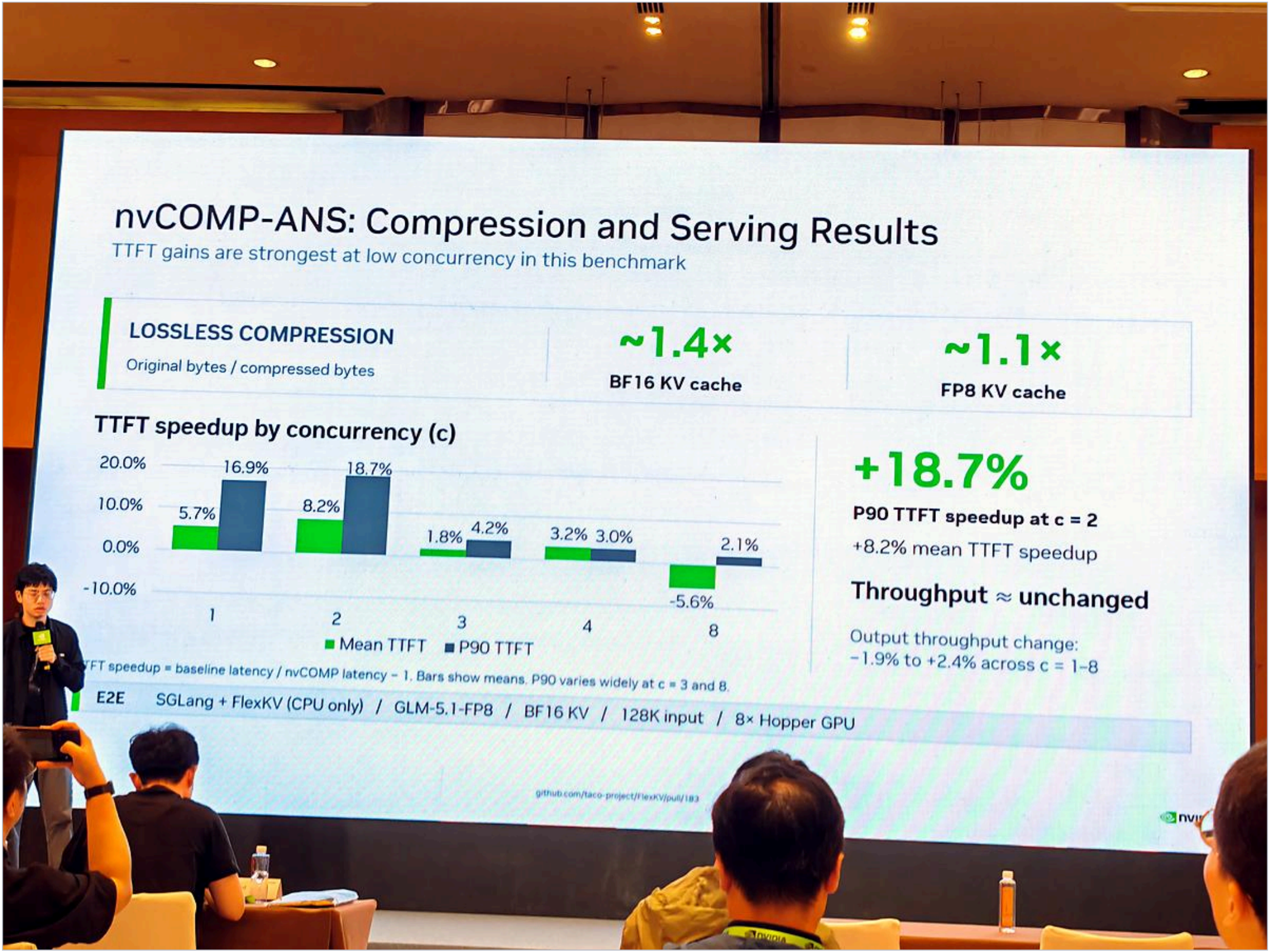

KVTC: Compression for Storage and Transfer
PCA and dynamic quantization across layers and heads
1 Cross-layer/head concat
layer × head
concatenated vector
Compression unit: one token
2 PCA transform
Rank principal components(PCs) by importance
3 Dynamic quantization
FP8
I4
I4
I2
0
Important PCs → more bits
OPTIONAL
nvCOMP DEFLATE
LOSSLESS · VARIABLE BYTES
Higher ratio · variable bytes
Runtime Attention KV Path
Load Compressed KV from CPU
Pure extra overhead vs. standard KV already in HBM
KVTC decompress on SM
Store standard KV in HBM
Stream HBM → SM
Attention
Why storage-oriented?
Granularity mismatch
Attention per layer/head
KVTC cross layer/head
Runtime overhead
dequant + inverse PCA
too heavy for online decode
8× – 64×
Compression ratio
3 models × 5 benchmarks
≤0.8 pt
Max accuracy drop with 16x CR
https://arxiv.org/abs/2511.01815
NVIDIA

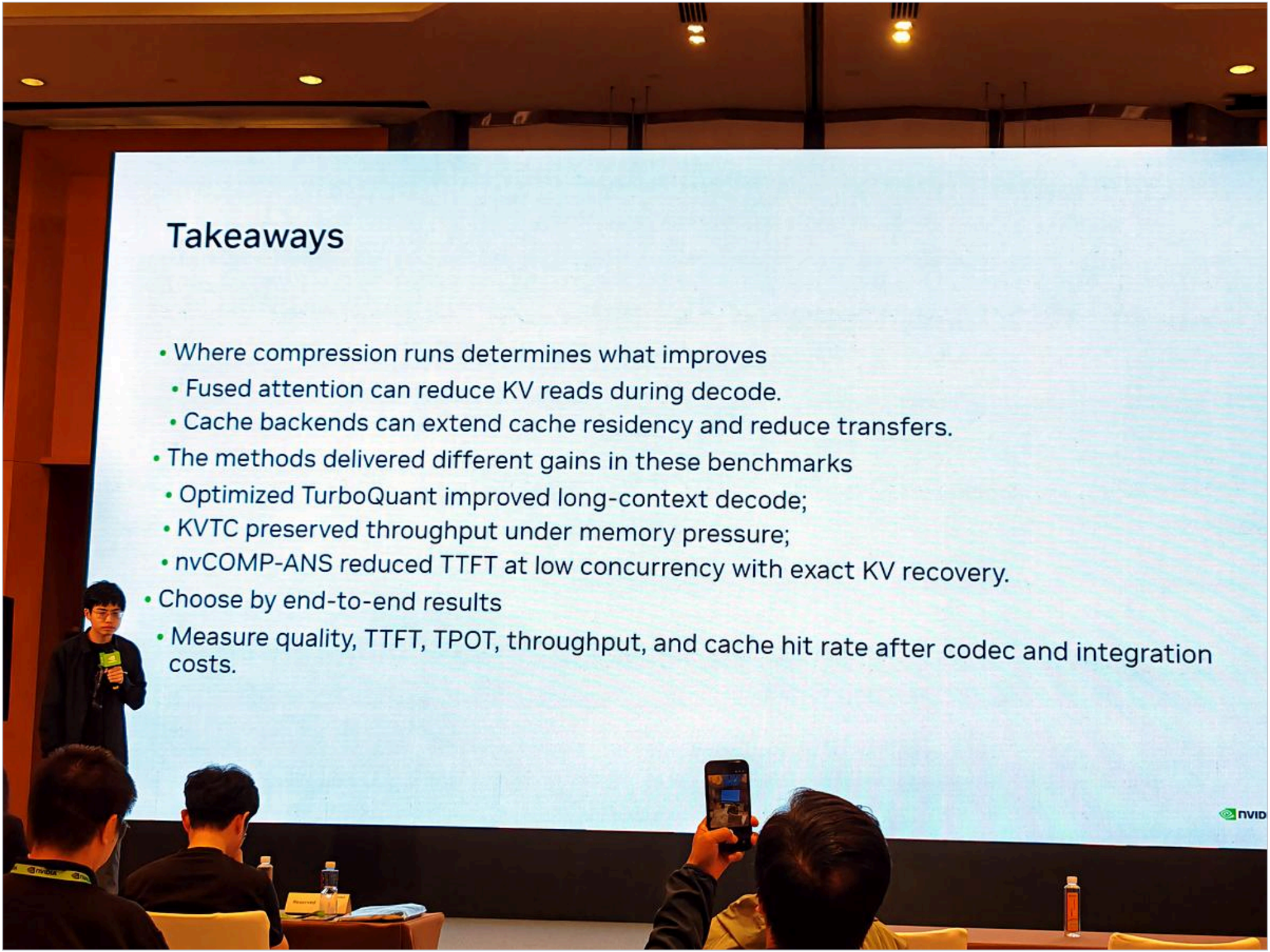

KVTC: Integration and Pipelined Decompression
A pluggable codec backend for inference engines and KV cache management frameworks
1 Pipelined decompression
Before
PCIe H2D
H2D
H2D
H2D
GPU compute
DQ0
Full GEMM ×8 · C0
DQ1
Full GEMM ×8 · C1
DQ2
Full GEMM ×8 · C2
After
warm-up
M-sharded TP-GEMM · M / 8 per GPU
PCIe H2D
H2D
H2D
H2D
GPU compute
DQ0
TP-GEMM C0
DQ1
TP-GEMM C1
DQ2
TP-GEMM C2
AllGather (M)
AG C0
AG C1
AG C2
[M,D] / GPU
Time →
saved time
2 Pluggable integration architecture
Current integration
Candidate adapter
Inference Engines
vLLM
SGLang
TensorRT-LLM
GPU Connector
KV Cache
LMCache
HiCache
Dynamo
Codec API
KVTC Algorithm
STORE / COMPRESS
KV blocks
Layout & chunking
PCA / LRPCA
DP-planned mixed precision
Packed chunks
13
NVIDIA


KVTC: Single-Request TTFT
Smaller transfers reduce TTFT by 5.5% to 10.6%
SETUP GLM-5.1 FP8 • 8× Hopper GPU, PCIe Gen5 • TP=8 • vLLM + LMCache • Compression ratio: 4.7×
TTFT (ms)
Prefill
Vanilla PCIe transfer
KVTC transfer + decompression
1000
800
600
400
200
0
204 ms
113
91
182 ms
98
84
−10.8%
407 ms
207
200
377 ms
180
197
−7.4%
591 ms
390
201
558 ms
355
203
−5.5%
843 ms
582
261
773 ms
514
259
−8.4%
uncompressed baseline
KVTC
32K
64K
128K
192K
Prefill = TTFT − LMCache load latency; stacked components par
red wall-clock TTFT.


KVTC: Throughput Under Memory Pressure
Compression prevents the cache-capacity cliff
Setup: GLM-5.1-FP8, TP=8 on 1 node with 8× Hopper GPU SXM 141GB (NVLink 900 GiB/s, PCIe Gen5 ≈46.5 GiB/s/GPU); vLLM + LMCache, 128GiB CPU-DRAM L1, 8K-token chunks, NUMA interleave=all, prefix caching off; KVTC CR=8×.
1 Problem — Vanilla degrades as memory shrinks
Concurrency = 16
Output tok/s
LMCache hit rate
Hit rate
Eviction pressure
80%
60%
50.0
40%
20%
11.9%
Host memory capacity (GiB) → decreasing
Takeaway: Vanilla hits a capacity cliff as memory shrinks: cache eviction lowers hit rate and throughput.
2 Resilience — KVTC holds performance under pressure
Capacity sweep · concurrency = 16 · Higher is better
KVTC
Vanilla
Host memory constrained
124.7
2.49×
50.0
Host memory capacity (GiB) → decreasing
3 Low overhead — Capacity sufficient (400 GiB)
Throughput delta: +3.6% to −2.1% · ~4.7× smaller host-memory footprint
Concurrency
Output tok/s V / K
Δ throughput
Mean TTFT V / K
v/lc: vanilla / kvtc
1 38.3 / 38.1 -0.5% 0.58 / 0.53 s
2 72.8 / 71.8 -1.4% 0.57 / 0.56 s
4 129.4 / 126.7 -2.1% 0.71 / 0.67 s
8 127.1 / 131.0 +3.1% 13.1 / 12.9 s
16 125.8 / 130.3 +3.6% 36.54 / 36.54 s
ity, KVTC reduces memory footprint by ~4.7× with negligible performance overhead.
2.49×
Output throughput
at 96 GiB / c16
−64%
Mean TTFT
108.8 → 39.1 s
+62.8 pp
LMCache hit rate
11.9% → 74.7%

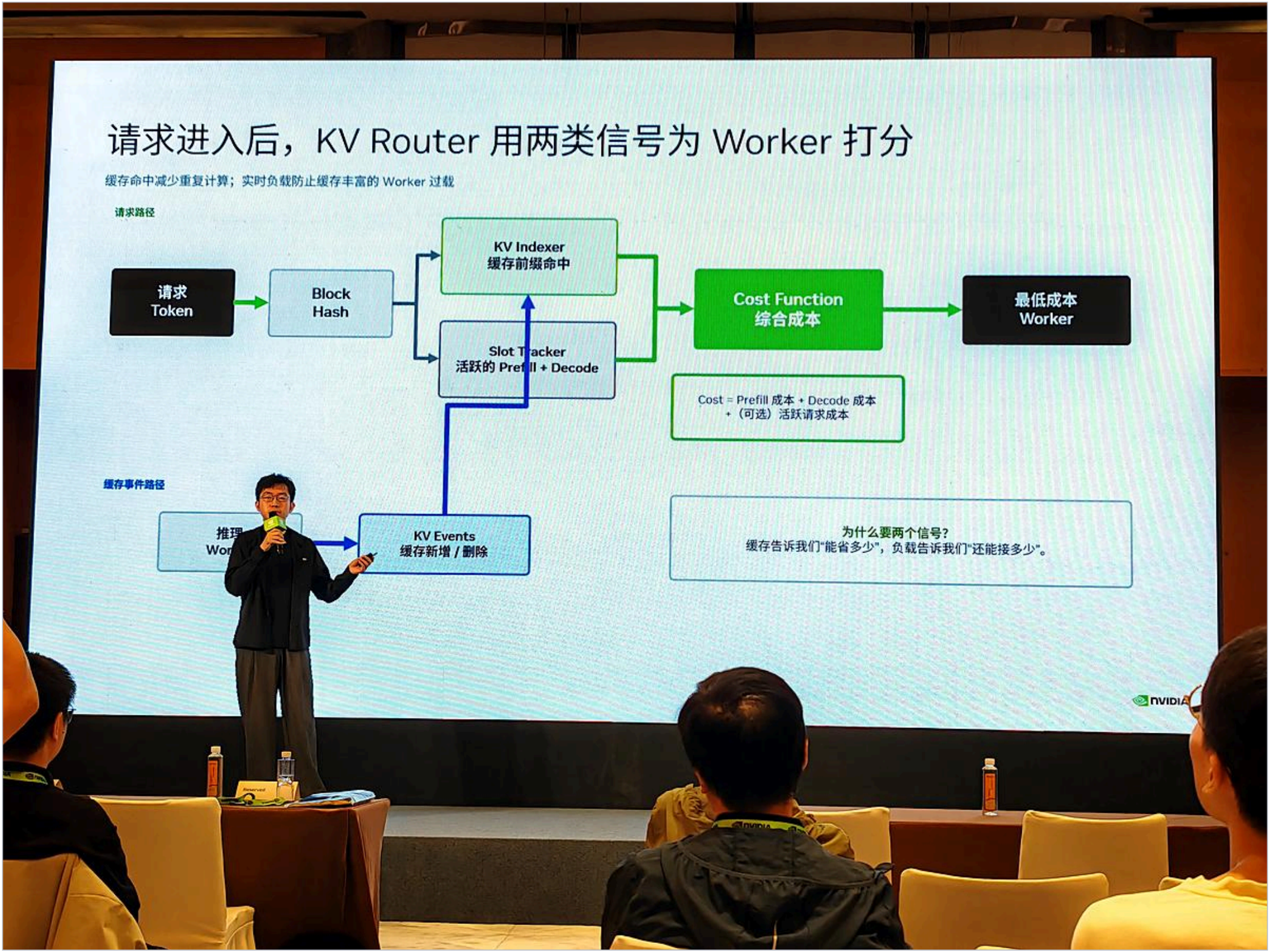

nvCOMP-ANS: Lossless KV Cache Transfer
GPU compression reduces data movement during offload and reload
GPU
CPU memory
SSD
Raw KV
BF16 / FP16 / FP8
nvCOMP ANS
Compress / decompress
D2H / H2D
Compressed bytes
Compressed payload
Per-chunk size metadata
Write / read
Host gather / scatter
Compressed KV Blocks
Fewer bytes per read/write
Fixed-size storage slots
Exact KV recovery
Lossless entropy coding restores the original values
Overlapped execution
Batched GPU codec with double-buffered transfers
FlexKV integration
MHA / GQA / MLA / DSA
TP / DP support
Fewer bytes move between tiers. CPU and SSD retain fixed-size cache slots.
16
https://developer.nvidia.com/nvcom

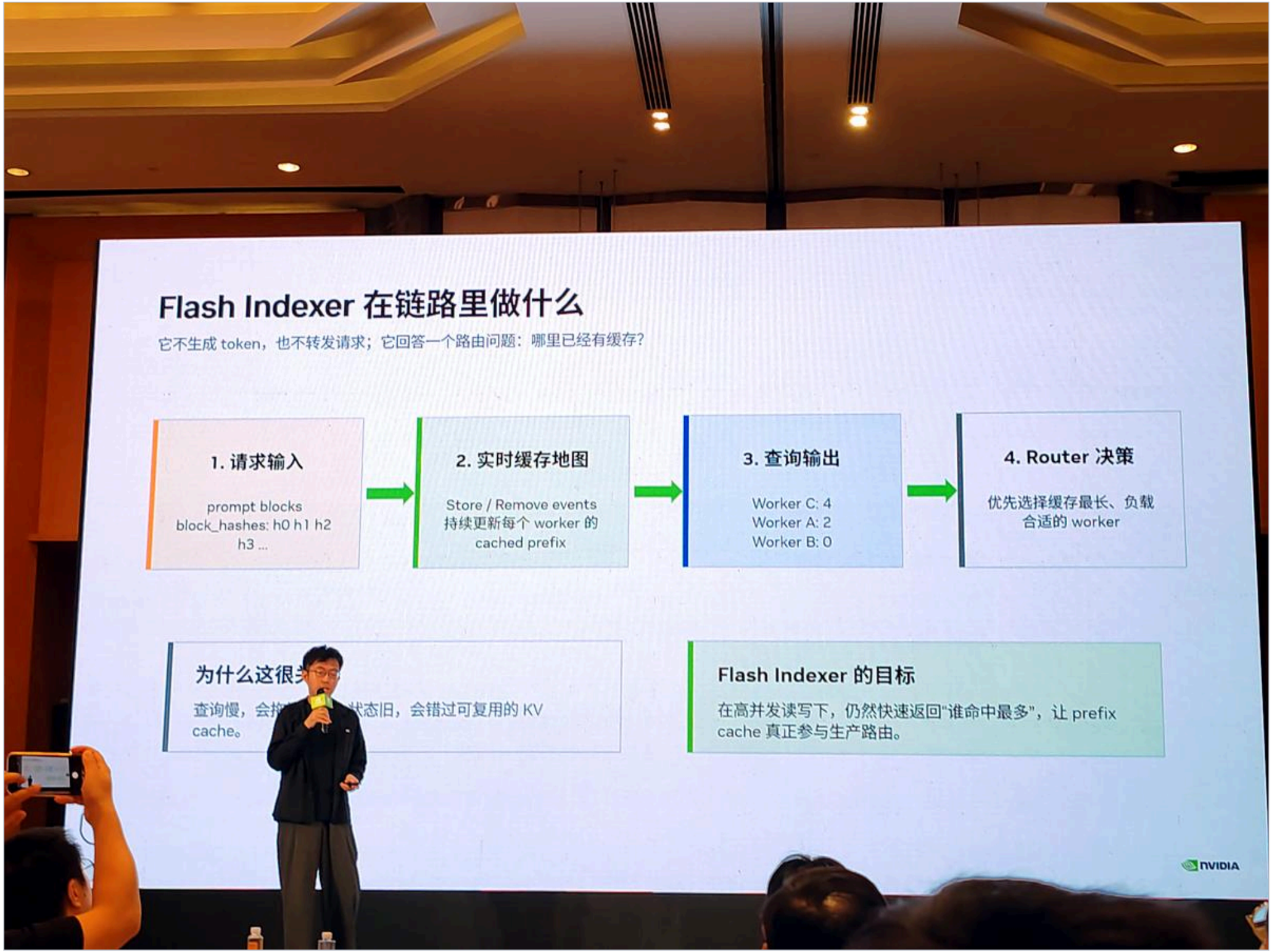

nvCOMP-ANS: Compression and Serving Results
TTFT gains are strongest at low concurrency in this benchmark
LOSSLESS COMPRESSION
Original bytes / compressed bytes
~1.4×
BF16 KV cache
~1.1×
FP8 KV cache
TTFT speedup by concurrency (c)
20.0%
10.0%
0.0%
-10.0%
5.7%
16.9%
8.2%
18.7%
1.8%
4.2%
3.2%
3.0%
-5.6%
2.1%
1
2
3
4
8
Mean TTFT
P90 TTFT
+18.7%
P90 TTFT speedup at c = 2
+8.2% mean TTFT speedup
Throughput ≈ unchanged
Output throughput change:
−1.9% to +2.4% across c = 1–8
FT speedup = baseline latency / nvCOMP latency − 1. Bars show means. P90 varies widely at c = 3 and 8.
E2E SGLang + FlexKV (CPU only) / GLM-5.1-FP8 / BF16 KV / 128K input / 8× Hopper GPU
github.com/taco-project/FlexKV/pull/183


Takeaways
• Where compression runs determines what improves
• Fused attention can reduce KV reads during decode.
• Cache backends can extend cache residency and reduce transfers.
• The methods delivered different gains in these benchmarks
• Optimized TurboQuant improved long-context decode;
• KVTC preserved throughput under memory pressure;
• nvCOMP-ANS reduced TTFT at low concurrency with exact KV recovery.
• Choose by end-to-end results
• Measure quality, TTFT, TPOT, throughput, and cache hit rate after codec and integration costs.


What Dynamo can do today
All kinds of disaggregation
• Prefill / decode disaggregation
• Attention / FFN disaggregation (GPU/LPU)
• Multimodal E/P/D pipeline
KV-aware router and Flash Indexer
Global KV management
• FlexKV / KVCR
• Multi-node distributed P2P KV reuse with router hints
Fault tolerance
• Frontend / KV router and Agg / Disagg FT
– Model Express and Snapshot
• Wide-EP FT - NIXL elastic EP
Planner
• Online active scaling of P/D workers
– Grove - Find Most Ideal Worker
Digital twin of large-scale MaaS - high-fidelity projection
• aisimulate
RL rollout acceleration and harness-based agentic inference
• ThunderAgent, nvext agent hints, etc.
~8.3K
GitHub stars
244K+
pip installs
9,000+
NVIDIA

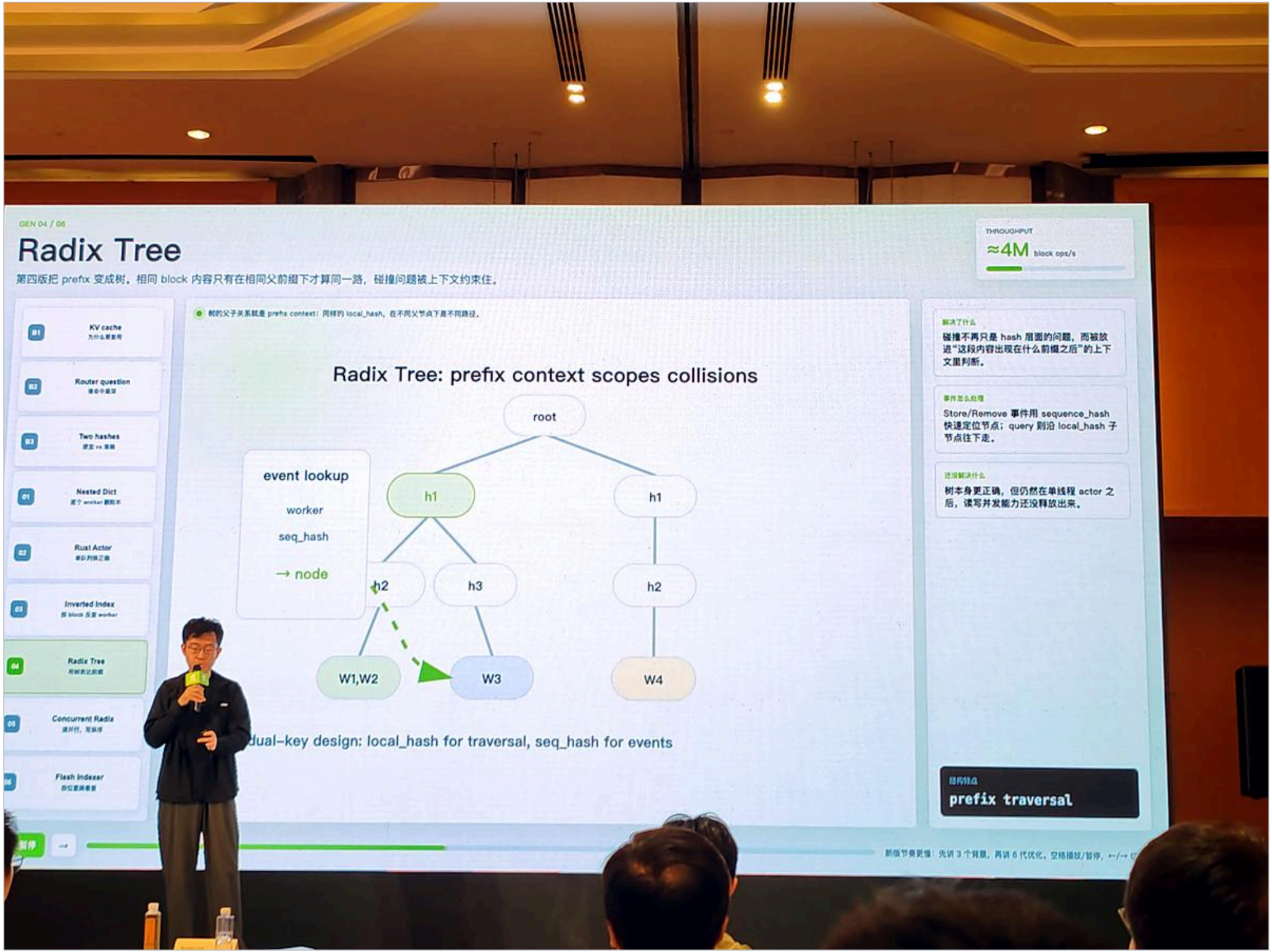

KV Router在生产环境中被广泛验证
Flash Indexer是支撑准确路由的基石
Baseten
2X faster TTFT & 1.6X TPS with KV aware routing
https://www.baseten.co/b w-baseten-achieved-2x-faster-inference-with-nvidia-dynamo/
Meituan
2x faster TTFT & + 20% more KV cache hit with KV routing
https://www.nvidia.com/en-us/on-demand/session/gtc26-s81943/
NVIDIA

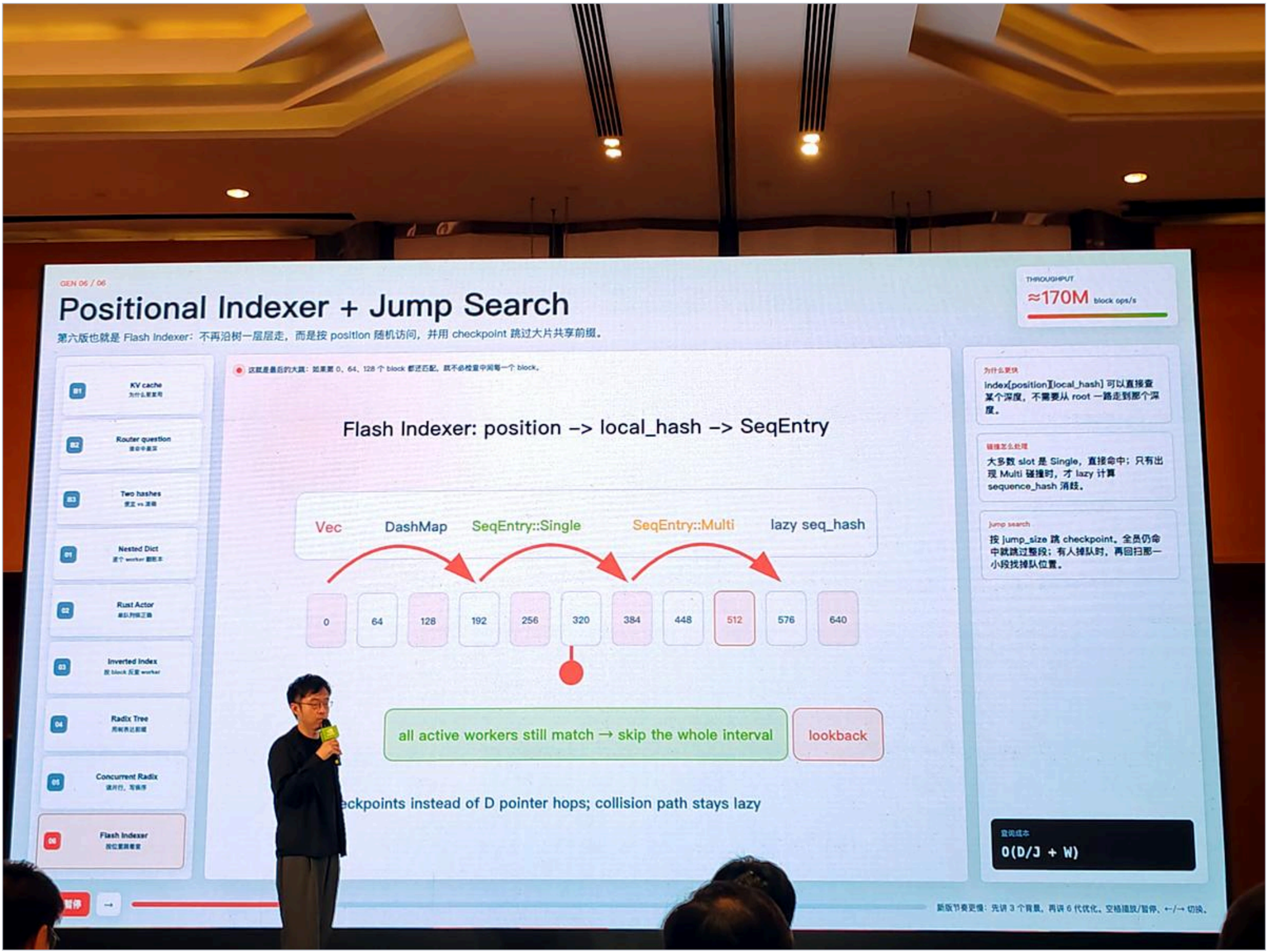

请求进入后，KV Router 用两类信号为 Worker 打分
缓存命中减少重复计算；实时负载防止缓存丰富的 Worker 过载
请求路径
请求 Token
Block Hash
KV Indexer 缓存前缀命中
Slot Tracker 活跃的 Prefill + Decode
Cost Function 综合成本
最低成本 Worker
Cost = Prefill 成本 + Decode 成本 + （可选）活跃请求成本
缓存事件路径
推理 Wor
KV Events 缓存新增 / 删除
为什么要两个信号？
缓存告诉我们"能省多少"，负载告诉我们"还能接多少"。
NVIDIA


Flash Indexer 在链路里做什么
它不生成 token，也不转发请求；它回答一个路由问题：哪里已经有缓存？
1. 请求输入
prompt blocks
block_hashes: h0 h1 h2 h3 ...
2. 实时缓存地图
Store / Remove events
持续更新每个 worker 的 cached prefix
3. 查询输出
Worker C: 4
Worker A: 2
Worker B: 0
4. Router 决策
优先选择缓存最长、负载合适的 worker
为什么这很关
查询慢，会拖
状态旧，会错过可复用的 KV cache。
Flash Indexer 的目标
在高并发读写下，仍然快速返回"谁命中最多"，让 prefix cache 真正参与生产路由。
NVIDIA


索引里存的是一张“位置到 worker”的地图
为了判断 prefix 是否连续，Flash Indexer 不能只看 block 内容，还要看它在第几个位置。
请求被切成 blocks
h0 h1 h2 h3 h4 h5 h6 h7
每个block有两个信息
position：第几块
block_hash：内容指纹
key -> value
(0, h0) -> Worker A, C, D
(1, h1) -> Worker A, C
(2, h2) -> Worker C
(3, h3) -> Worker C
同一个 block_hash 可能在不同位置出现；只有位置也一致，才说明这段 prefix 真的连续。
NVIDIA


GEN 01 / 06
Naive Nested Dict
第一版像一本按 worker 分册的账本：要找某个 prefix，只能先翻 W1，再翻 W2，再翻 W3。
THROUGHPUT
≈385K block ops/s
KV cache
Router question
Two hashes
Nested Dict
Rust Actor
Inverted Index
Radix Tree
Concurrent Radix
Flash Indexer
Nested Dict: 逐个 worker 翻账本
h1
h2
h3
h4
W1 的缓存账本
W2 的缓存账本
W3 的缓存账本
pos 0
pos 1
pos 2
pos 3
match depth = 2
match depth = 1
match depth = 4
慢的原因：外层扫 worker，内层扫 prefix depth。worker 越多、prompt 越长，翻得越久。
h3* 提醒：只看 local_hash 还可能把"同内容、不同前缀位置"的 block 混淆。
为什么慢
每个请求都要扫所有 worker。worker 越多、prompt 越长，查询越像在全仓库逐柜翻找。
为什么还不够准
local_hash 只代表 block 内容。相同内容出现在不同位置时，可能被误认为 prefix 连续命中。
这一代的价值
它暴露了两个核心问题：访问模式不对，以及 local_hash 碰撞需要 prefix context。
查询成本
O(W × D)


GEN 04 / 06
Radix Tree
第四版把 prefix 变成树。相同 block 内容只有在相同父前缀下才算同一路，碰撞问题被上下文约束住。
THROUGHPUT
≈4M block ops/s
01 KV cache
02 Router question
03 Two hashes
01 Nested Dict
02 Rust Actor
05 Inverted Index
04 Radix Tree
05 Concurrent Radix
06 Flash Indexer
树的父子关系就是 prefix context：同样的 local_hash，在不同父节点下是不同路径。
Radix Tree: prefix context scopes collisions
root
event lookup
worker
seq_hash
→ node
h1
h1
h2
h3
h2
W1,W2
W3
W4
dual-key design: local_hash for traversal, seq_hash for events
解决了什么
碰撞不再只是 hash 层面的问题，而被放进“这段内容出现在什么前缀之后”的上下文里判断。
事件怎么处理
Store/Remove 事件用 sequence_hash 快速定位节点；query 则沿 local_hash 子节点往下走。
还没解决什么
树本身更正确，但仍然在单线程 actor 之后，读写并发能力还没释放出来。
结构特点
prefix traversal

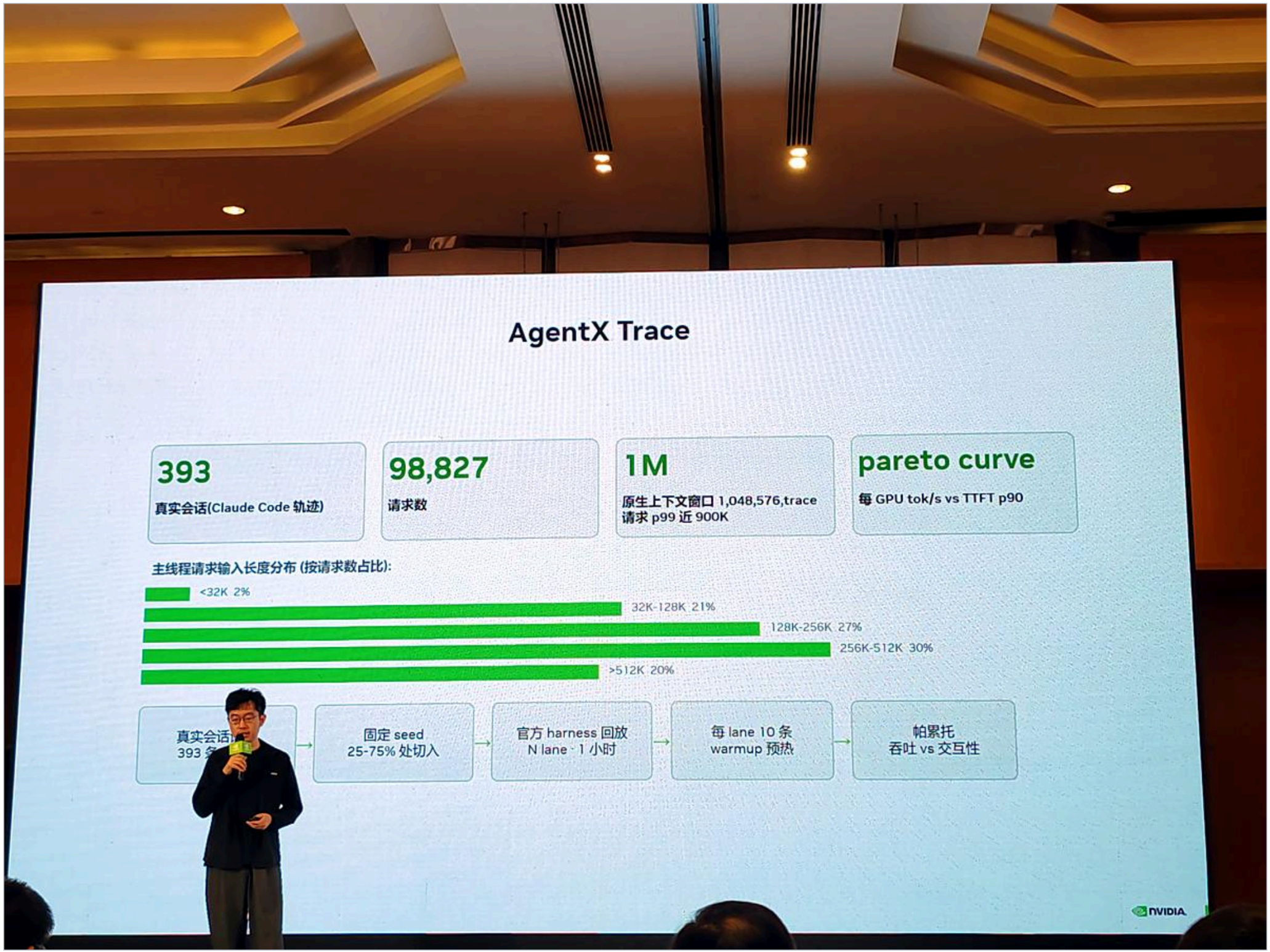

GEN 06 / 06
Positional Indexer + Jump Search
第六版也就是 Flash Indexer：不再沿树一层层走，而是按 position 随机访问，并用 checkpoint 跳过大片共享前缀。
THROUGHPUT
≈170M block ops/s
01 KV cache
02 Router question
03 Two hashes
01 Nested Dict
02 Rust Actor
03 Inverted Index
04 Radix Tree
05 Concurrent Radix
06 Flash Indexer
这就是最后的大跳：如果第 0、64、128 个 block 都还匹配，就不必检查中间每一个 block。
Flash Indexer: position -> local_hash -> SeqEntry
Vec
DashMap
SeqEntry::Single
SeqEntry::Multi
lazy seq_hash
0
64
128
192
256
320
384
448
512
576
640
all active workers still match → skip the whole interval
lookback
eckpoints instead of D pointer hops; collision path stays lazy
为什么更快
index[position][local_hash] 可以直接查某个深度，不需要从 root 一路走到那个深度。
碰撞怎么处理
大多数 slot 是 Single，直接命中；只有出现 Multi 碰撞时，才 lazy 计算 sequence_hash 消歧。
Jump search
按 jump_size 跳 checkpoint。全员仍命中就跳过整段；有人掉队时，再回扫那一小段找掉队位置。
查询成本
O(D/J + W)
暂停
→
新版节奏更慢：先讲 3 个背景，再讲 6 代优化。空格播放/暂停，←/→ 切换。

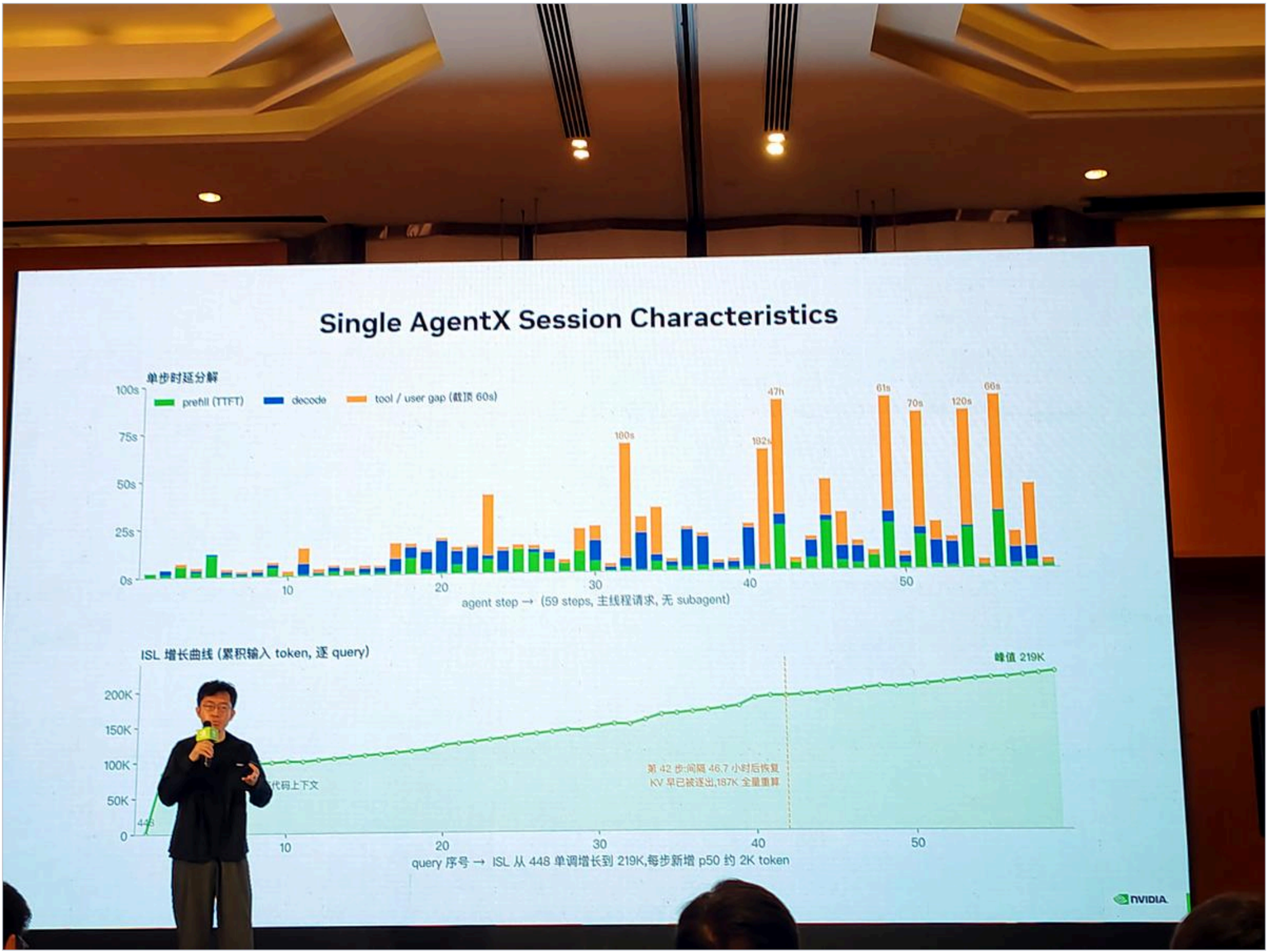

性能数据
ACHIEVED VS. OFFERED THROUGHPUT (HIGHER IS BETTER)
PEAK THROUGHPUT
170M
83M
4M
810K
385K
48×
Naive
385K
v0.1 Radix
4M
Flash
170M
提升
440x / 42x
Benchmark setup
• 24-core Arrow Lake 285K desktop
• Mooncake FAST'25 trace + mock engine
• 16,384 GPU blocks，prefix caching enabled
• 24 concurrent event-processing threads
sh Indexer 的价值不只在“知道缓存在哪里”，还在“这个查询足够快”，足以支撑大规模请求
NVIDIA

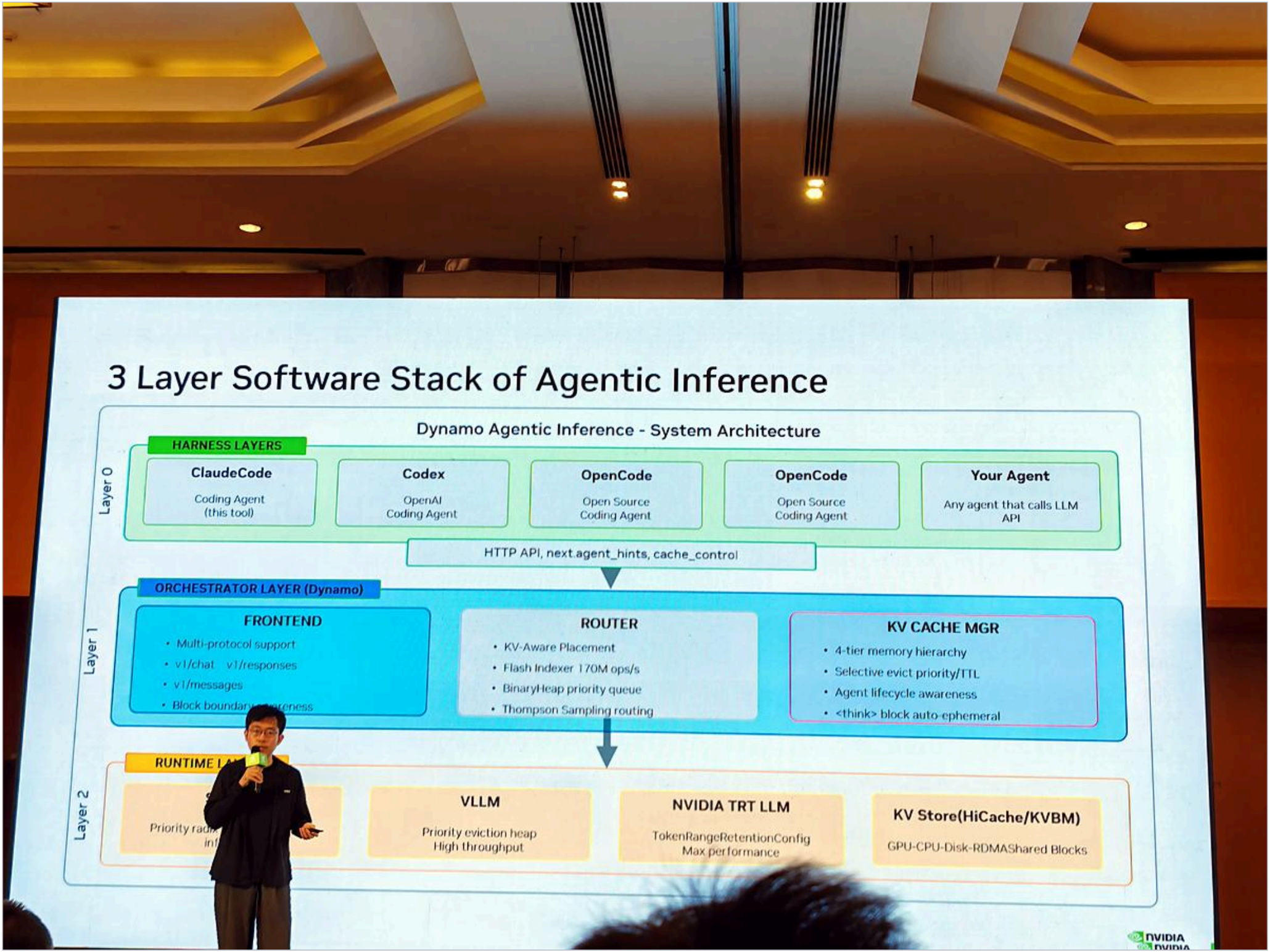

Standalone KV Indexer：定位
把 KV cache 的索引能力从 router/frontend 里拆出来，变成可独立查询的服务。
它是什么
• 独立维护 cached KV blocks 的索引
• 订阅 worker 的 KV cache 变化事件
• 通过查询接口返回 prefix overlap
它不是什么
• 不是 standalone router
• 不做最终 worker selection
• 不维护完整 request lifecycle
适合什么时候用
• 调试 router 看到的 KV state
• 外部系统做 custom routing
• 观察不同 worker / tier 的 cache 分布
6
NVIDIA

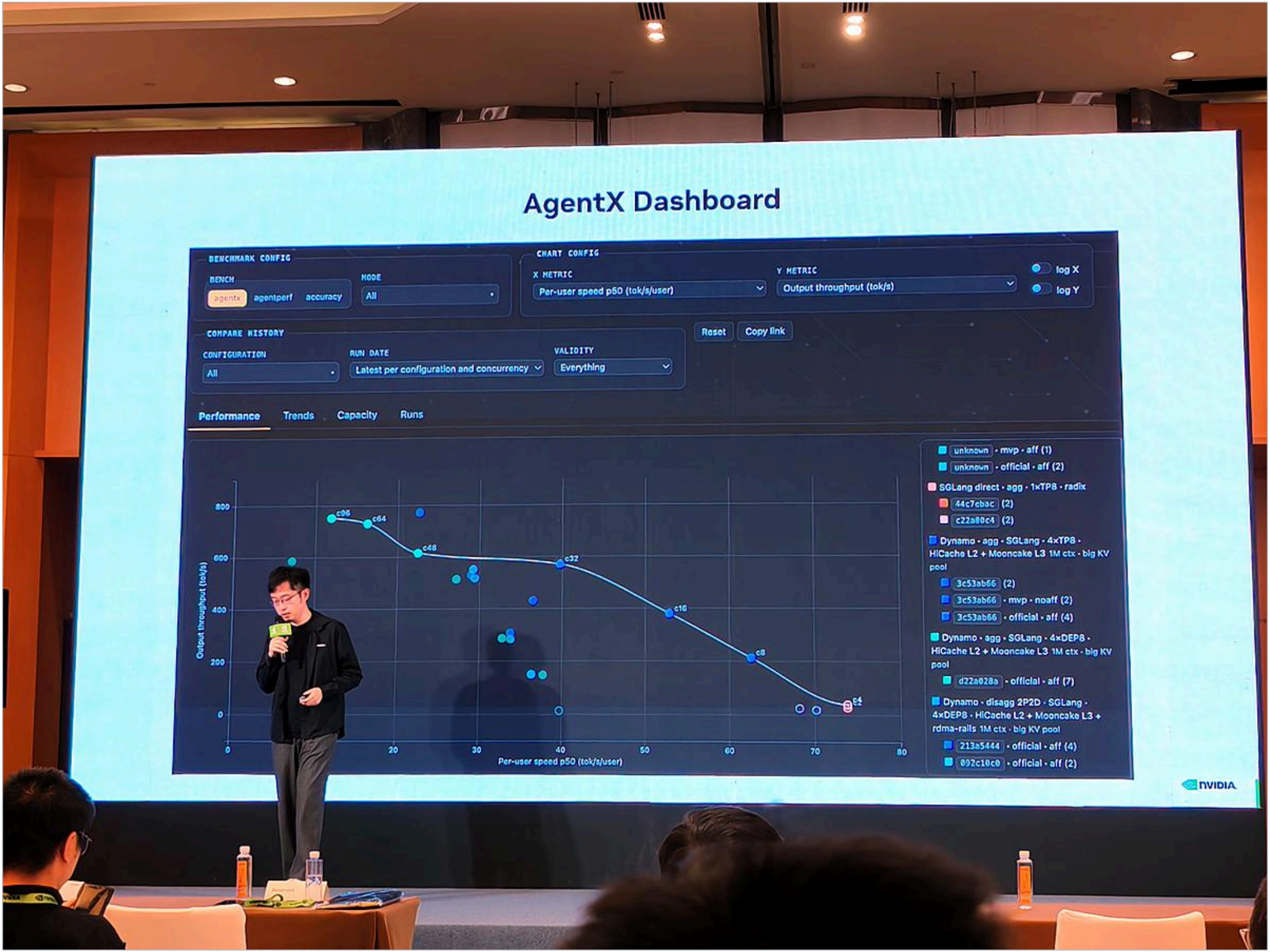

Standalone KV Indexer: 架构
ZMQ 事件面负责 ingest，HTTP 请求面负责注册、查询和观测。
Worker 1
ZMQ PUB
Worker 2
ZMQ PUB
KV events
Standalone Indexer
Worker Registry
ZMQ SUB Listeners
(model, tenant) -> Radix Tree
HTTP API
管理类 API
• GET /health
• POST /register /unregister
• GET /workers
查询类 API
• POST /query
• POST /query_by_hash
恢复类 API
• GET /dump
• POST /register_peer /deregister_peer
• GET /peers
查询结果按 mat s 返回，并可拆分 gpu / cpu / disk tier
NVIDIA

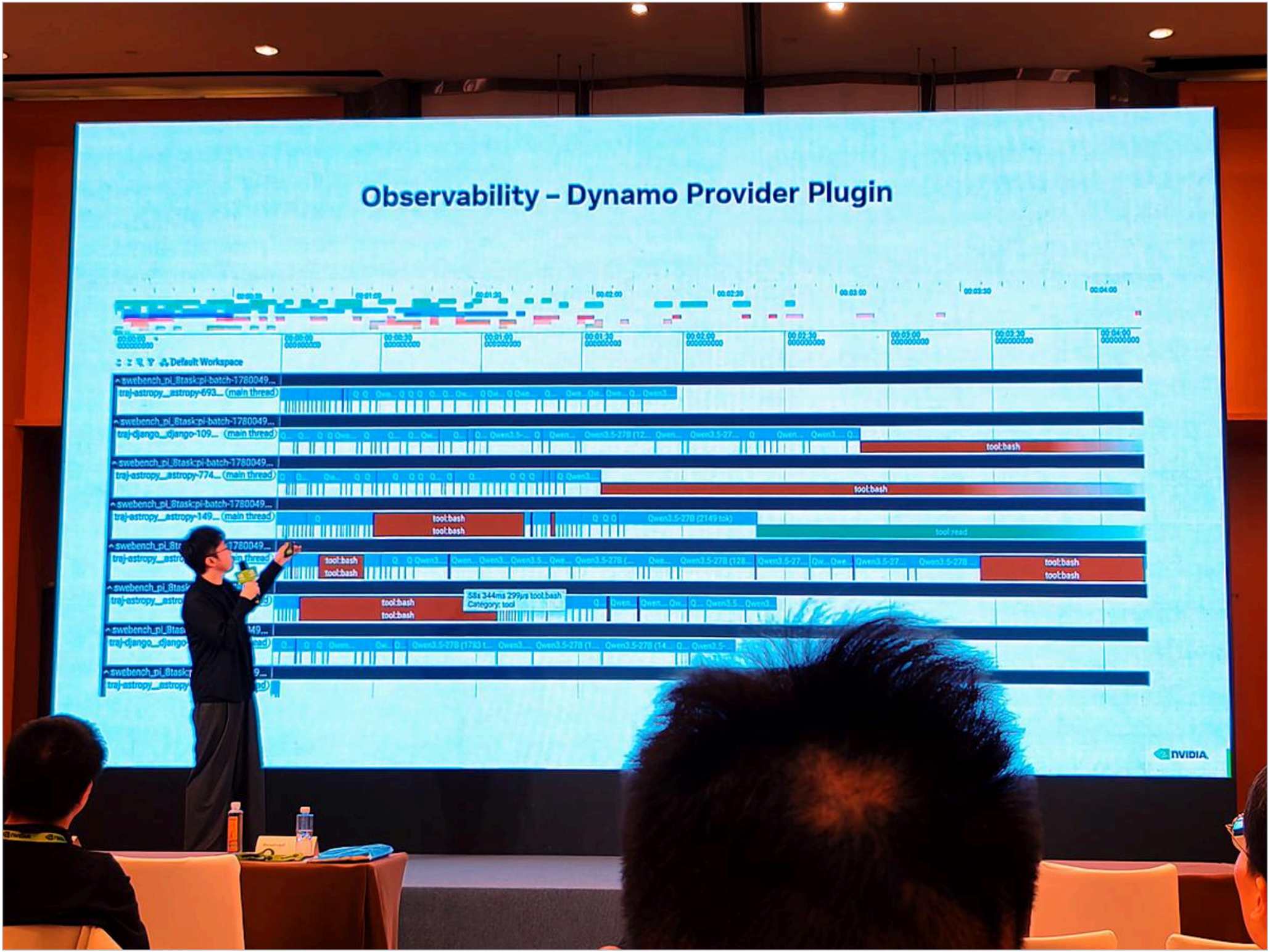

SLAs of New Agentic Era
Traditional Inference
TTFT — Time to First Token
TPOT — Time per Output Token
Request E2E latency
满足 SLA 的千卡 QPS
以单次请求为中心
Agentic Inference
Trajectory E2E latency
Time to First Answer Token
System throughput
AA-AgentPerf: SLO 下最大并发 agent 数
Global KV cache hit rate > 90%
HiCache / FlexKV,P2P KV reuse
API price — SLA 下并发 session 成本
核心变化:单次请求指标 → trajectory / session 级系统 SLA

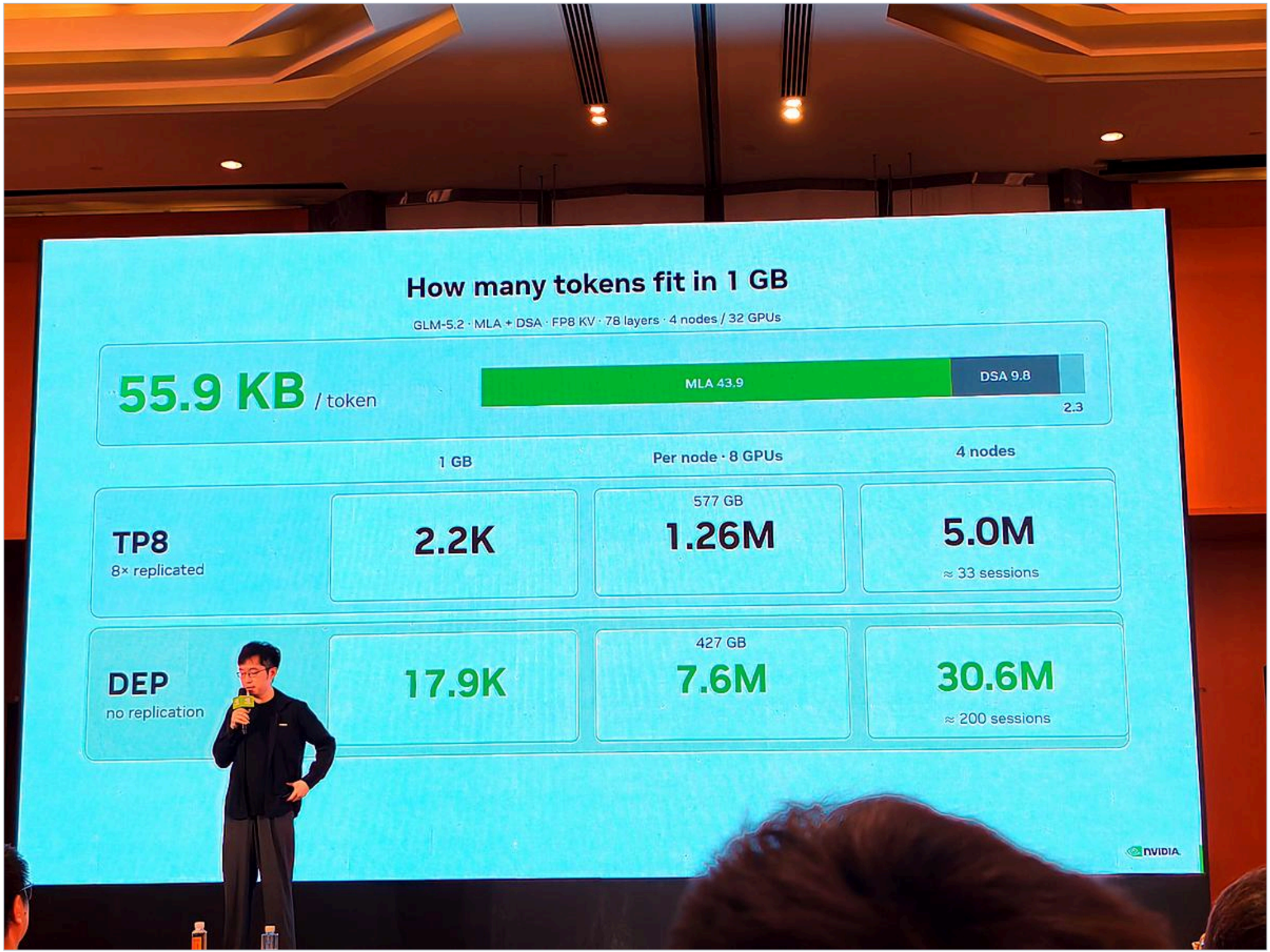

AgentX Trace
393
真实会话(Claude Code 轨迹)
98,827
请求数
1M
原生上下文窗口 1,048,576,trace
请求 p99 近 900K
pareto curve
每 GPU tok/s vs TTFT p90
主线程请求输入长度分布 (按请求数占比):
<32K 2%
32K-128K 21%
128K-256K 27%
256K-512K 30%
>512K 20%
固定 seed
25-75% 处切入
官方 harness 回放
N lane · 1 小时
每 lane 10 条
warmup 预热
帕累托
吞吐 vs 交互性
NVIDIA

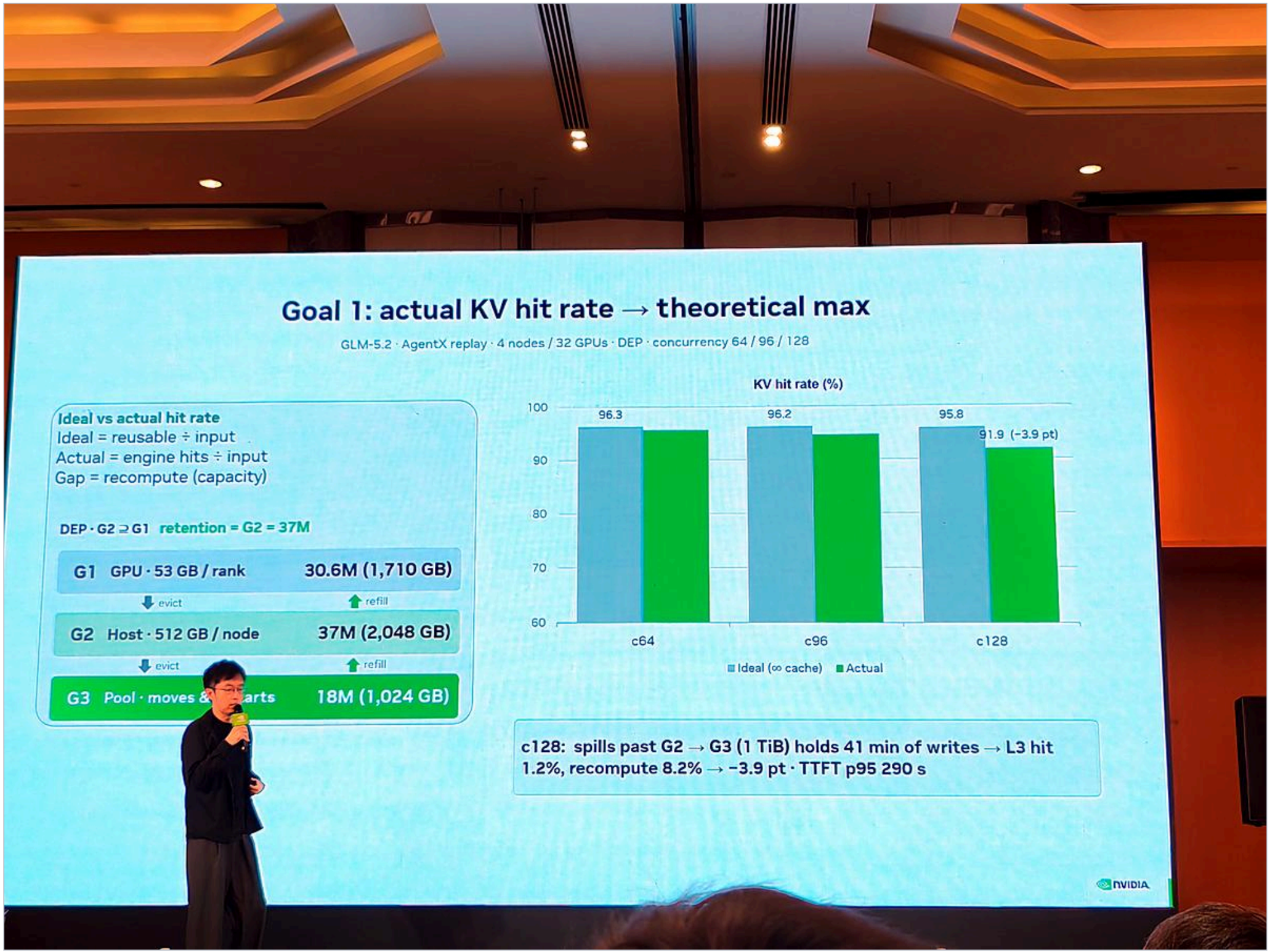

Single AgentX Session Characteristics
单步时延分解
prefill (TTFT)
decode
tool / user gap (截顶 60s)
100s
75s
50s
25s
0s
10
20
30
40
50
180s
182s
47h
61s
70s
120s
66s
agent step → (59 steps, 主线程请求, 无 subagent)
ISL 增长曲线 (累积输入 token, 逐 query)
峰值 219K
200K
150K
100K
50K
0
448
代码上下文
第 42 步:间隔 46.7 小时后恢复
KV 早已被逐出,187K 全量重算
query 序号 → ISL 从 448 单调增长到 219K,每步新增 p50 约 2K token
NVIDIA

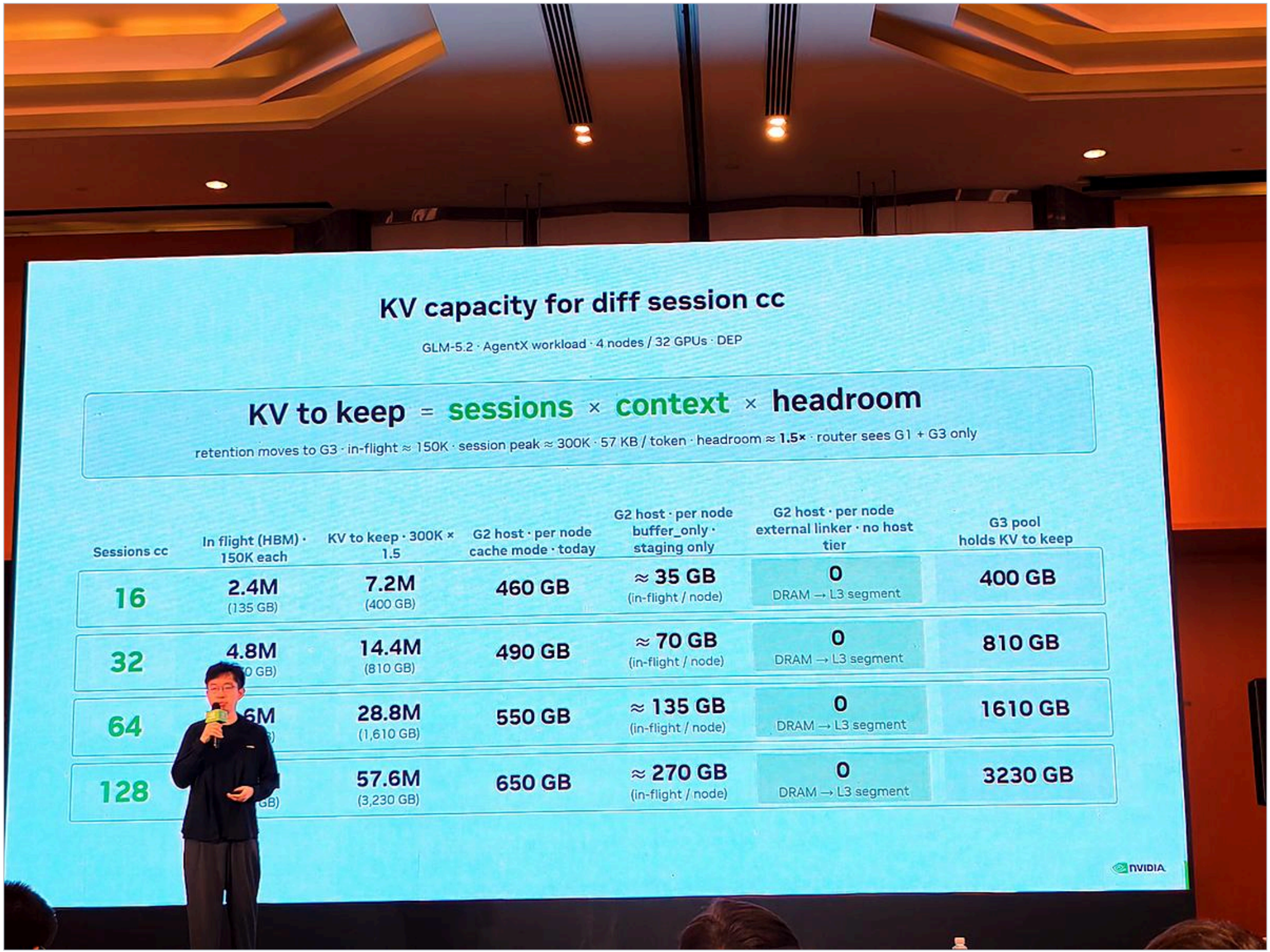

3 Layer Software Stack of Agentic Inference
Dynamo Agentic Inference - System Architecture
HARNESS LAYERS
Layer 0
ClaudeCode
Coding Agent
(this tool)
Codex
OpenAI
Coding Agent
OpenCode
Open Source
Coding Agent
OpenCode
Open Source
Coding Agent
Your Agent
Any agent that calls LLM API
HTTP API, next.agent_hints, cache_control
ORCHESTRATOR LAYER (Dynamo)
Layer 1
FRONTEND
Multi-protocol support
v1/chat v1/responses
v1/messages
ROUTER
KV-Aware Placement
Flash Indexer 170M ops/s
BinaryHeap priority queue
Thompson Sampling routing
KV CACHE MGR
4-tier memory hierarchy
Selective evict priority/TTL
Agent lifecycle awareness
<think> block auto-ephemeral
Layer 2
VLLM
Priority eviction heap
High throughput
NVIDIA TRT LLM
TokenRangeRetentionConfig
Max performance
KV Store(HiCache/KVBM)
GPU-CPU-Disk-RDMAShared Blocks


AgentX Dashboard
BENCHMARK CONFIG
BENCH
agentx
agentperf
accuracy
MODE
All
CHART CONFIG
X METRIC
Per-user speed p50 (tok/s/user)
Y METRIC
Output throughput (tok/s)
log X
log Y
COMPARE HISTORY
CONFIGURATION
All
RUN DATE
Latest per configuration and concurrency
VALIDITY
Everything
Reset
Copy link
Performance
Trends
Capacity
Runs
Output throughput (tok/s)
Per-user speed p50 (tok/s/user)
c96
c64
c48
c32
c16
c8
c4
unknown · mvp · aff (1)
unknown · official · aff (2)
SGLang direct · agg · 1×TP8 · radix
44c7ebac (2)
c22a80c4 (2)
Dynamo · agg · SGLang · 4×TP8 · HiCache L2 + Mooncake L3 1M ctx · big KV pool
3c53ab66 (2)
3c53ab66 · mvp · noaff (2)
3c53ab66 · official · aff (4)
Dynamo · agg · SGLang · 4×DEP8 · HiCache L2 + Mooncake L3 1M ctx · big KV pool
d22a028a · official · aff (7)
Dynamo · disagg 2P2D · SGLang · 4×DEP8 · HiCache L2 + Mooncake L3 + rdma-rails 1M ctx · big KV pool
213a5444 · official · aff (4)
092c10c0 · official · aff (2)
NVIDIA

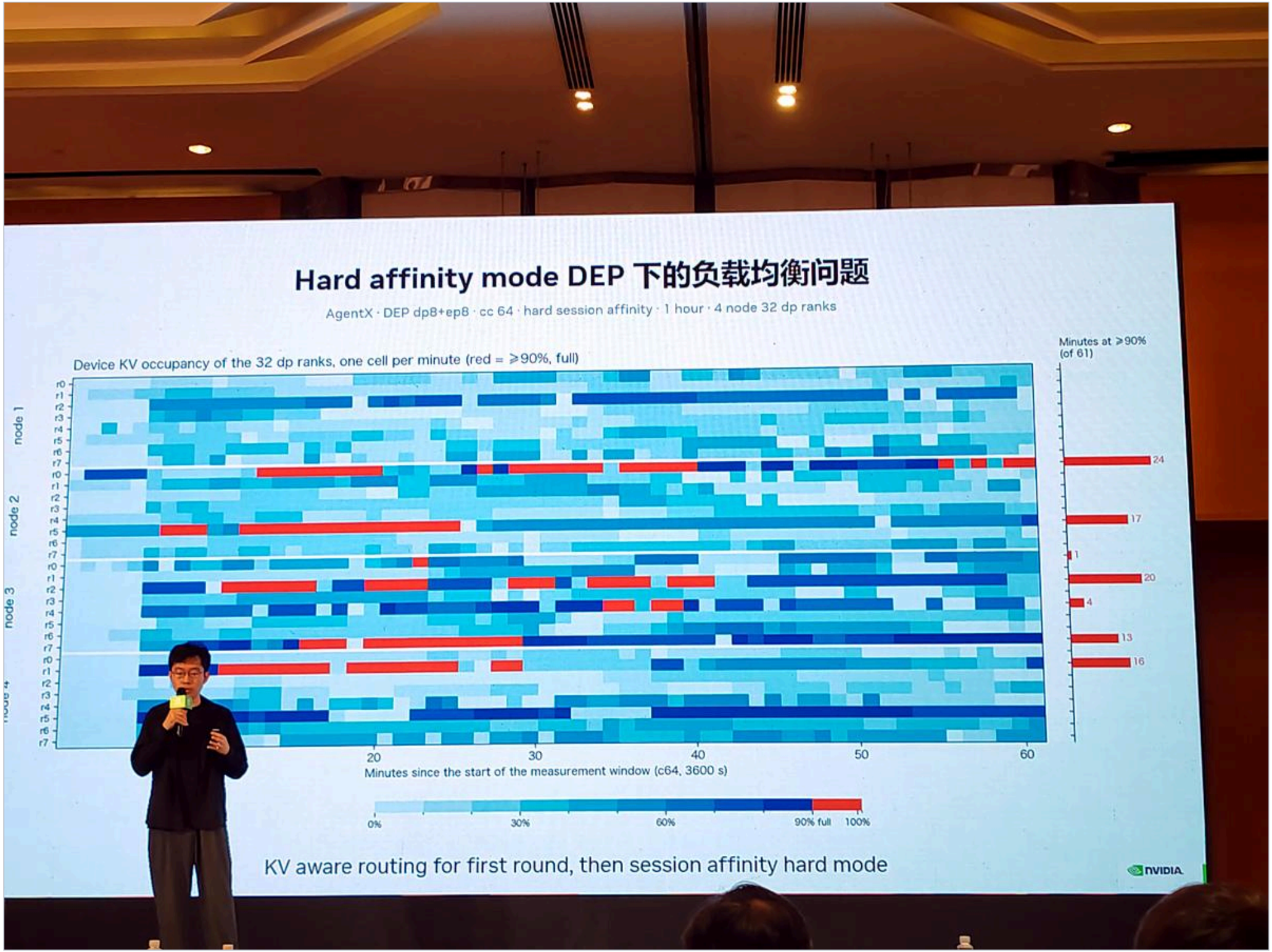

Observability – Dynamo Provider Plugin
Default Workspace
tool:bash
tool:read
NVIDIA


How many tokens fit in 1 GB
GLM-5.2 · MLA + DSA · FP8 KV · 78 layers · 4 nodes / 32 GPUs
55.9 KB / token
MLA 43.9
DSA 9.8
2.3
1 GB
Per node · 8 GPUs
4 nodes
TP8
8× replicated
2.2K
577 GB
1.26M
5.0M
≈ 33 sessions
DEP
no replication
17.9K
427 GB
7.6M
30.6M
≈ 200 sessions
NVIDIA

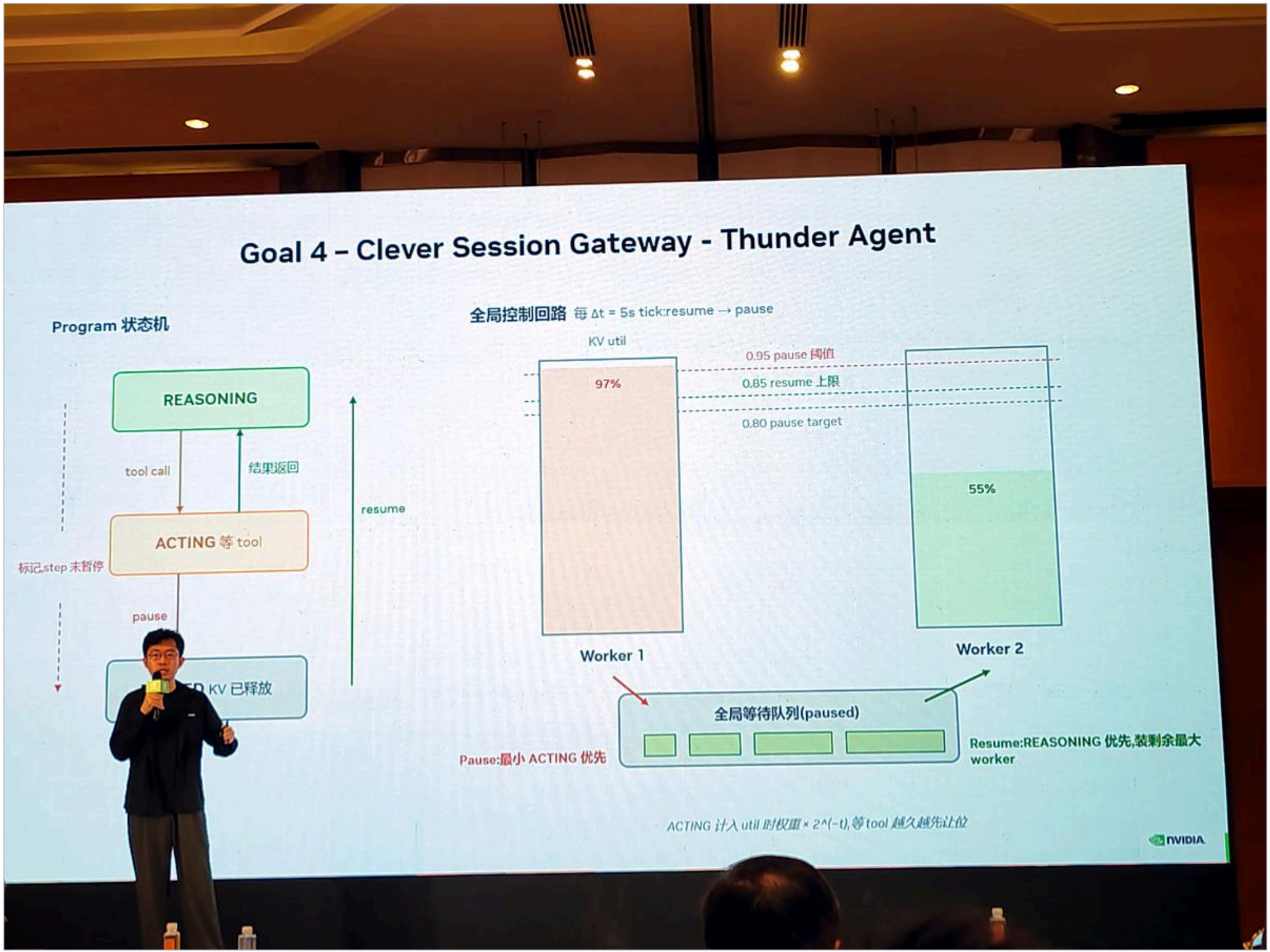

Goal 1: actual KV hit rate → theoretical max
GLM-5.2 · AgentX replay · 4 nodes / 32 GPUs · DEP · concurrency 64 / 96 / 128
Ideal vs actual hit rate
Ideal = reusable ÷ input
Actual = engine hits ÷ input
Gap = recompute (capacity)
DEP · G2 ⊇ G1 retention = G2 = 37M
G1 GPU · 53 GB / rank 30.6M (1,710 GB)
evict
refill
G2 Host · 512 GB / node 37M (2,048 GB)
evict
refill
G3 Pool · moves & arts 18M (1,024 GB)
KV hit rate (%)
100
90
80
70
60
96.3
96.2
95.8
91.9 (−3.9 pt)
c64
c96
c128
Ideal (∞ cache)
Actual
c128: spills past G2 → G3 (1 TiB) holds 41 min of writes → L3 hit 1.2%, recompute 8.2% → −3.9 pt · TTFT p95 290 s
NVIDIA

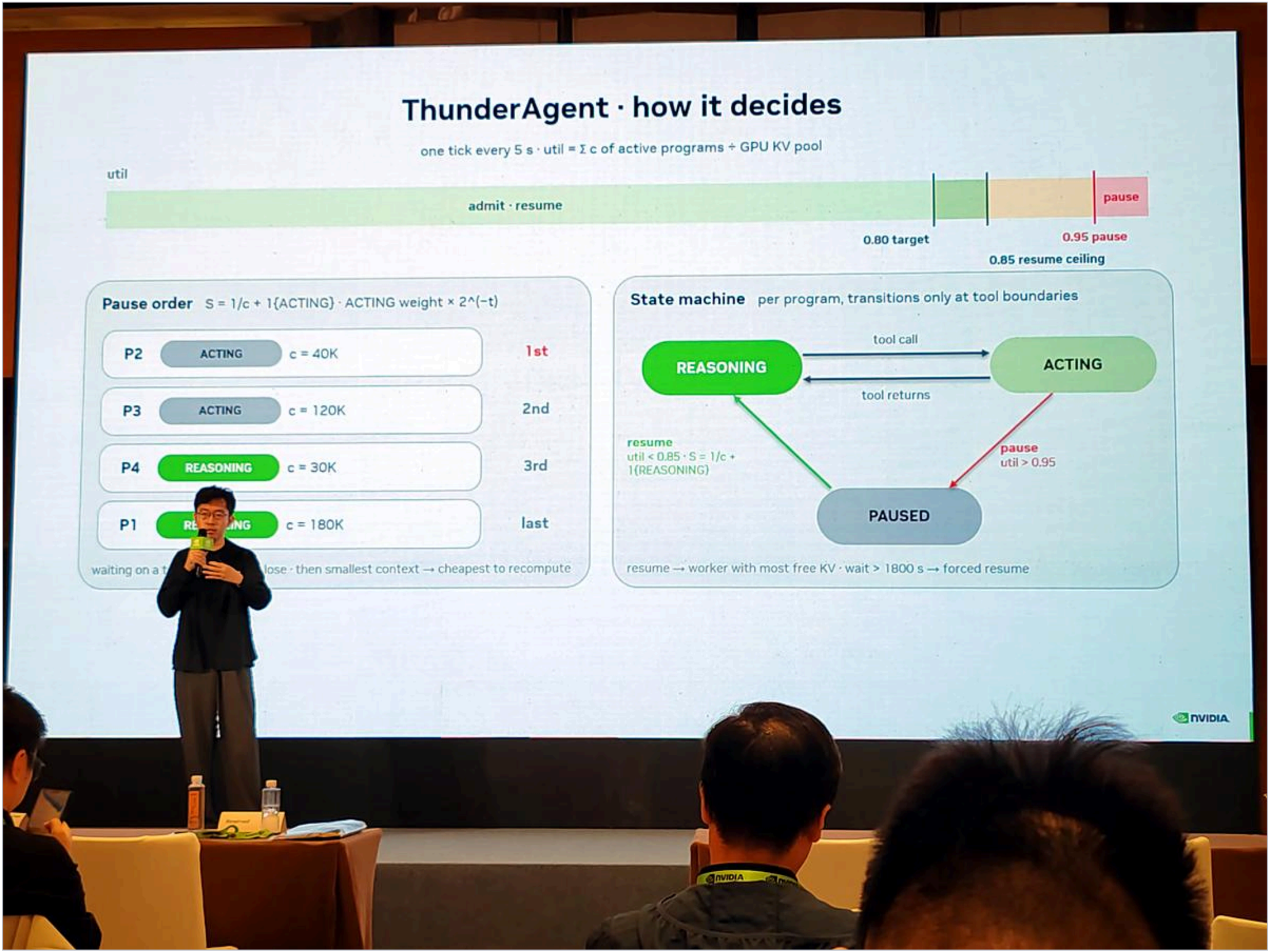

KV capacity for diff session cc
GLM-5.2 · AgentX workload · 4 nodes / 32 GPUs · DEP
KV to keep = sessions × context × headroom
retention moves to G3 · in-flight ≈ 150K · session peak ≈ 300K · 57 KB / token · headroom ≈ 1.5× · router sees G1 + G3 only
Sessions cc
In flight (HBM) · 150K each
KV to keep · 300K × 1.5
G2 host · per node cache mode · today
G2 host · per node buffer_only · staging only
G2 host · per node external linker · no host tier
G3 pool holds KV to keep
16
2.4M (135 GB)
7.2M (400 GB)
460 GB
≈ 35 GB (in-flight / node)
0 DRAM → L3 segment
400 GB
32
4.8M
14.4M (810 GB)
490 GB
≈ 70 GB (in-flight / node)
0 DRAM → L3 segment
810 GB
64
28.8M (1,610 GB)
550 GB
≈ 135 GB (in-flight / node)
0 DRAM → L3 segment
1610 GB
128
57.6M (3,230 GB)
650 GB
≈ 270 GB (in-flight / node)
0 DRAM → L3 segment
3230 GB
NVIDIA

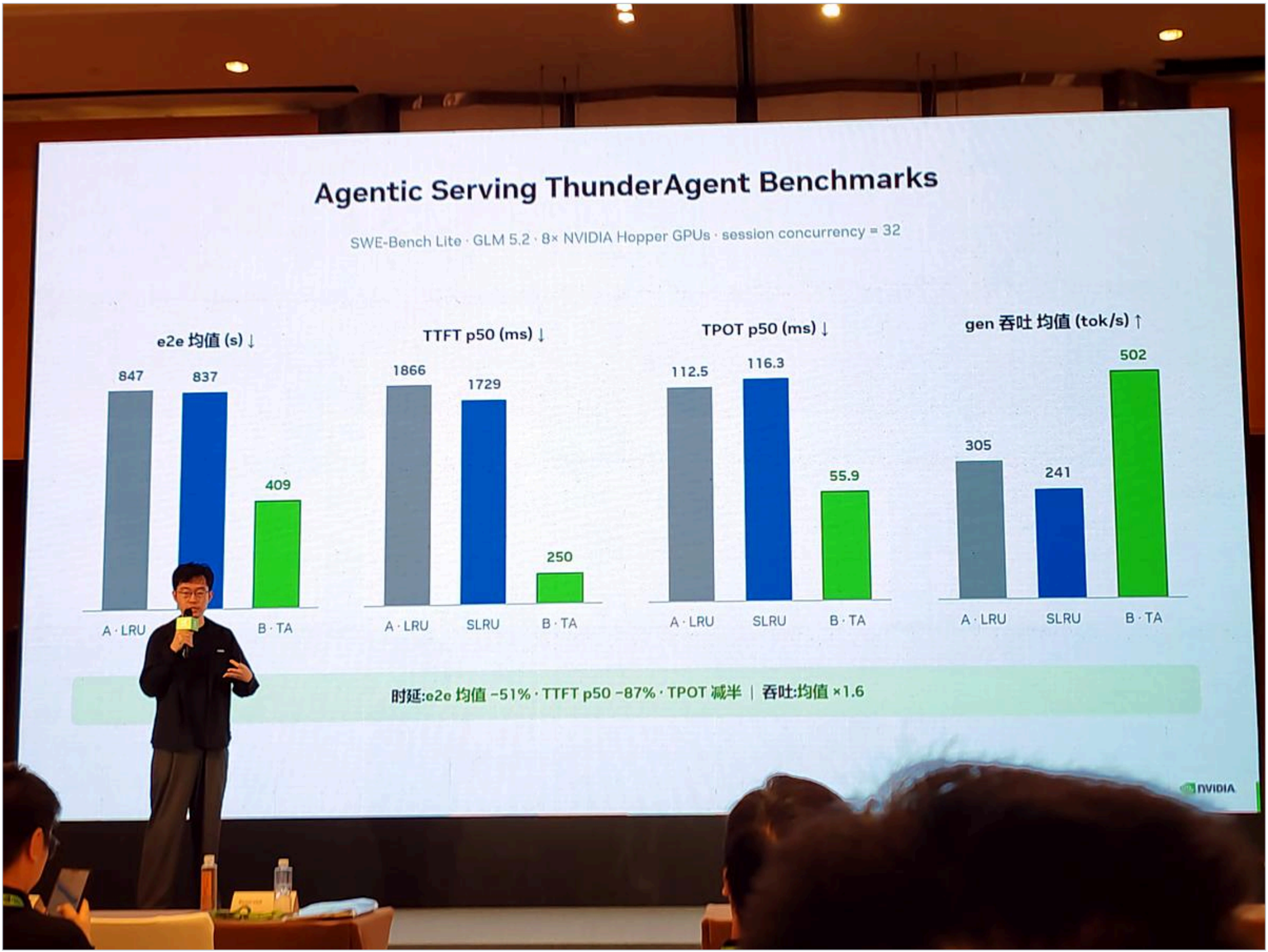

Goal 2: decode concurrency must keep up
GLM-5.2 · same AgentX load · 4 nodes / 32 GPUs · x-axis = set session concurrency (lanes) · in-flight counts requests, incl. subagents
TP8 · KV replicated on all 8 GPUs
KV ×1
1.26M tokens / node
ceiling ≈ 33 requests in flight
DEP (dp8 + ep8) · no replication
KV #1
KV #2
KV #3
KV #4
KV #5
KV #6
KV #7
KV #8
7.6M tokens / node (6×)
scales to 100+ sessions
Output throughput (tok/s)
900
600
300
0
754
753
615
596
430
101
c8
c48
c64
c96
c128
session concurrency (lanes)
DEP (dp8 + ep8)
TP8
Requests in flight vs set session concurrency
160
120
80
40
0
requests in flight
158
103
65
38
25
24
11
5
33
c8
c16
c32
c48
c64
c96
c128
set session concurrency (lanes)
DEP · requests in flight
TP8 · requests in flight
set sessions (1:1)
TP8 ceiling ≈ 33
NVIDIA

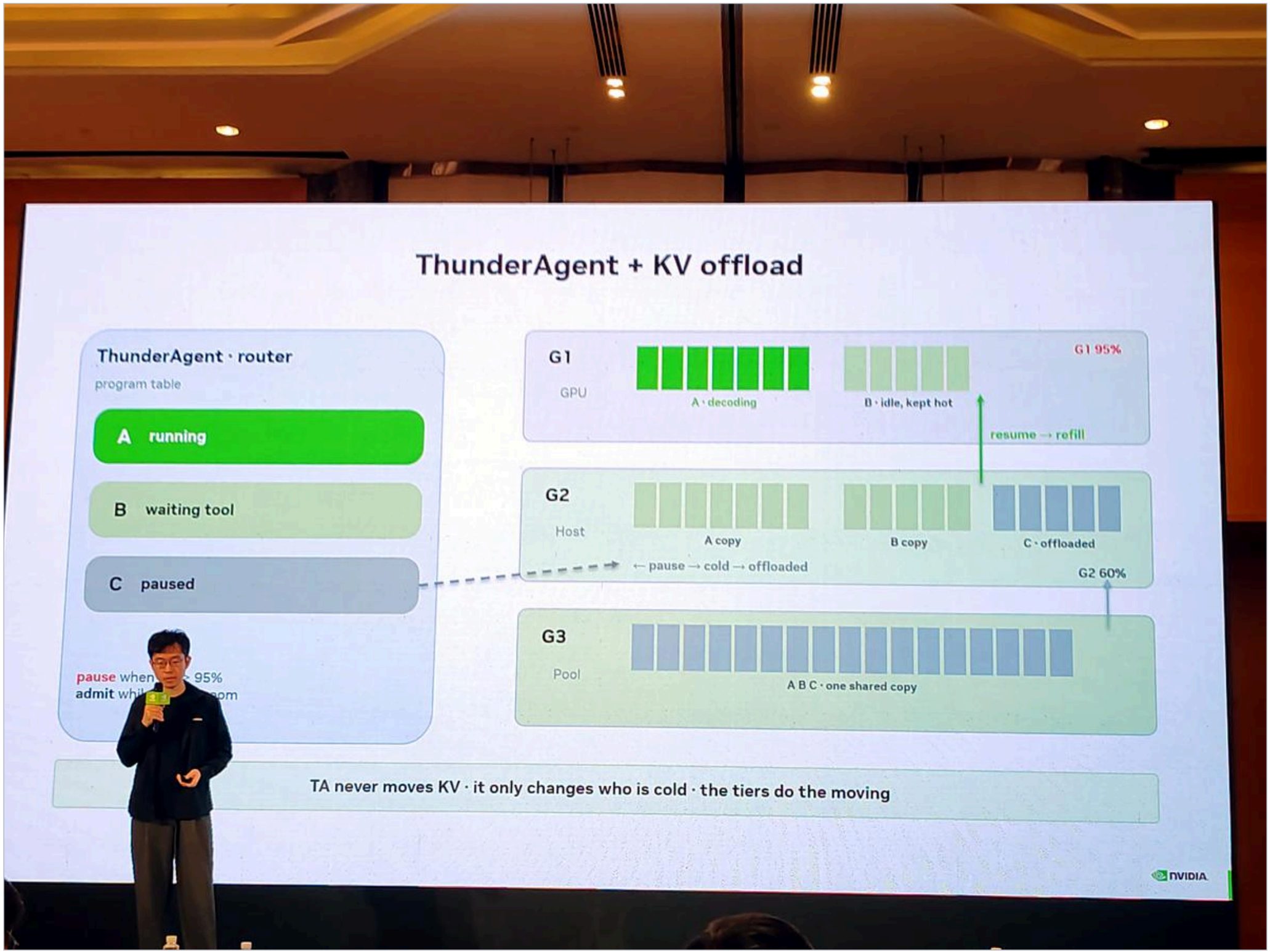

Hard affinity mode DEP 下的负载均衡问题
AgentX · DEP dp8+ep8 · cc 64 · hard session affinity · 1 hour · 4 node 32 dp ranks
Device KV occupancy of the 32 dp ranks, one cell per minute (red = ⩾90%, full)
node 1
node 2
node 3
Minutes at ⩾90% (of 61)
24
17
1
20
4
13
16
20
30
40
50
60
Minutes since the start of the measurement window (c64, 3600 s)
0%
30%
60%
90% full
100%
KV aware routing for first round, then session affinity hard mode
NVIDIA

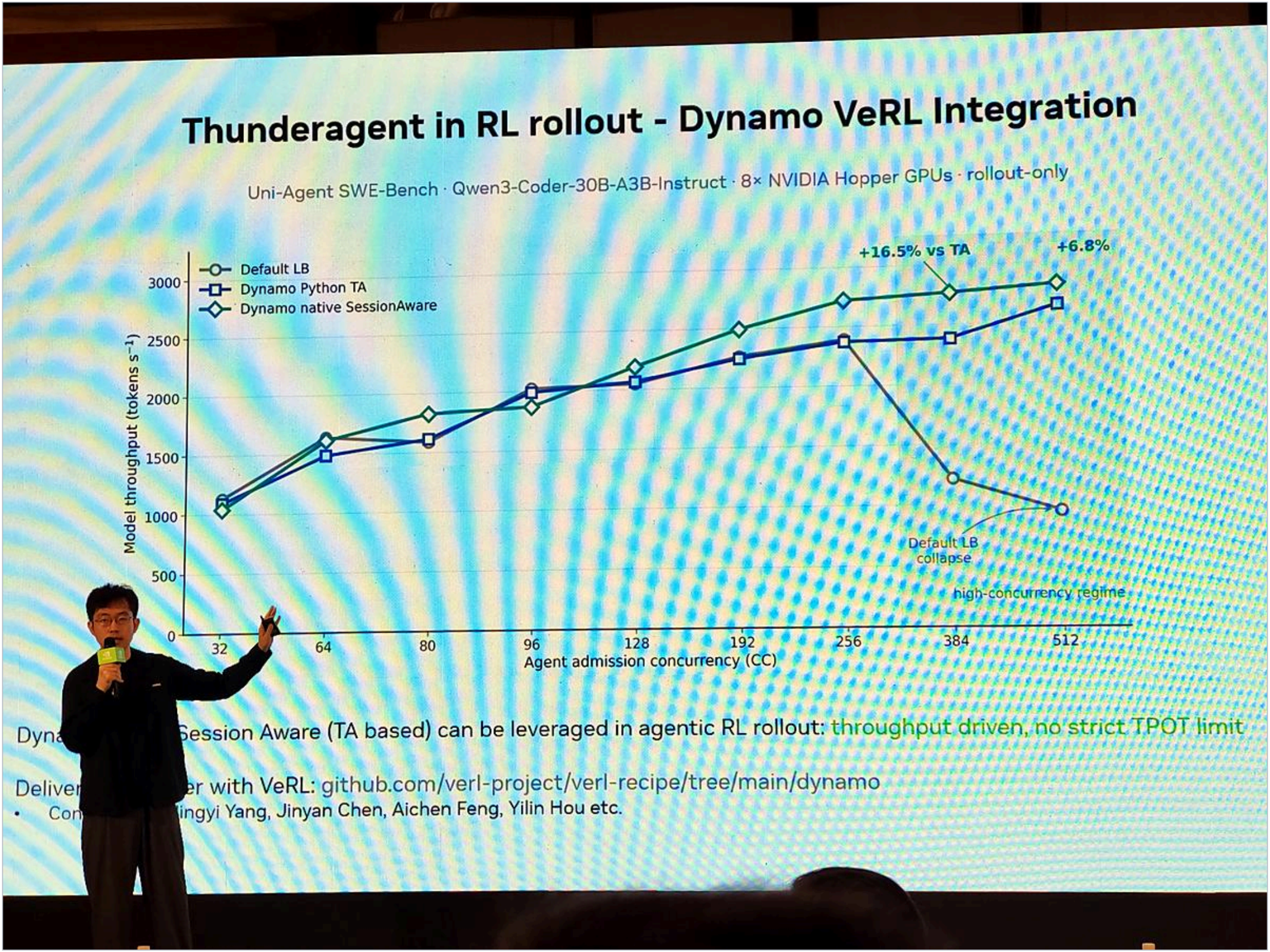

Goal3: Agentic Routing Strategy
KV-aware Router
每轮独立打分,看全局
看:缓存前缀块 + in-flight负载
Hard Affinity
首轮定终身,不看负载
看:X-Dynamo-Session-ID
Soft Affinity
默认留原地,过载reroute + KVCR
看:绑定当建议 + 过载阈值
Dynamo KV Router
① route this turn: rank 1 full → rank 6
② hint: prefix is in rank 1 host DRAM
GPU KV
1 2 3 4 5 6 7 8
session soft → rank 1
bound
full
④ deliver → GPU · prefill only the new tail
free
KVCR host DRAM
prefix
copy
③ NIXL peer-to-peer pull
Dynamo KVCR
rank full → route to free rank
1 2 3 4 5 6 7 8
on soft → rank 1
bound
full
free
KV
G3 pool · one copy
NVIDIA


Goal 4 – Clever Session Gateway - Thunder Agent
Program 状态机
REASONING
tool call
结果返回
ACTING 等 tool
标记,step 未暂停
pause
KV 已释放
resume
全局控制回路 每 Δt = 5s tick:resume → pause
KV util
97%
0.95 pause 阈值
0.85 resume 上限
0.80 pause target
55%
Worker 1
Worker 2
全局等待队列(paused)
Pause:最小 ACTING 优先
Resume:REASONING 优先,装剩余最大 worker
ACTING 计入 util 时权重 × 2^(−t),等 tool 越久越先让位
NVIDIA


ThunderAgent · how it decides
one tick every 5 s · util = Σ c of active programs ÷ GPU KV pool
util
admit · resume
pause
0.80 target
0.95 pause
0.85 resume ceiling
Pause order S = 1/c + 1{ACTING} · ACTING weight × 2^(−t)
P2 ACTING c = 40K 1st
P3 ACTING c = 120K 2nd
P4 REASONING c = 30K 3rd
P1 RE ING c = 180K last
lose · then smallest context → cheapest to recompute
State machine per program, transitions only at tool boundaries
REASONING
tool call
tool returns
ACTING
resume
util < 0.85 · S = 1/c + 1{REASONING}
pause
util > 0.95
PAUSED
resume → worker with most free KV · wait > 1800 s → forced resume
NVIDIA


Agentic Serving ThunderAgent Benchmarks
SWE-Bench Lite · GLM 5.2 · 8× NVIDIA Hopper GPUs · session concurrency = 32
e2e 均值 (s) ↓
847
837
409
A · LRU
B · TA
TTFT p50 (ms) ↓
1866
1729
250
A · LRU
SLRU
B · TA
TPOT p50 (ms) ↓
112.5
116.3
55.9
A · LRU
SLRU
B · TA
gen 吞吐 均值 (tok/s) ↑
305
241
502
A · LRU
SLRU
B · TA
时延:e2e 均值 −51% · TTFT p50 −87% · TPOT 减半 | 吞吐:均值 ×1.6
NVIDIA


ThunderAgent + KV offload
ThunderAgent · router
program table
A running
B waiting tool
C paused
pause when
admit whil
95%
om
G1
GPU
A · decoding
B · idle, kept hot
G1 95%
resume → refill
G2
Host
A copy
B copy
C · offloaded
← pause → cold → offloaded
G2 60%
G3
Pool
A B C · one shared copy
TA never moves KV · it only changes who is cold · the tiers do the moving
NVIDIA


Thunderagent in RL rollout - Dynamo VeRL Integration
Uni-Agent SWE-Bench · Qwen3-Coder-30B-A3B-Instruct · 8× NVIDIA Hopper GPUs · rollout-only
Default LB
Dynamo Python TA
Dynamo native SessionAware
+16.5% vs TA
+6.8%
Model throughput (tokens s⁻¹)
0
500
1000
1500
2000
2500
3000
32
64
80
96
128
192
256
384
512
Agent admission concurrency (CC)
Default LB collapse
high-concurrency regime
Session Aware (TA based) can be leveraged in agentic RL rollout: throughput driven, no strict TPOT limit
er with VeRL: github.com/verl-project/verl-recipe/tree/main/dynamo
ingyi Yang, Jinyan Chen, Aichen Feng, Yilin Hou etc.


Assets & Future Work
Assets
Docs
Flash Indexer — Dynamo Digest
docs.nvidia.com/dynamo/digest/flash-indexer
ThunderAgent Program Scheduler — User Guide
docs.nvidia.com/dynamo/user-guides/agents/thunder-agent-program-scheduler
PRs · ThunderAgent & SessionAware
#11447 feat(router): ThunderAgent admission strategy
#11787 refactor(router): policy-class admission
#11616 feat(admission): session-aware admission policy
github.com/ai-dynamo/dynamo/pull/11447 · /11787 · /11616
gin — agent-plugins
com/ai-dynamo/agent-plugins/tree/main/pi-plugin
ecipe(verl-recipe)
Ongoing Works
· Simulation for Agentic Infra
AIsimulate, tuning engine, scheduling and kv management before deploy
· Enhance program aware routing
More advanced admission control:Thunderagent v2, deal with moderate cc
Soft affinity mode + KVCR
· Programmatic KV block hints (SGLang RFC #27574) etc.
Dynamo router 下发 Share / Prefetch / Demote / Pin / Retain soft hint
NVIDIA